

Lecture #11 : Advanced Topics in TCP

Dohyung Kim

Today...

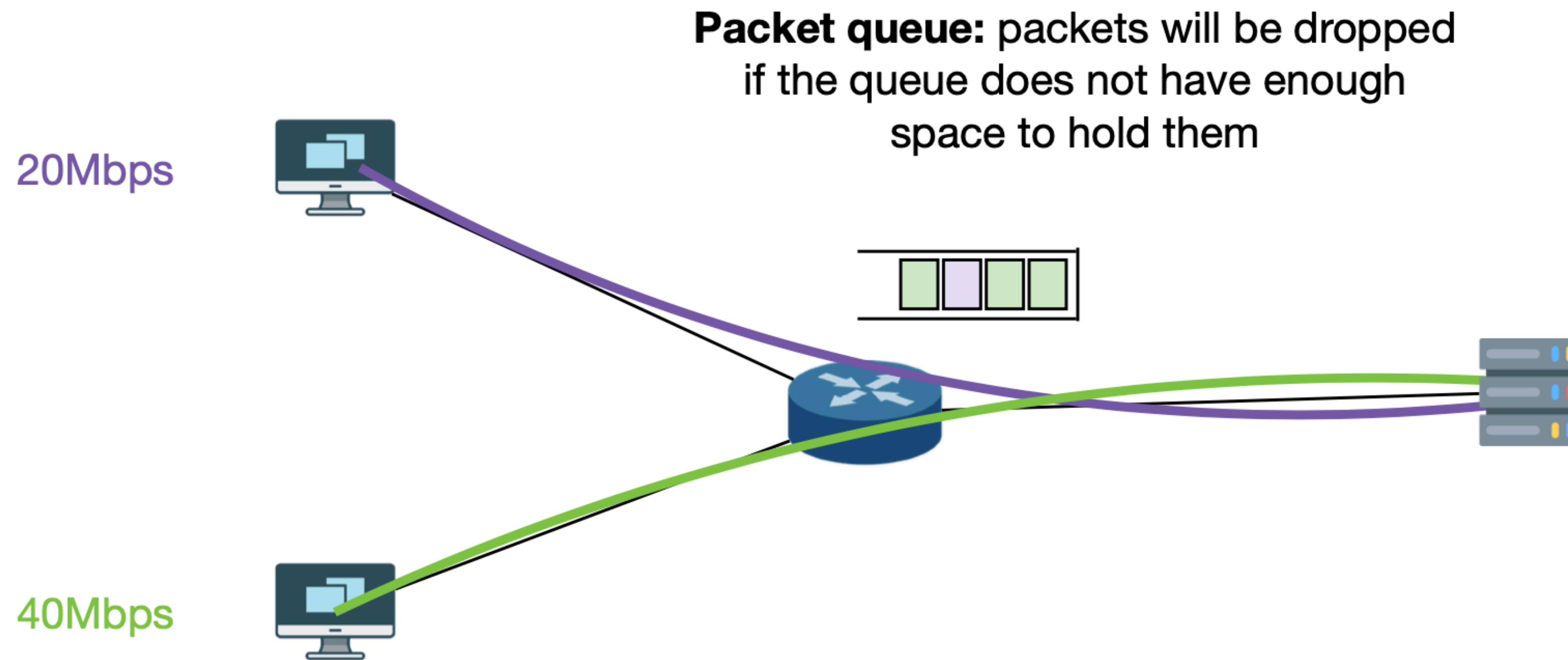
- Network transport (congestion control)

Advancements in network transport design

- TCP congestion control
- Multi-path TCP
- QUIC & HTTP 3.0

Congestion control

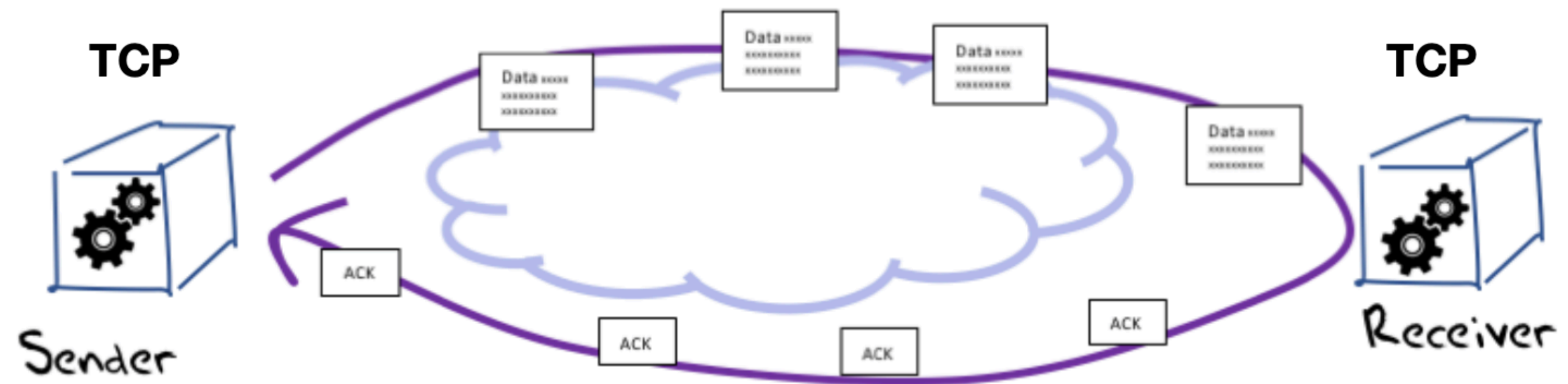
- Congestion control aims to determine the **rate to send data** on a connection



- * Sender does not overrun the network capability
- * Network is efficiently utilized
- * Should be implement in a distributed manner (no centralized coordinator)

Transport control protocol (TCP)

- Send data across the network as fast as possible, but not faster
- End-to-end design
 - The end hosts control the sending rate, based on ACKs
 - Does not rely on functionalities provided by the network
 - May leverage in-network mechanisms like ECN for performance improvement



How did TCP work before 1988?

- Congestion collapse (breakdowns in performance) noted on NSFNet in 1986

⇒ "40Kbps link operates at as low as 32bps"

1.5. Operation

As noted above, the primary purpose of the TCP is to provide reliable, securable logical circuit or connection service between pairs of processes. To provide this service on top of a less reliable internet communication system requires facilities in the following areas:

- Basic Data Transfer
- Reliability
- Flow Control
- Multiplexing
- Connections
- Precedence and Security

RFC 793

What do we really mean by congestion?

- Water pipe analogue
 - The path of a TCP connection as a water pipe that has space to hold water
 - The available space of the water pipe changes depending on the other connections
 - If we pump too fast, the water pipe will overflow; too slow, not efficient
 - TCP tries to keep the water pipe just full
- Less about congestion control, more about resource allocation
 - How to wisely allocate space for all connections
- “**Packet conservation**” is key to this idea

We do not need congestion control if there is no congestion which is also created by us!



Packet conservation

Congestion Avoidance and Control*

Van Jacobson[†]
Lawrence Berkeley Laboratory

Michael J. Karels[‡]
University of California at Berkeley

November, 1988

Introduction

Computer networks have experienced an explosive growth over the past few years and with that growth have come severe congestion problems. For example, it is now common to see internet gateways drop 10% of the incoming packets because of local buffer overflows. Our investigation of some of these problems has shown that much of the cause lies in transport protocol implementations (*not* in the protocols themselves): The ‘obvious’ ways to implement a window-based transport protocol can result in exactly the wrong behavior

ACM SIGCOMM 1988

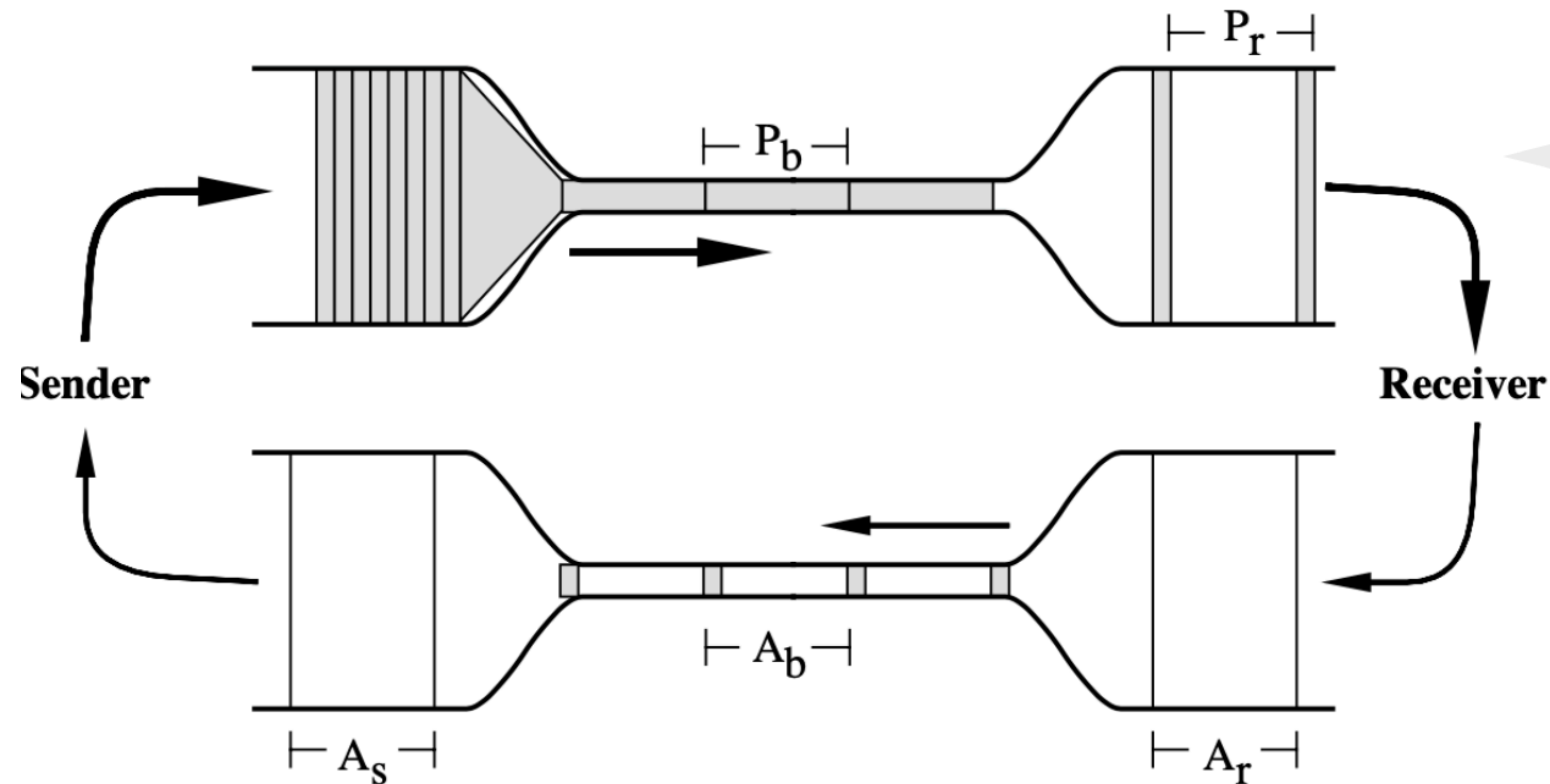
Packet Conservation Principle

For a connection in equilibrium, i.e., running stably with a full window of data in transit, a new packet should not be put into the network until an old packet leaves.

How does TCP achieve this?

Self-clocking

- Assume the following connection is in an equilibrium



“So, if packets after the first burst are sent only in response to an ACK, the sender’s packet spacing will exactly match the packet time on the slowest link in the path.”

$$A_s = A_b = A_r = P_b = P_r$$

* Observations

- New packets are naturally sent
 1. when a packet leaves
 2. at the bottleneck rate

Two more challenges to solve

How to quickly reach equilibrium in the first place?



TCP idea: slow-start

How to adapt to the available space in the path accordingly?

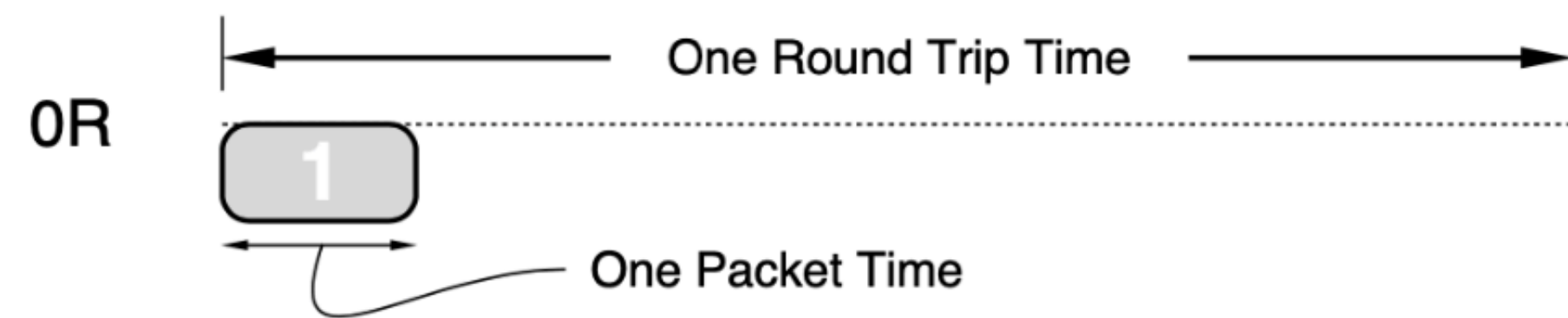


TCP idea: additive increase multiplicative decrease (AIMD)

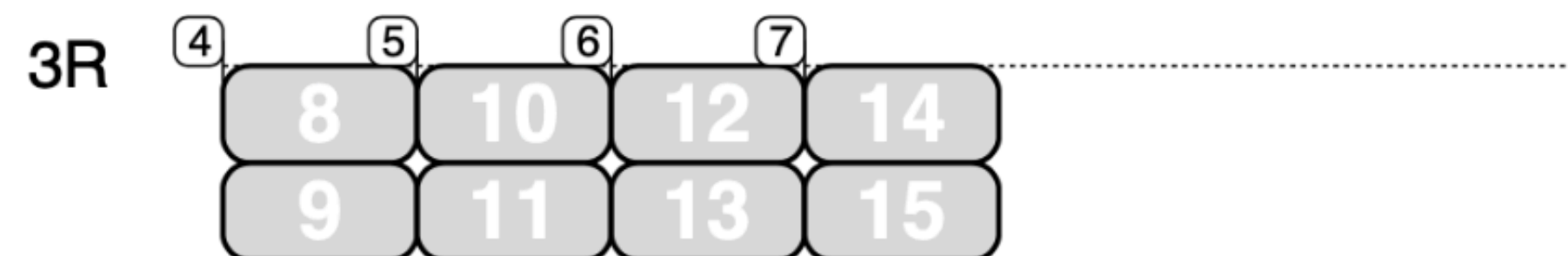
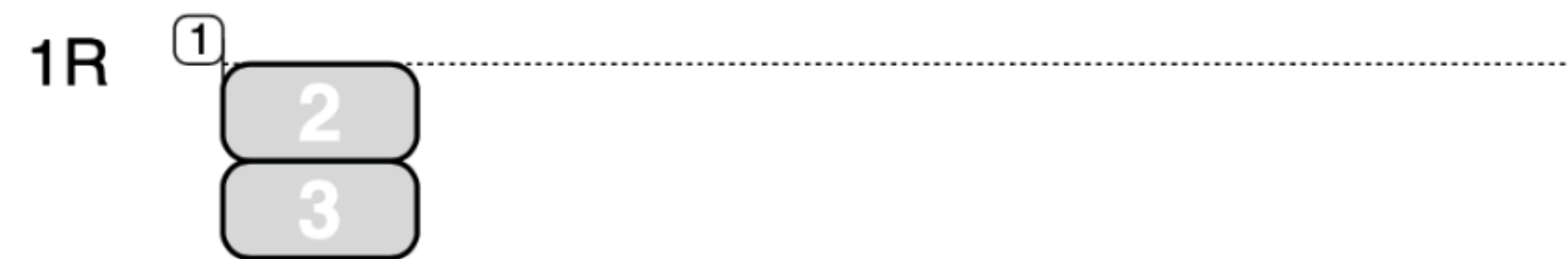
These two ideas are overloaded on the TCP sliding window mechanism for congestion avoidance

TCP slow-start

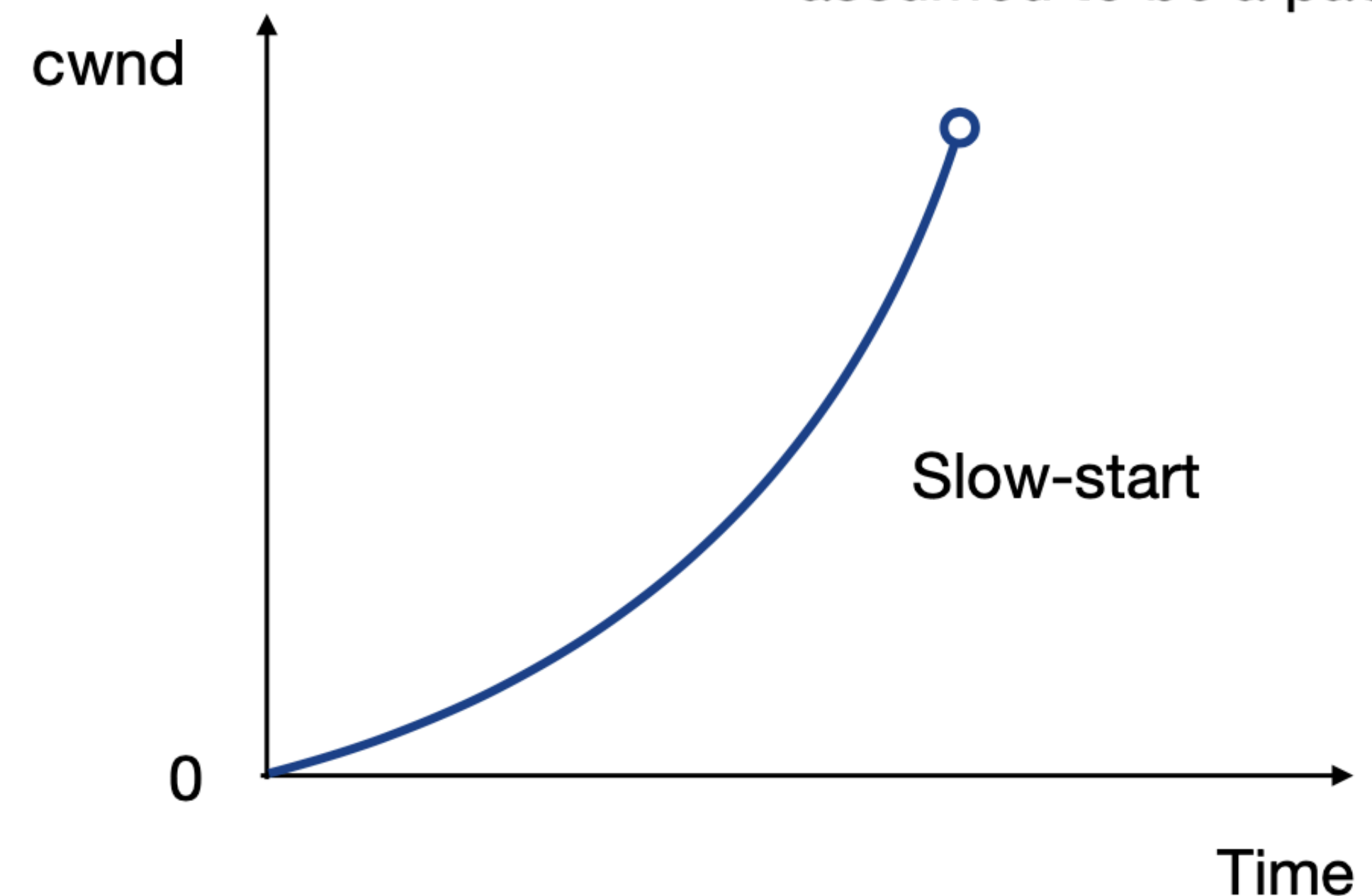
- Upon receiving an ACK, increase the congestion window (cwnd) by 1



Window size = $\min(\text{cwnd}, \text{rwnd})$

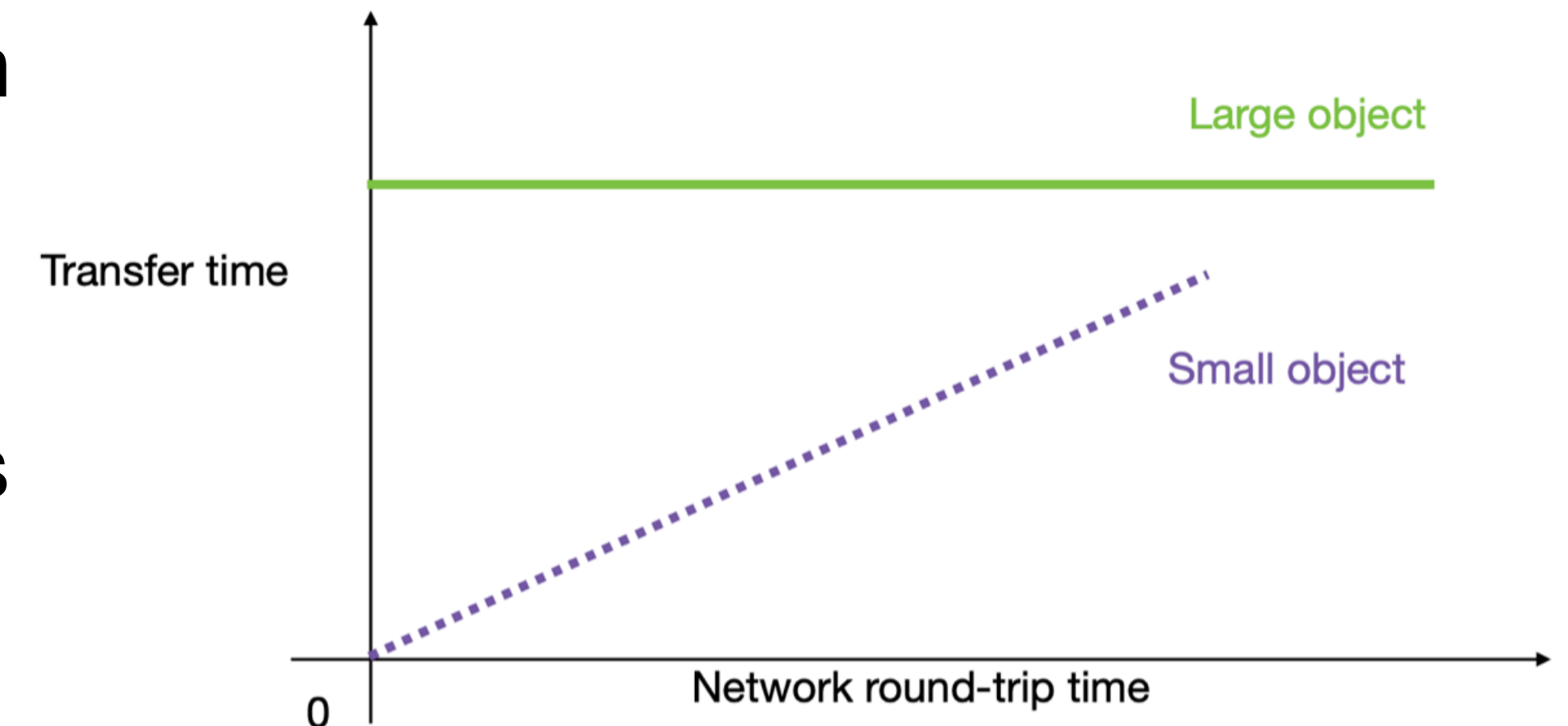


Timeout or 3 duplicate ACKs:
assumed to be a packet drop



Recall the data transfer time problem

- Assume that a network link with 10Gbps bandwidth and 1 ms round-trip-time, how long does it take to transfer 100KB of data on the link?
 - Slow-start before reaching the maximum bandwidth
 - $100\text{KB} = 1460\text{B} \times (0+1+2+4+8+16+32+6)$
 - In total 7 round-trips in the slow-start phase $\rightarrow 7\text{ms}$

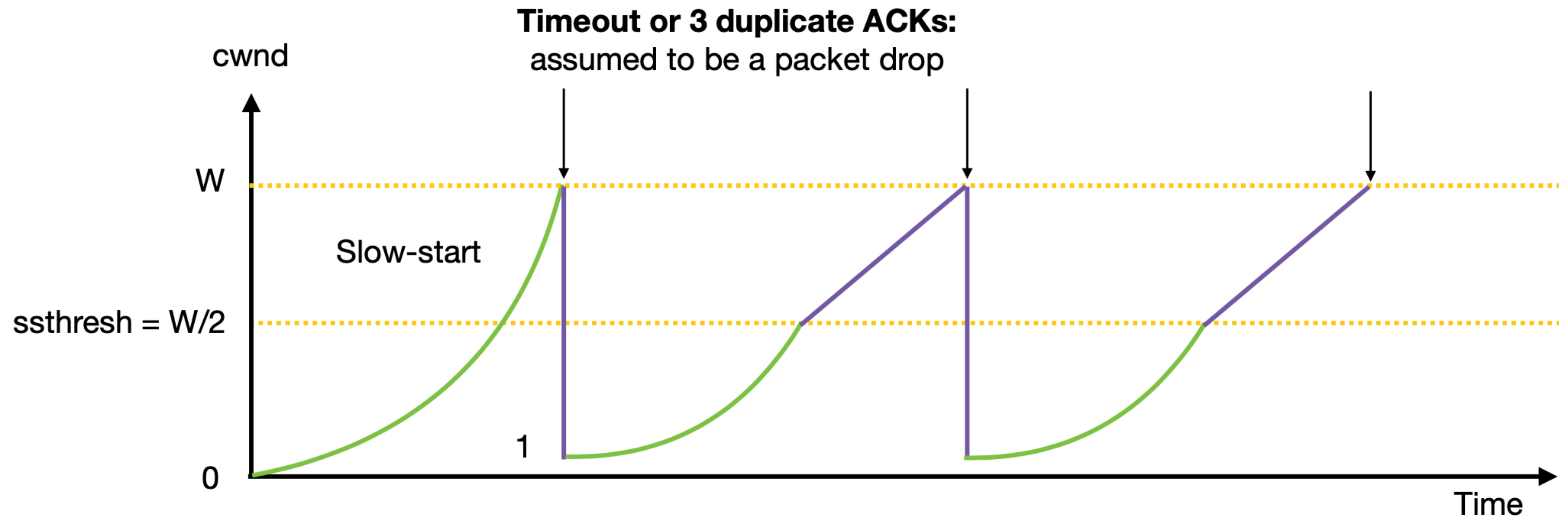


- More generally, how long does it take to reach a window size of W in slot-start?
 - $\text{RTT} \times \log_2 W$
 - Exponential growth $\rightarrow$ slow-start is not slow at all

TCP AIMD

- TCP Tahoe

What is a duplicate ACK?

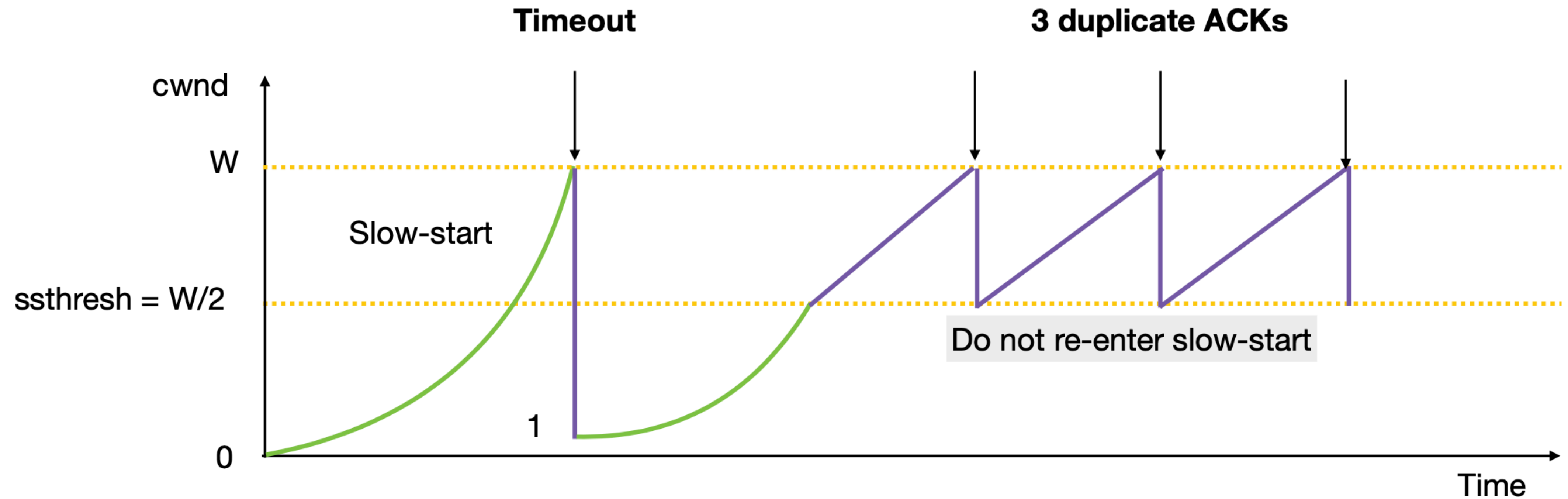


```
if cwnd < ssthresh:    //slow-start
    cwnd += 1
else:                  // AIMD
    cwnd += 1 / cwnd
```

TCP AIMD

- TCP Reno

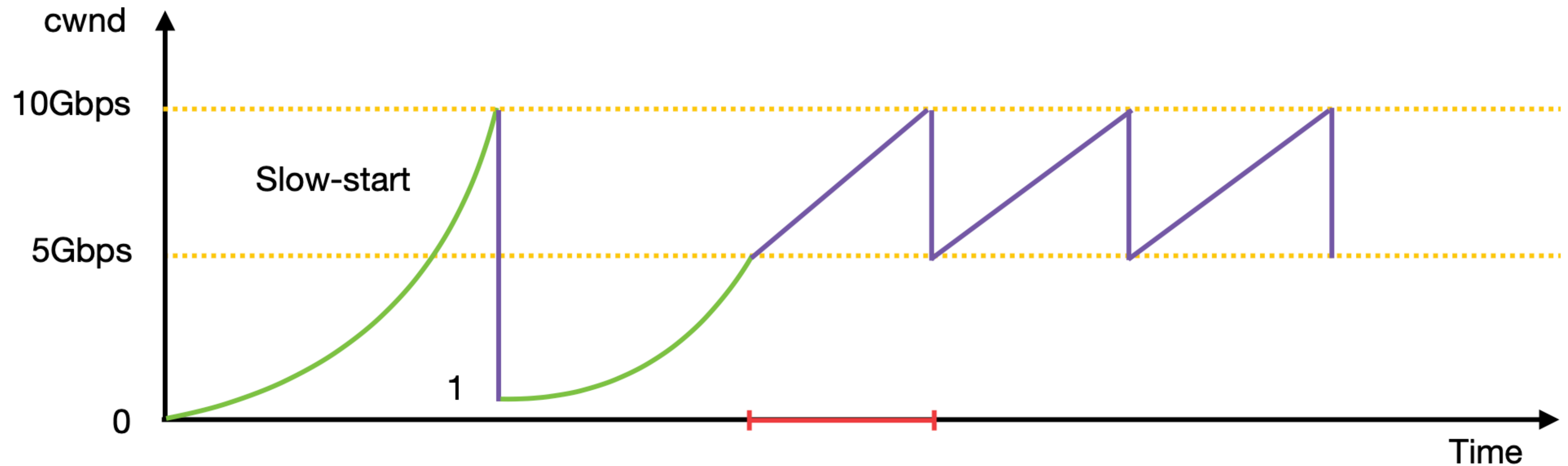
What is a duplicate ACK?



```
if cwnd < ssthresh:    //slow-start
    cwnd += 1
else:                  // AIMD
    cwnd += 1 / cwnd
```

TCP AIMD

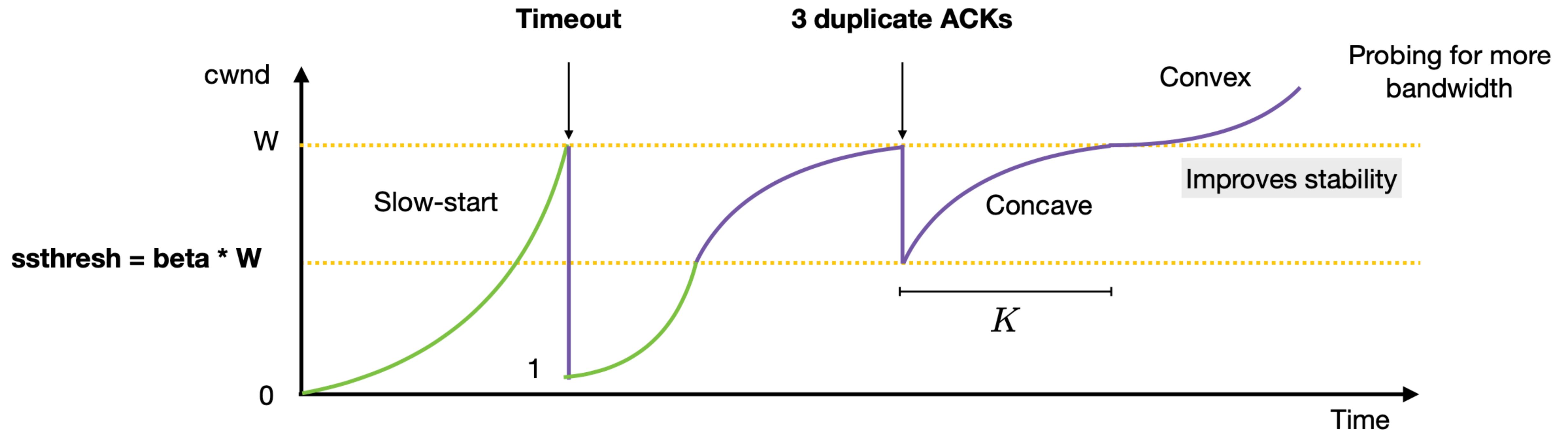
- TCP Reno problem



How long does it take to ramp up? The cwnd increases by 1460B every RTT $\rightarrow$ 428K RTTs needed. For a connection with RTT of 30ms, this is around 3.6 hours!

TCP AIMD

- TCP Cubic (in linux kernel since v.2.6.19)



Use a cubic function to adjust window size based on time, not RTT:

$$W(t) = C(t - K)^3 + W_{max}$$

Other TCP variants

- Many many versions of TCP have been proposed
 - Loss-based
 - Delay-based
 - ...

Variant ↕	Feedback ↕	Required changes ↕	Benefits ↕	Fairness ↕
(New) Reno	Loss	—	—	Delay
Vegas	Delay	Sender	Less loss	Proportional
High Speed	Loss	Sender	High bandwidth	
BIC	Loss	Sender	High bandwidth	
CUBIC	Loss	Sender	High bandwidth	
C2TCP ^{[9][10]}	Loss/Delay	Sender	Ultra-low latency and high bandwidth	
NATCP ^[11]	Multi-bit signal	Sender	Near Optimal Performance	
Elastic-TCP	Loss/Delay	Sender	High bandwidth/short & long-distance	
Agile-TCP	Loss	Sender	High bandwidth/short-distance	
H-TCP	Loss	Sender	High bandwidth	
FAST	Delay	Sender	High bandwidth	Proportional
Compound TCP	Loss/Delay	Sender	High bandwidth	Proportional
Westwood	Loss/Delay	Sender	L	
Jersey	Loss/Delay	Sender	L	
BBR ^[12]	Delay	Sender	BLVC, Bufferbloat	
CLAMP	Multi-bit signal	Receiver, Router	V	Max-min
TFRC	Loss	Sender, Receiver	No Retransmission	Minimum delay
XCP	Multi-bit signal	Sender, Receiver, Router	BLFC	Max-min
VCP	2-bit signal	Sender, Receiver, Router	BLF	Proportional
MaxNet	Multi-bit signal	Sender, Receiver, Router	BLFSC	Max-min
JetMax	Multi-bit signal	Sender, Receiver, Router	High bandwidth	Max-min
RED	Loss	Router	Reduced delay	
ECN	Single-bit signal	Sender, Receiver, Router	Reduced loss	

Check TCP variant on your system

- On Mac

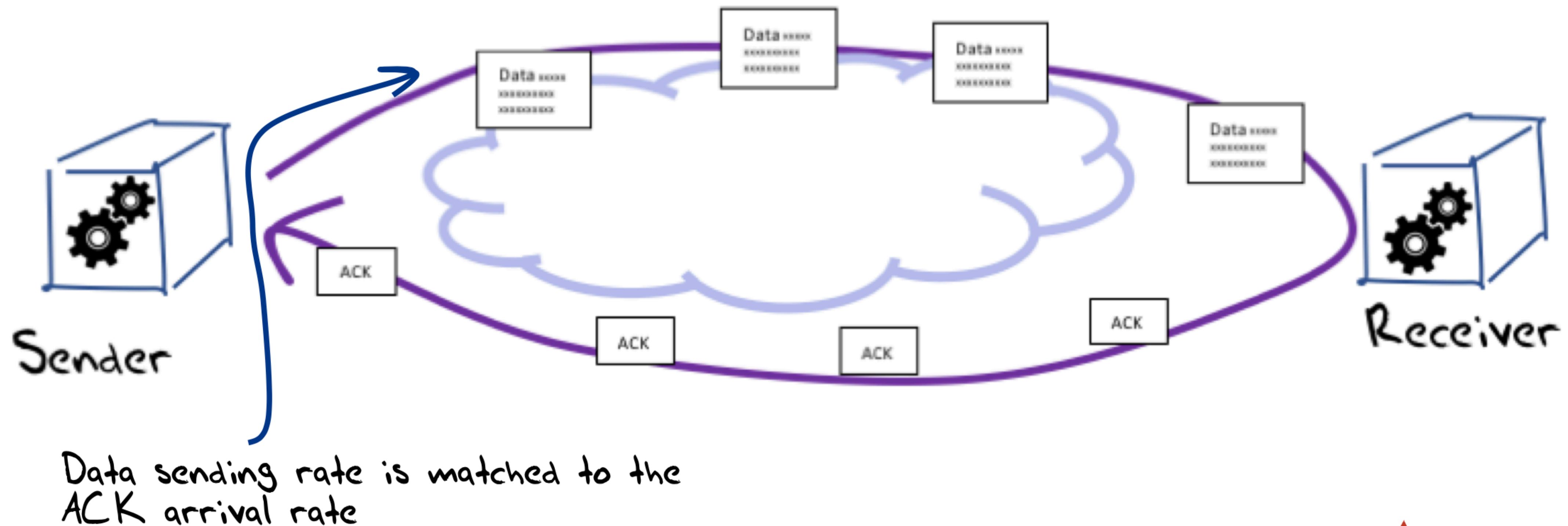
```
~ — -bash
[DohyungKim:~ trulife$ sysctl net | grep net.inet.tcp | egrep -i "cubic|reno|bbr"
net.inet.tcp.newreno_sockets: 0
net.inet.tcp.cubic_sockets: 65
net.inet.tcp.use_newreno: 0
net.inet.tcp.cubic_tcp_friendliness: 0
net.inet.tcp.cubic_fast_convergence: 0
net.inet.tcp.cubic_use_minrtt: 0
net.inet.tcp.cubic_minor_fixes: 1
net.inet.tcp.cubic_rfc_compliant: 1
DohyungKim:~ trulife$
```

- On Linux

```
[dhkim@icnserver:~$ cat /proc/sys/net/ipv4/tcp_congestion_control
cubic
dhkim@icnserver:~$
```

Packet loss is not a good indicator of congestion

- TCP is an ACK pacing protocol (recall self-clocking)



Packet loss is not a good indicator of congestion

- TCP is an ACK pacing protocol (recall self-clocking)
- ACK pacing protocols relate to a **past** network state, not necessarily the current network state
 - The ACK signal shows the rate of data that left the network at the receiver that occurred at $1/2$ RTT back in time
 - So if there is data loss, the ACK signal of that loss is already $1/2$ RTT old!
→ **TCP should react quickly to ‘bad’ news!**
- If there is no data loss, that is also old news
→ **TCP reacts too conservatively to ‘good’ news!**

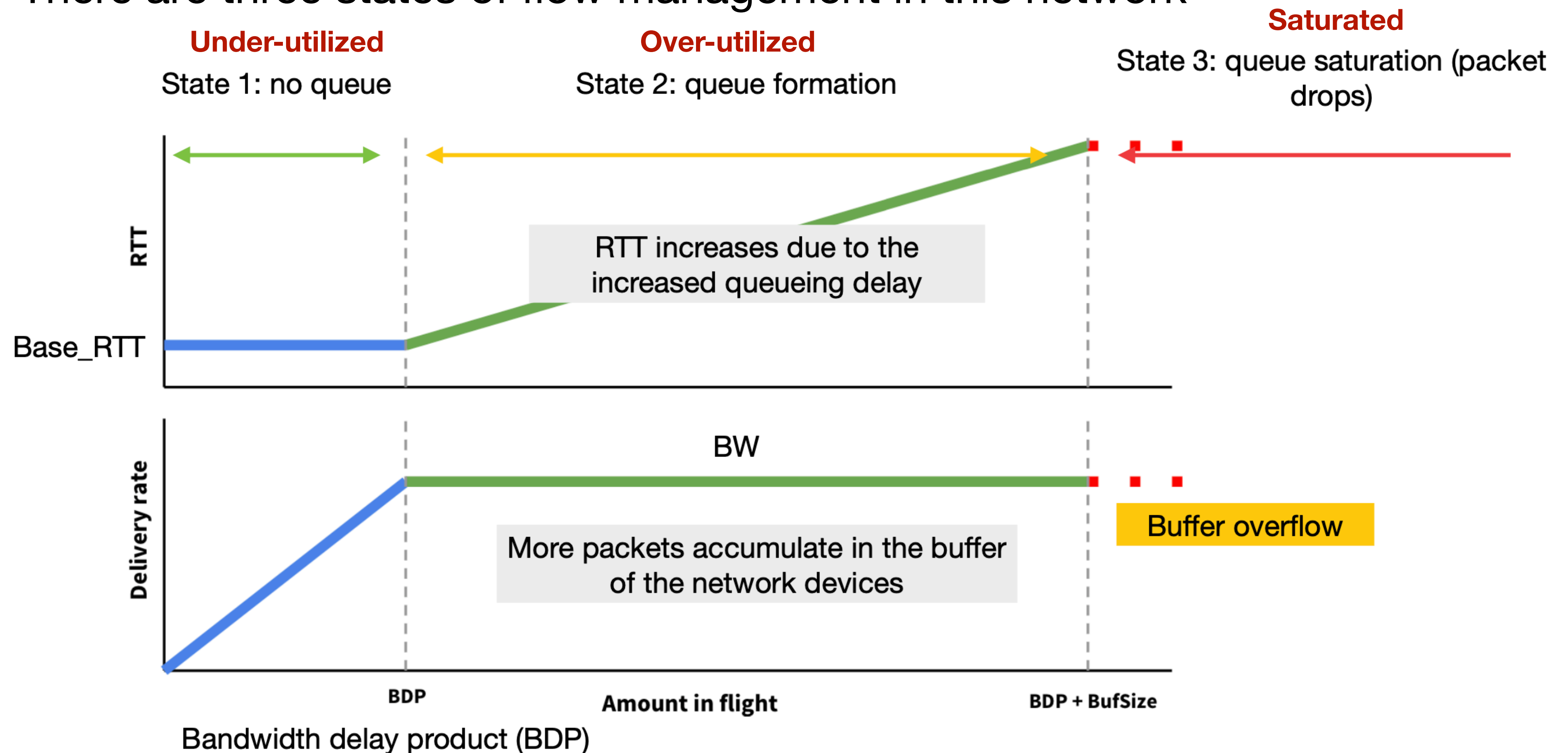
Packet loss is not a good indicator of congestion

- TCP is an ACK pacing protocol (recall self-clocking)
- ACK pacing protocols relate to a **past** network state, not necessarily the current network state

How can we do better?

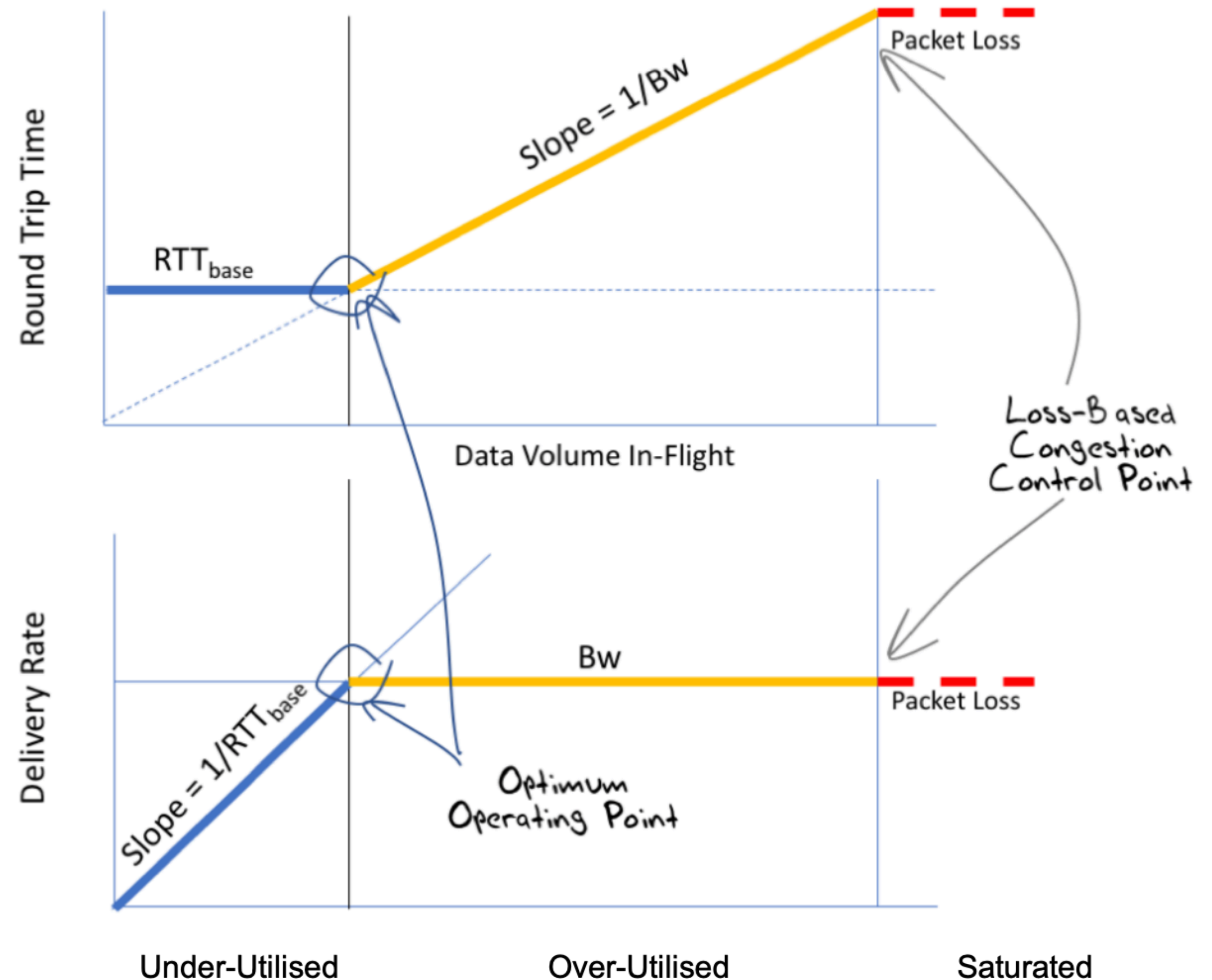
Flow management states

- There are three states of flow management in this network



Flow management states

- Loss-based control system probes upward to the saturation point, and back off to what they guess is the under-utilized state in order to let the queues drain
- But the **optimal operational point** for the flow is at the **state change from under to over-utilized**



How to detect the optimal operating point

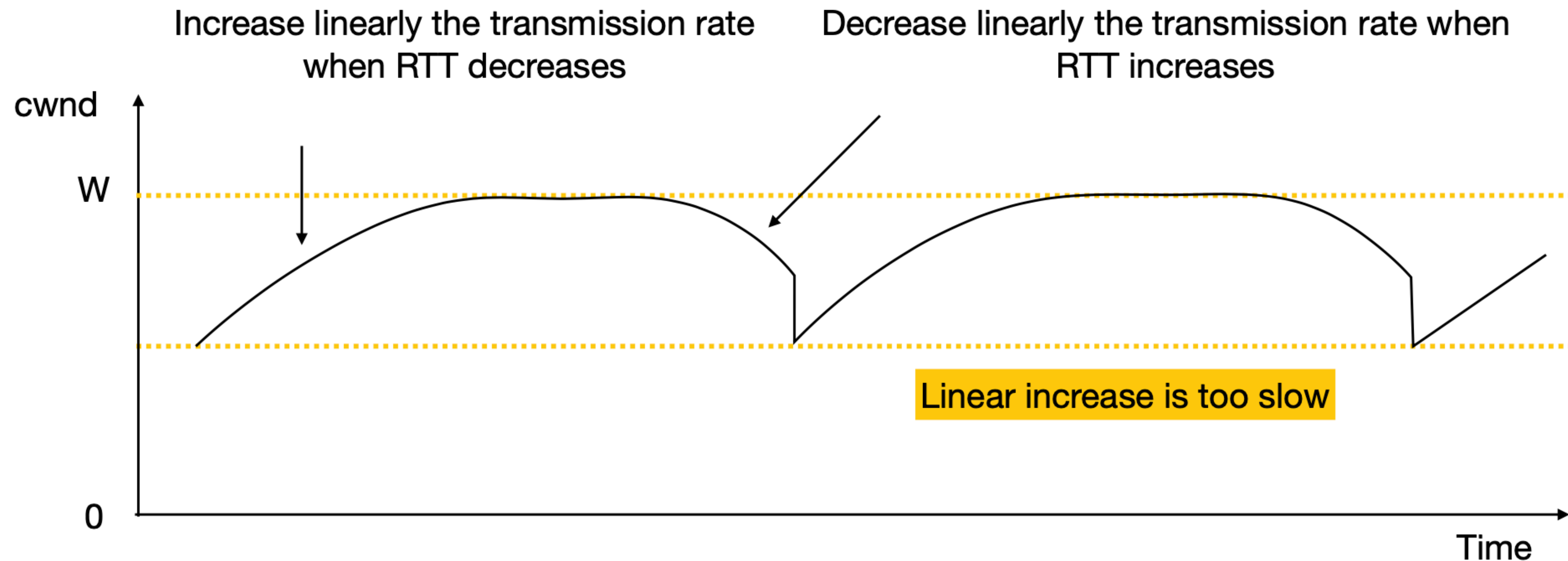
- Instead of shooting for packet losses, try to find the best operating point more explicitly

"By carefully measuring the round-trip time!"
- Bottleneck Bandwidth and Round-trip propagation time (BBR) was developed at Google in 2016
- Congestion-based congestion control



TCP Vegas

- Try to operate at the optimal point by adjusting sending rate based on measured RTT



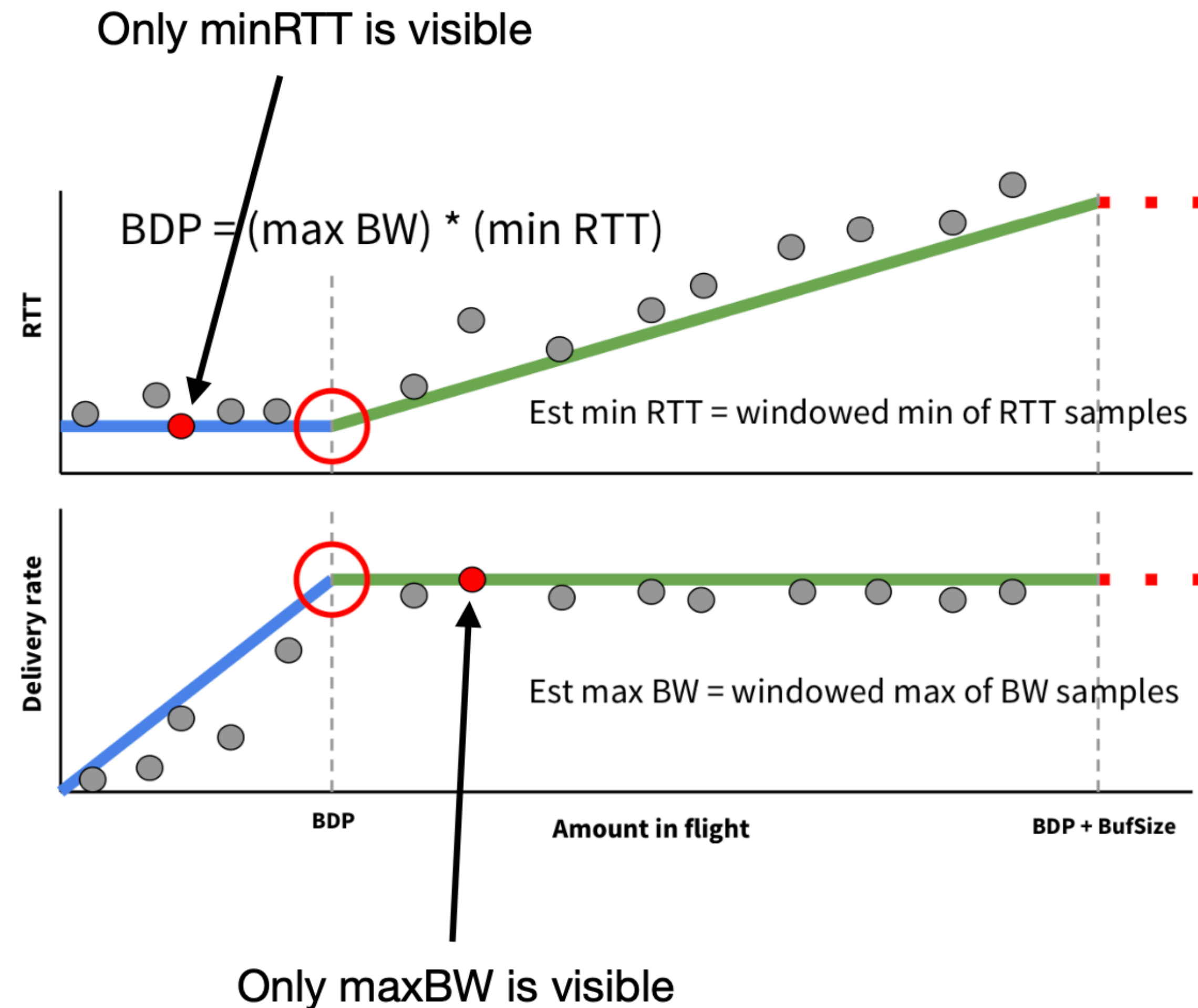
Not competitive to flows with loss-based congestion control algorithms, why?

BBR

- Probe the path capacity intermittently
- Probe the path capacity by increasing the sending rate for a short interval
 - If the RTT of the probe equals to the RTT of the previous state, then there is available path bandwidth that could be utilized
 - If the RTT of the probe rises, then the path is likely to be at the onset of queueing and no further path bandwidth is available
- Do not alter the path bandwidth estimate in response to packet loss
- Pace the sending packets to avoid the need for network buffer rate adaptation
- Maintain a long term stable estimate of path RTT and a shorter updated estimate of the bottleneck capacity of the path

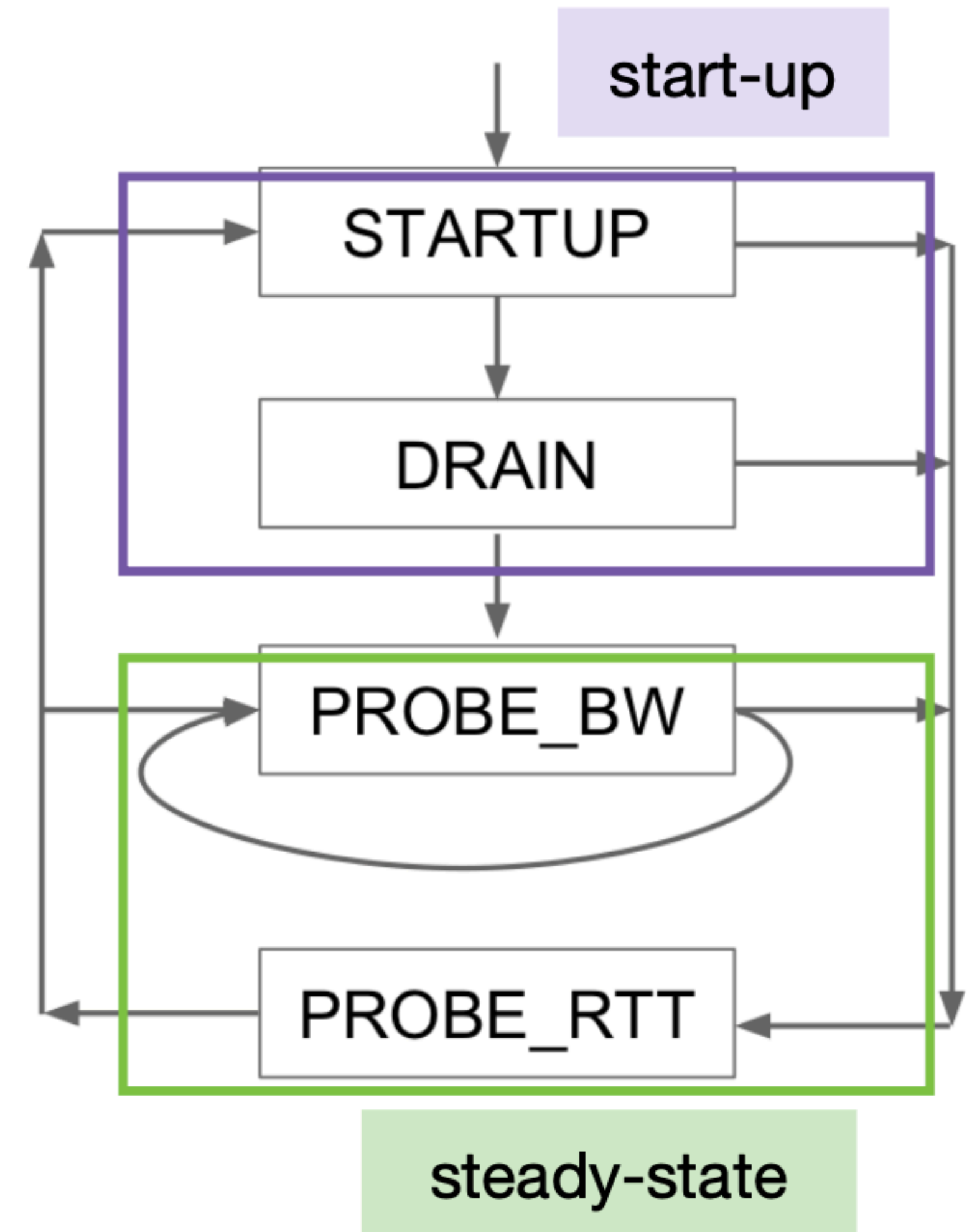
BBR

- Model the network
 - Estimate maxBW and MinRTT on every ACK
- Control sending rate based on the model
 - Probe both maxBW and minRTT, to feed the model samples
 - Pace near estimated BW, to reduce queues and losses
 - Change pace rate to keep in-flight near BDP (full pipe but small queue)



BBR state machine

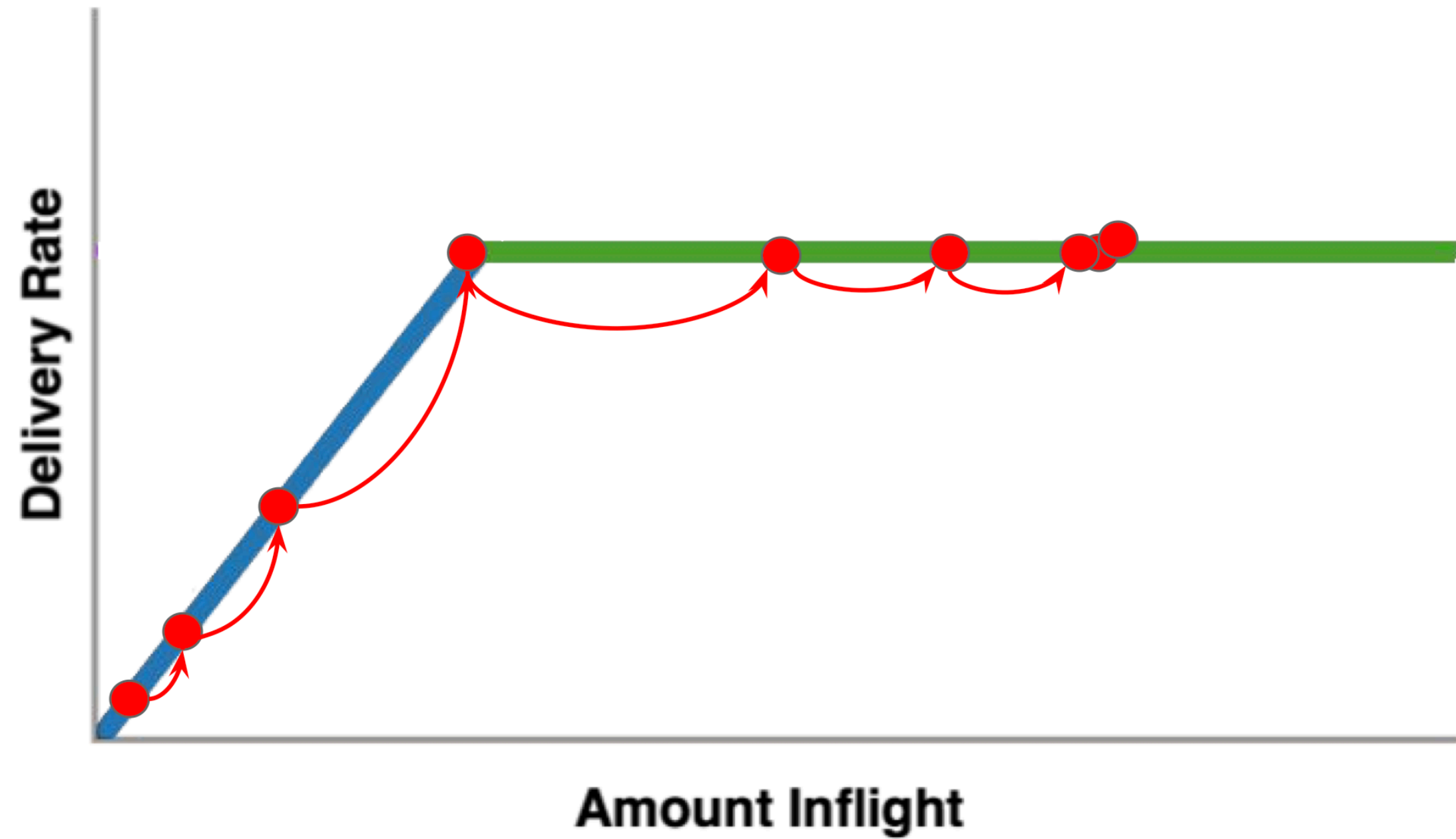
- STARTUP
 - Exponential BW search
- DRAIN
 - Drain the queue created during the STARTUP
- PROBE_BW
 - Explore maxBW, drain queue, cruise
- PROBE_RTT
 - Occasionally drain queue to refresh minRTT



BBR states

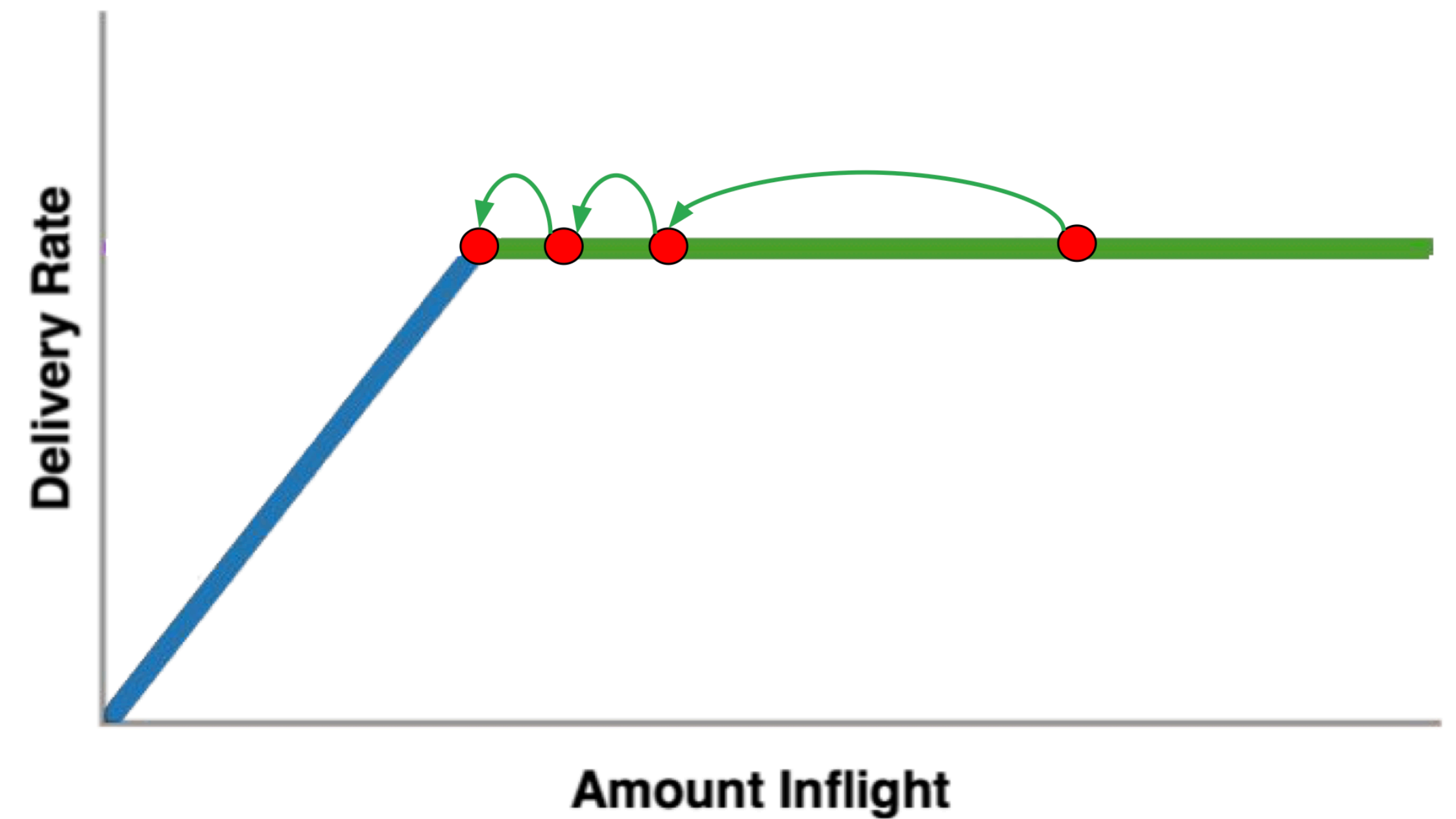
STARTUP

: While delivery rate is increasing



DRAIN

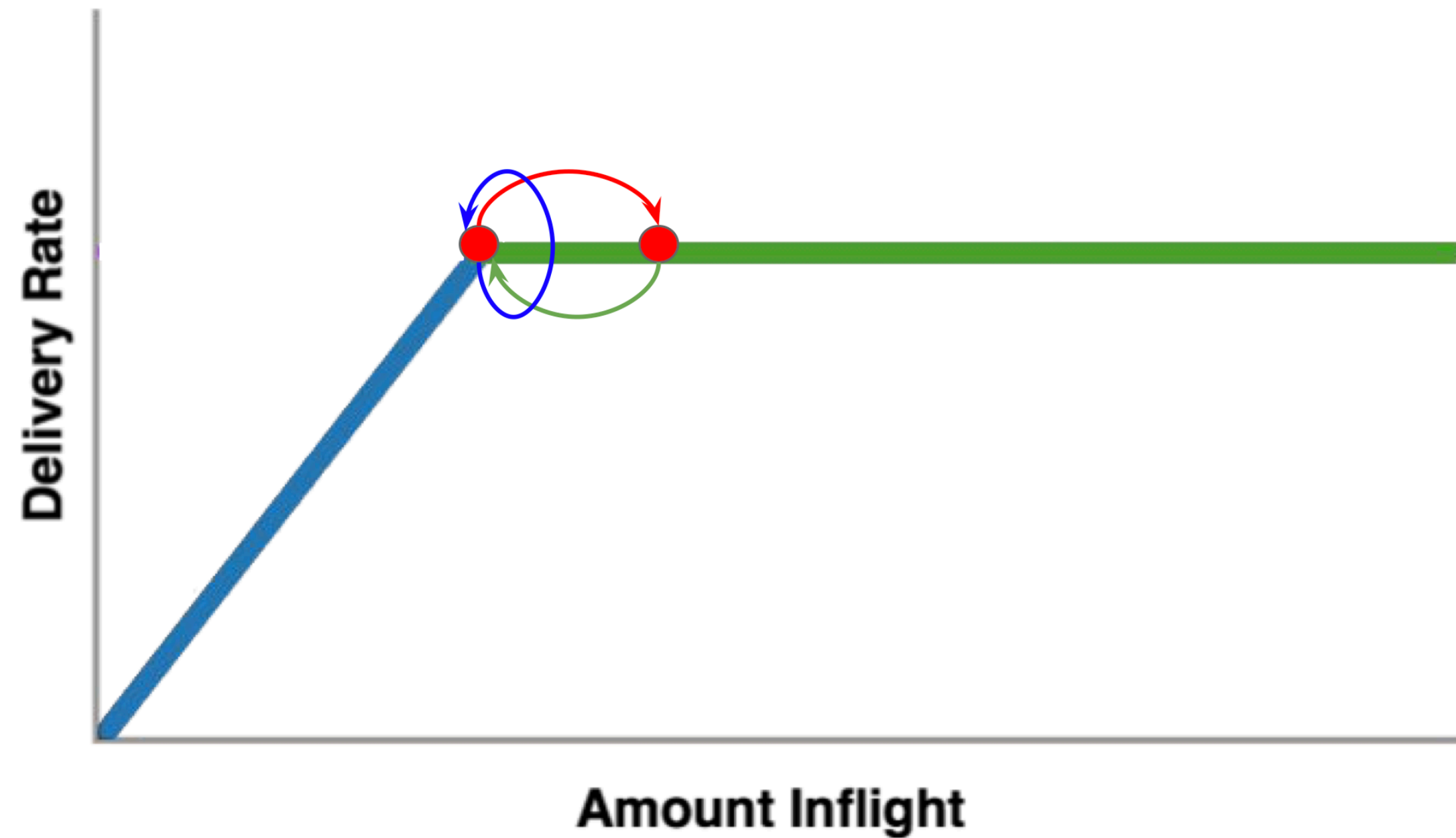
: While delivery rate is not decreasing



BBR states

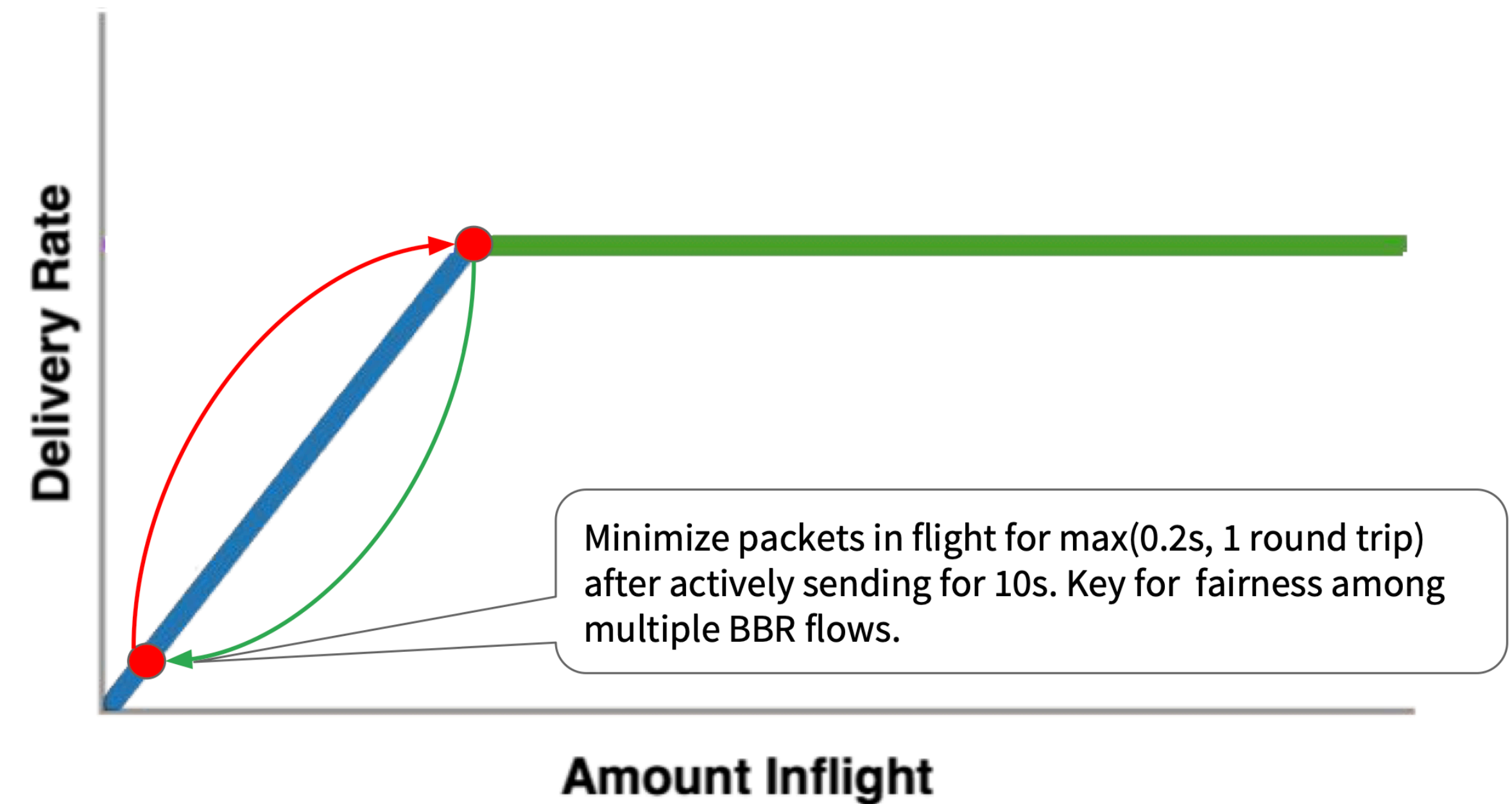
PROB_BW

: While delivery rate is increasing



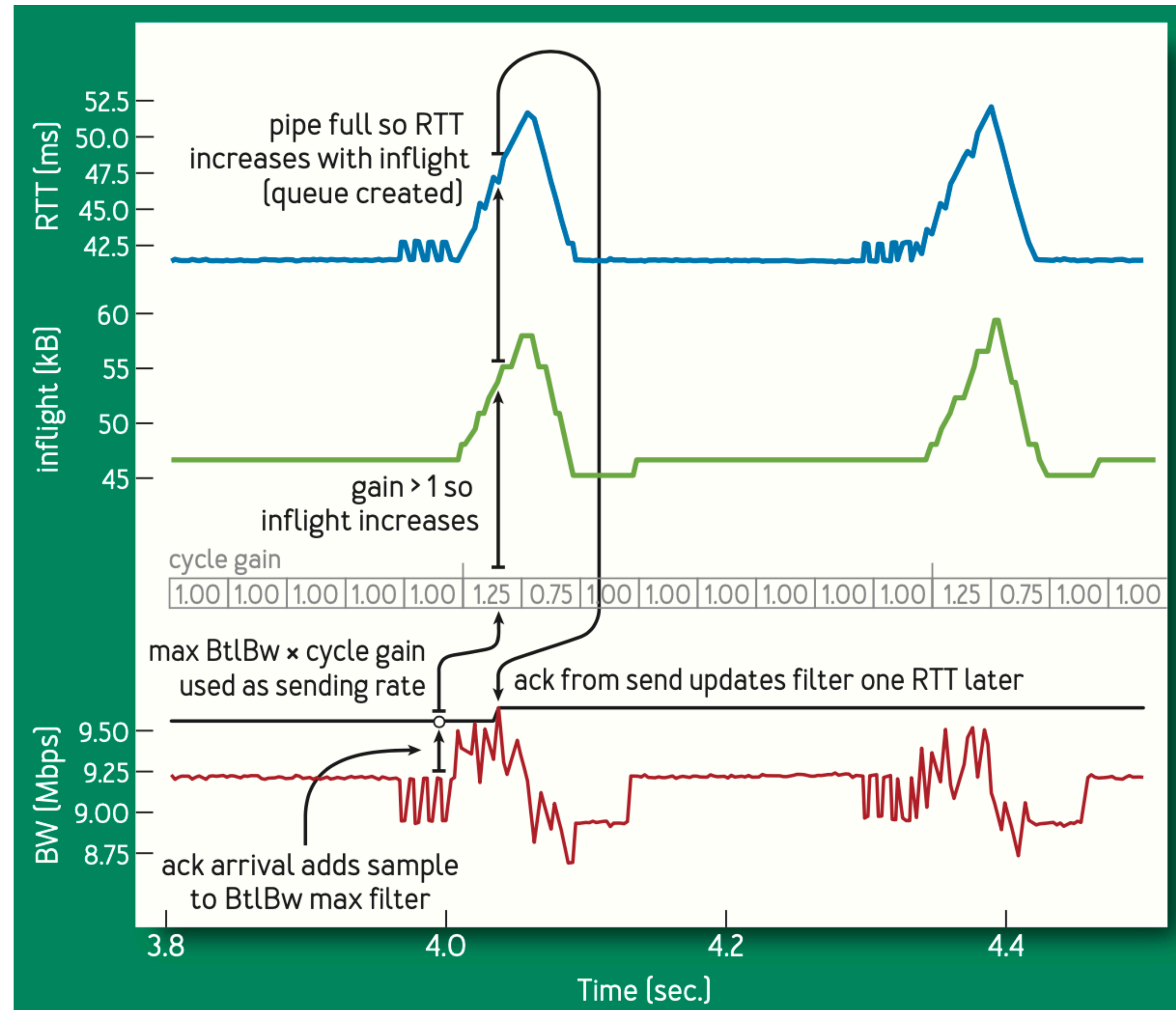
PROB_RTT

: sending 4 packet during one RTT temporarily

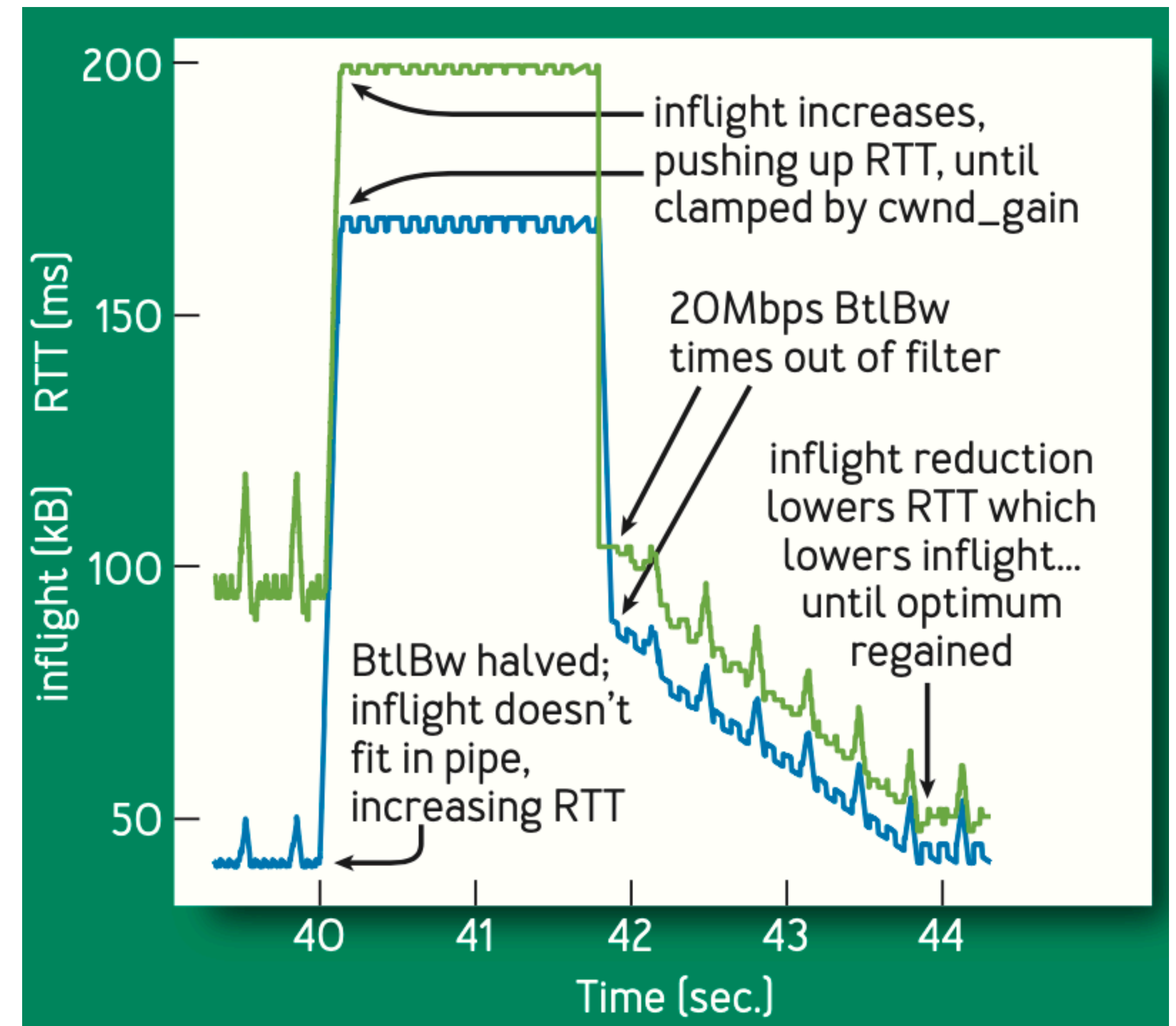
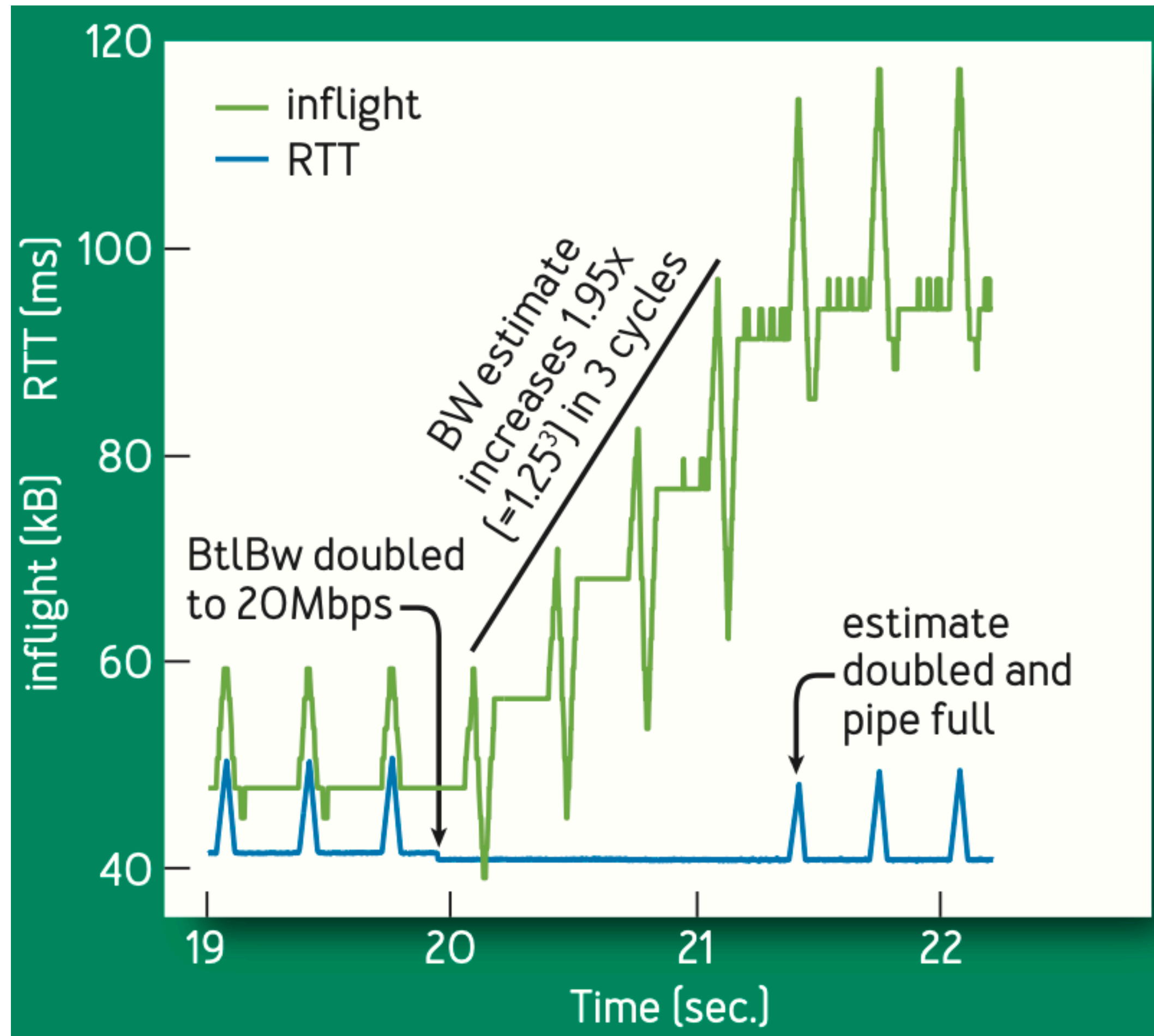


Steady-state behavior

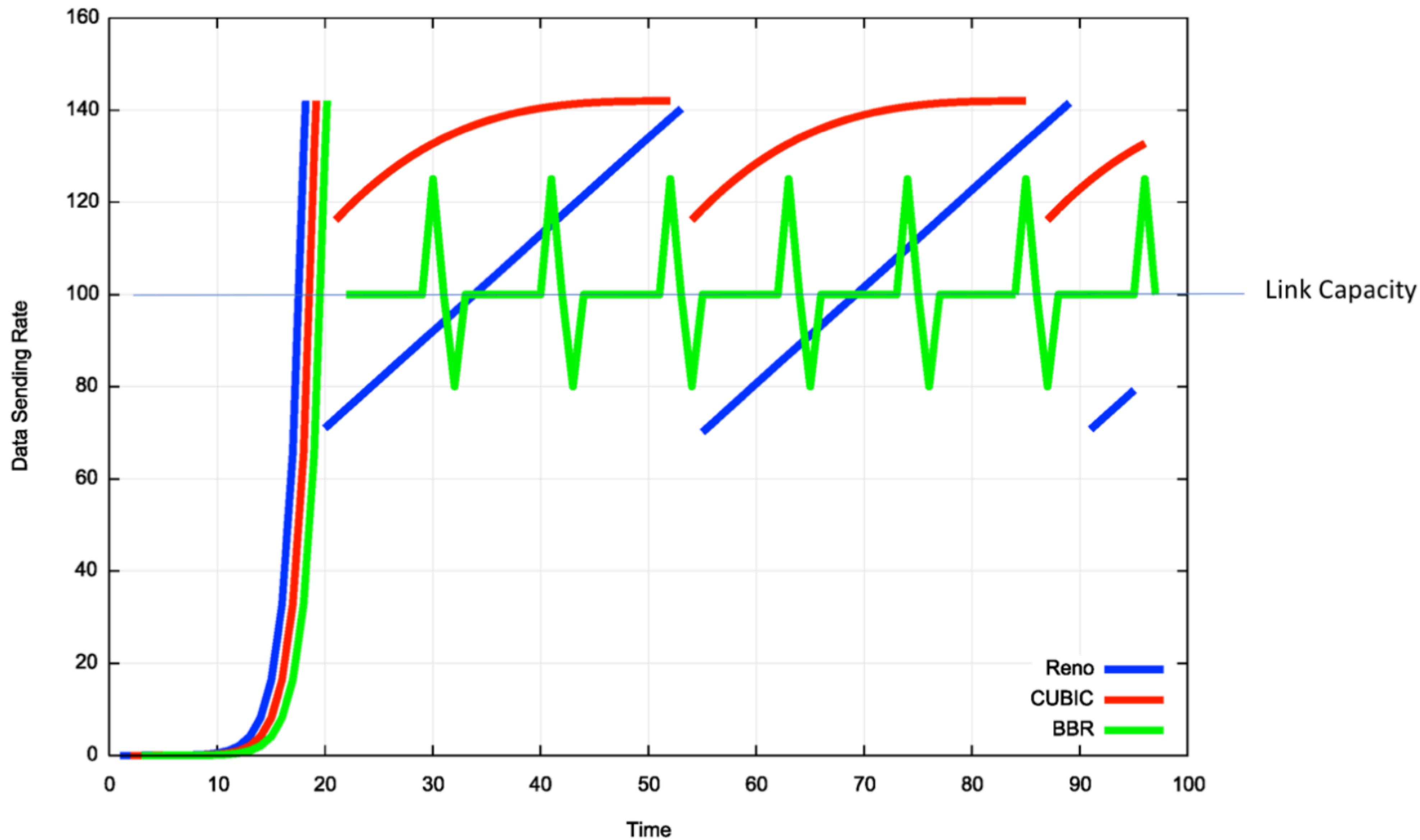
- Probe these estimates regularly, but not continuously
 - Maintain the sending rate at this estimated bottleneck capacity rate for 6 RTT intervals
- For the next RTT, raise the sending rate by 25% and sample the RTT
- If the RTT increased across this probe, then for the next RTT drop the sending rate by 25%
- Otherwise assume this is the new bottleneck bandwidth



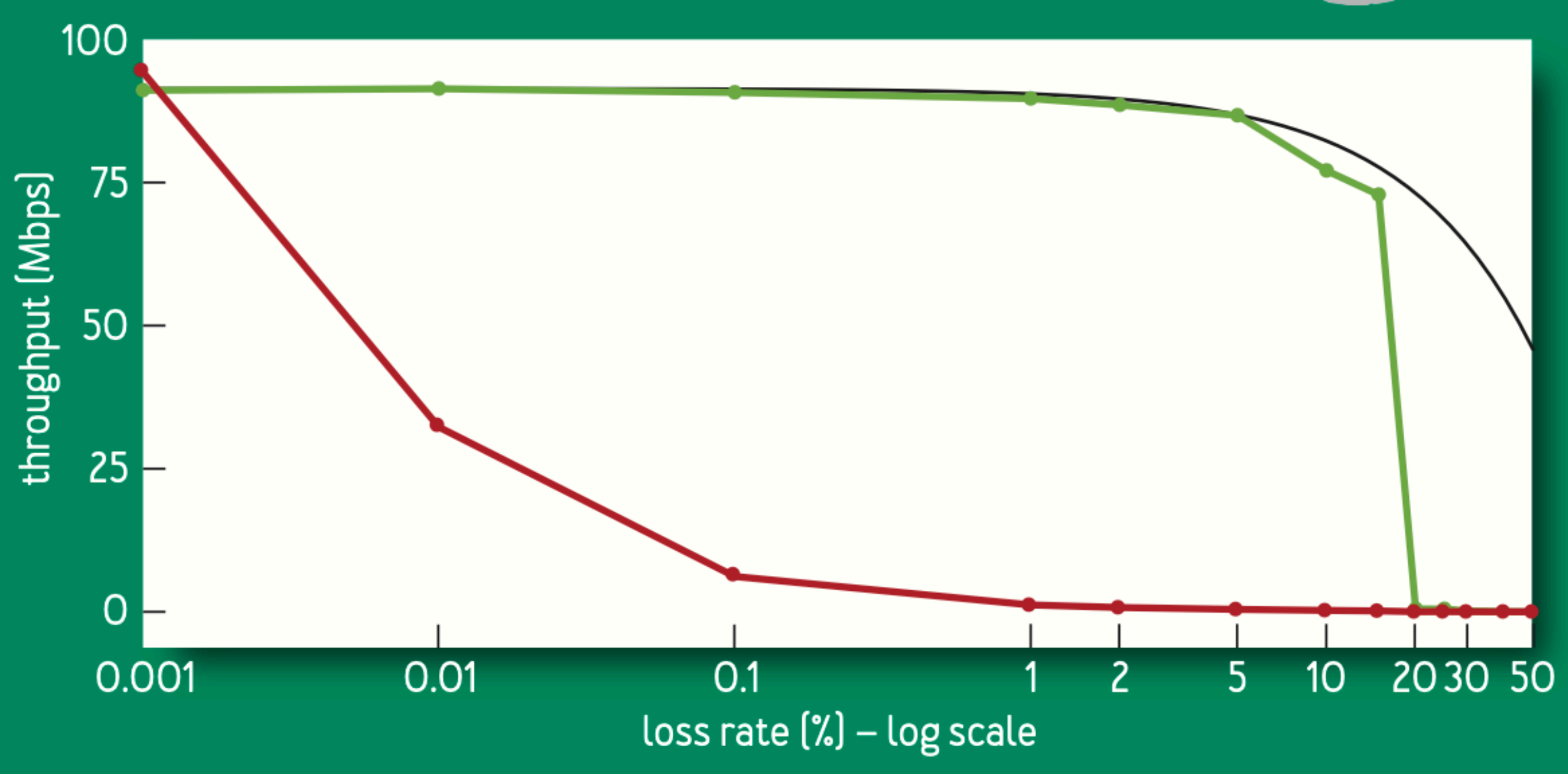
BBR reaction to sudden BW change



BBR, Reno, and CUBIC

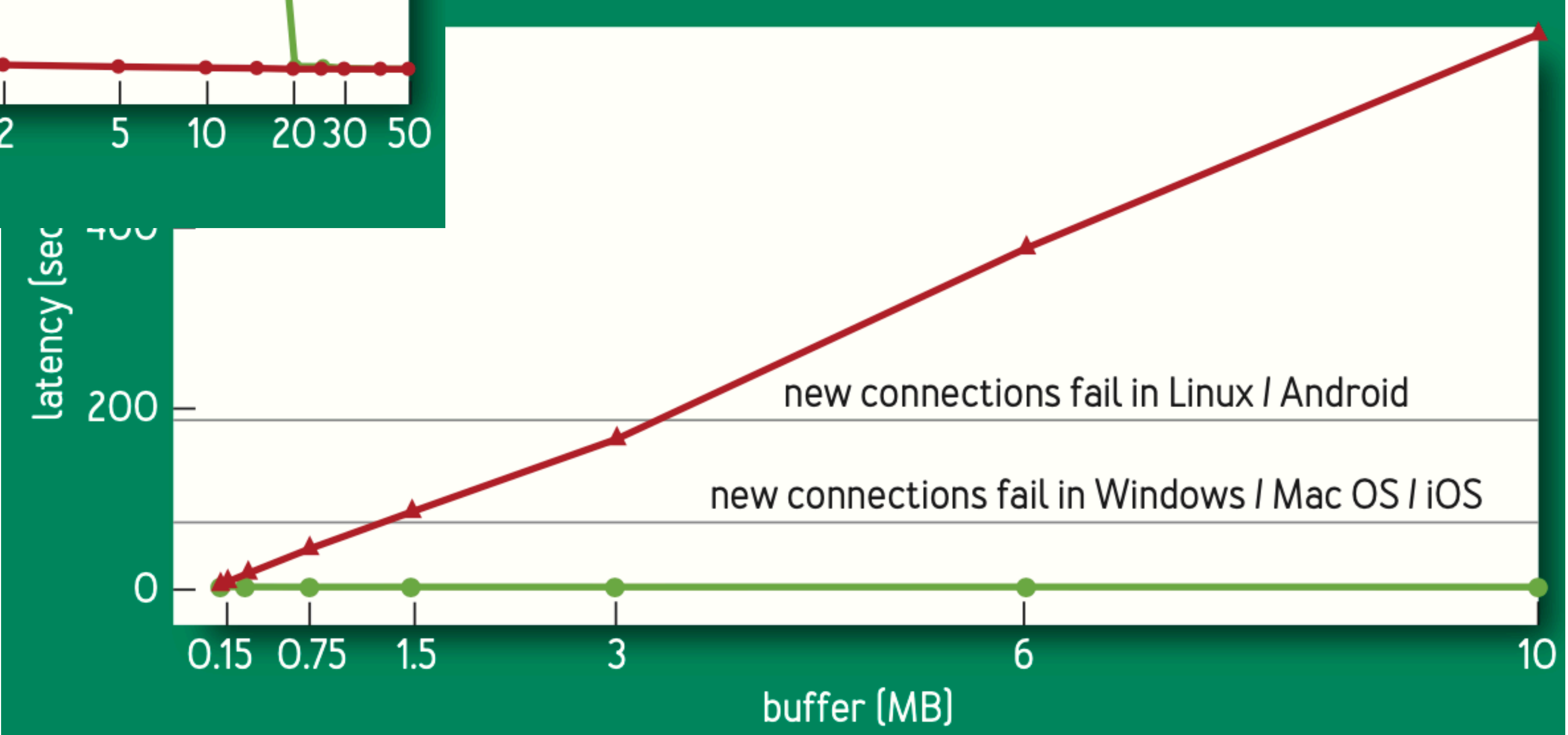


BBR performance gains



Low queue delay even with large buffer

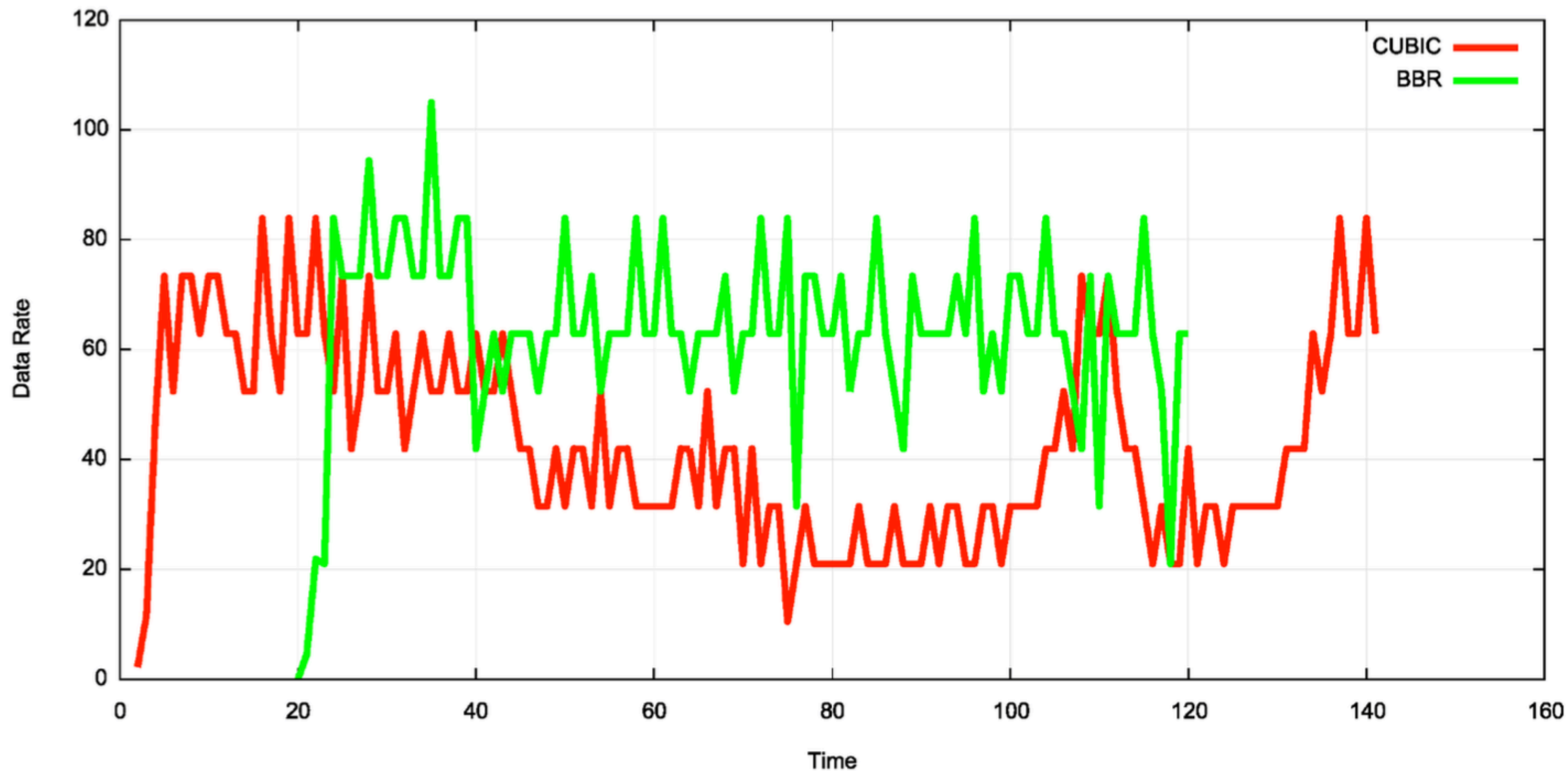
Fully utilizes the bandwidth even under the high loss situation



BBR vs. CUBIC

- Fairness is important when deployed on the Internet

BBR is able to compete with other flows and defend its bandwidth



Congestion control at the scale?

Revisiting TCP Congestion Control Throughput Models & Fairness Properties At Scale

Adithya Abraham Philip, Ranysha Ware,
Rukshani Athapathu, Justine Sherry, Vyas Sekar
Carnegie Mellon University
United States of America

Abstract

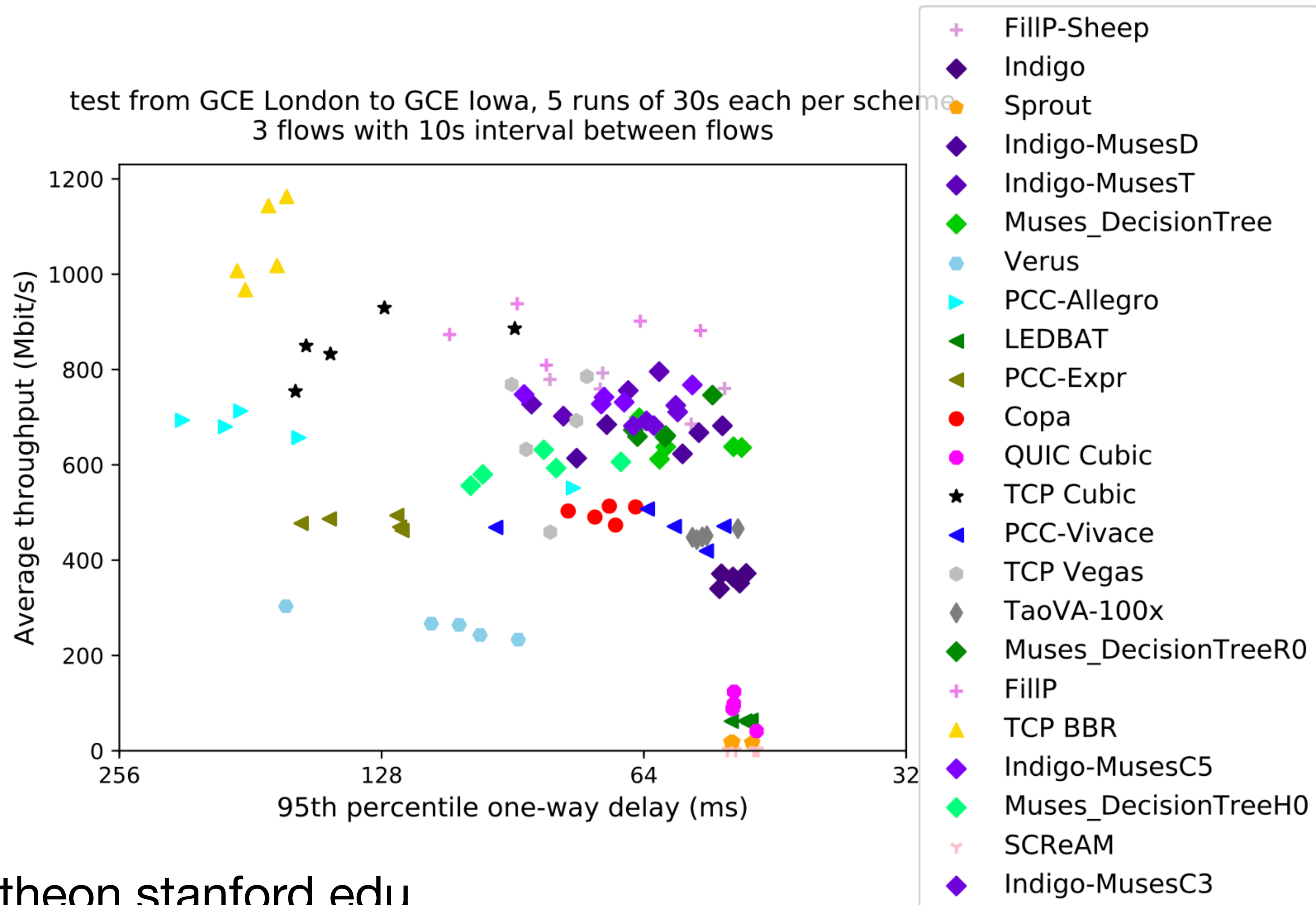
Much of our understanding of congestion control algorithm (CCA) throughput and fairness is derived from models and measurements that (implicitly) assume congestion occurs in the last mile. That is, these studies evaluated CCAs in “small scale” edge settings at the scale of tens of flows and up to a few hundred Mbps bandwidths. However, recent measurements show that congestion can also occur at the core of the Internet on inter-provider links, where thousands of flows share high bandwidth links. Hence, a natural question is: Does our understanding of CCA throughput and fairness continue to hold at the scale found in the core of the Internet, with 1000s of flows and Gbps bandwidths?

Our preliminary experimental study finds that some expectations derived in the edge setting do not hold at scale. For example, using loss rate as a parameter to the Mathis model to estimate TCP NewReno throughput works well in edge settings, but does not provide accurate throughput estimates when thousands of flows

1 Introduction

Congestion control algorithms (CCAs) are a fundamental building block of the modern Internet, and the networking community has been analyzing and designing CCAs for over three decades [12, 18, 26, 37, 39, 48]. Of specific interest to us is the wide-area setting. In this setting, *throughput* and *fairness* are important CCA properties, as they determine the effectiveness with which data can be transferred across the Internet, and the ability for multiple TCP flows to co-exist. To this end, many past efforts have used systematic models and experimental studies to understand these properties. For instance, the model by Mathis et al. [37] and Padhye et al. [41] predict the throughput of a NewReno flow as a function of packet loss and round-trip time (RTT), and the BBR model by Ware et al. [48] predicts the throughput of BBR when competing with other CCAs. Application developers can use such results to decide which CCA best suits the network conditions they experience or to debug performance issues.

Measured using nodes across the world



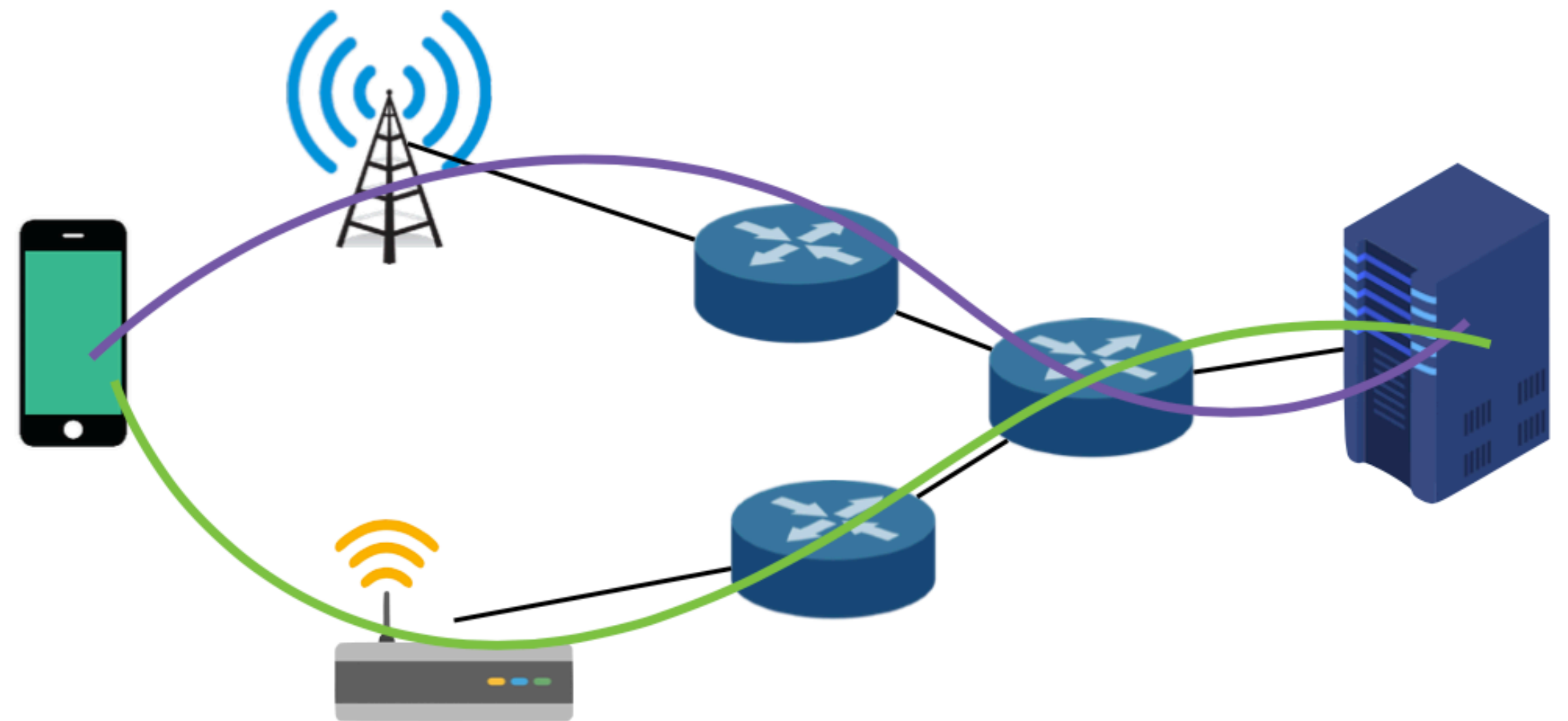
Advancements in network transport design

- TCP congestion control
- Multi-path TCP
- QUIC & HTTP 3.0

Multi-path transport

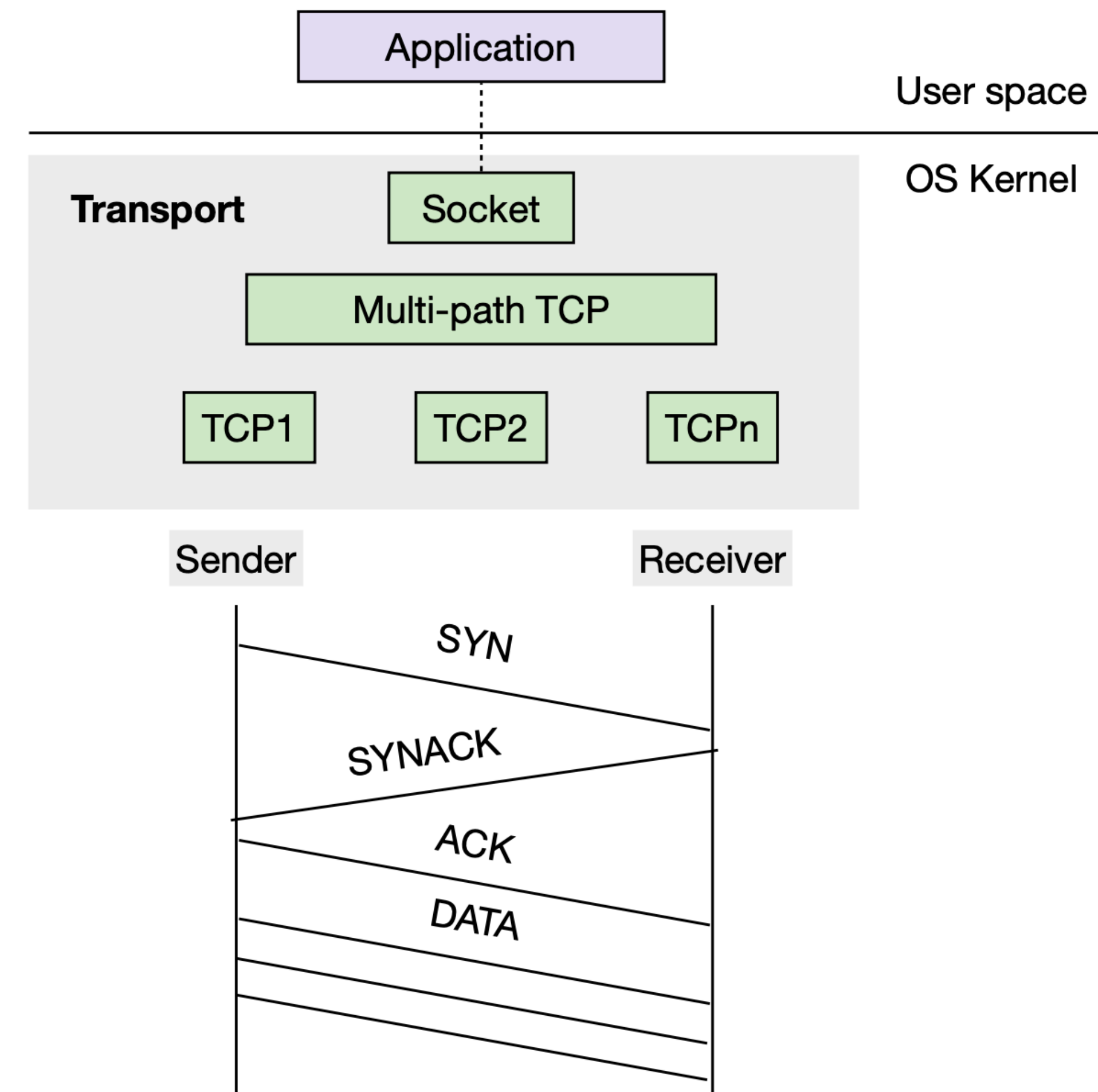
- Multi-homed devices become popular
 - Mobile devices (with cellular and WiFi at the same time)
 - High-end servers (multiple NICs)
 - Data centers (rich connectivity with many paths)

- Benefits of multi-path
 - Higher throughput
 - Failover from one path to another
 - Seamless mobility



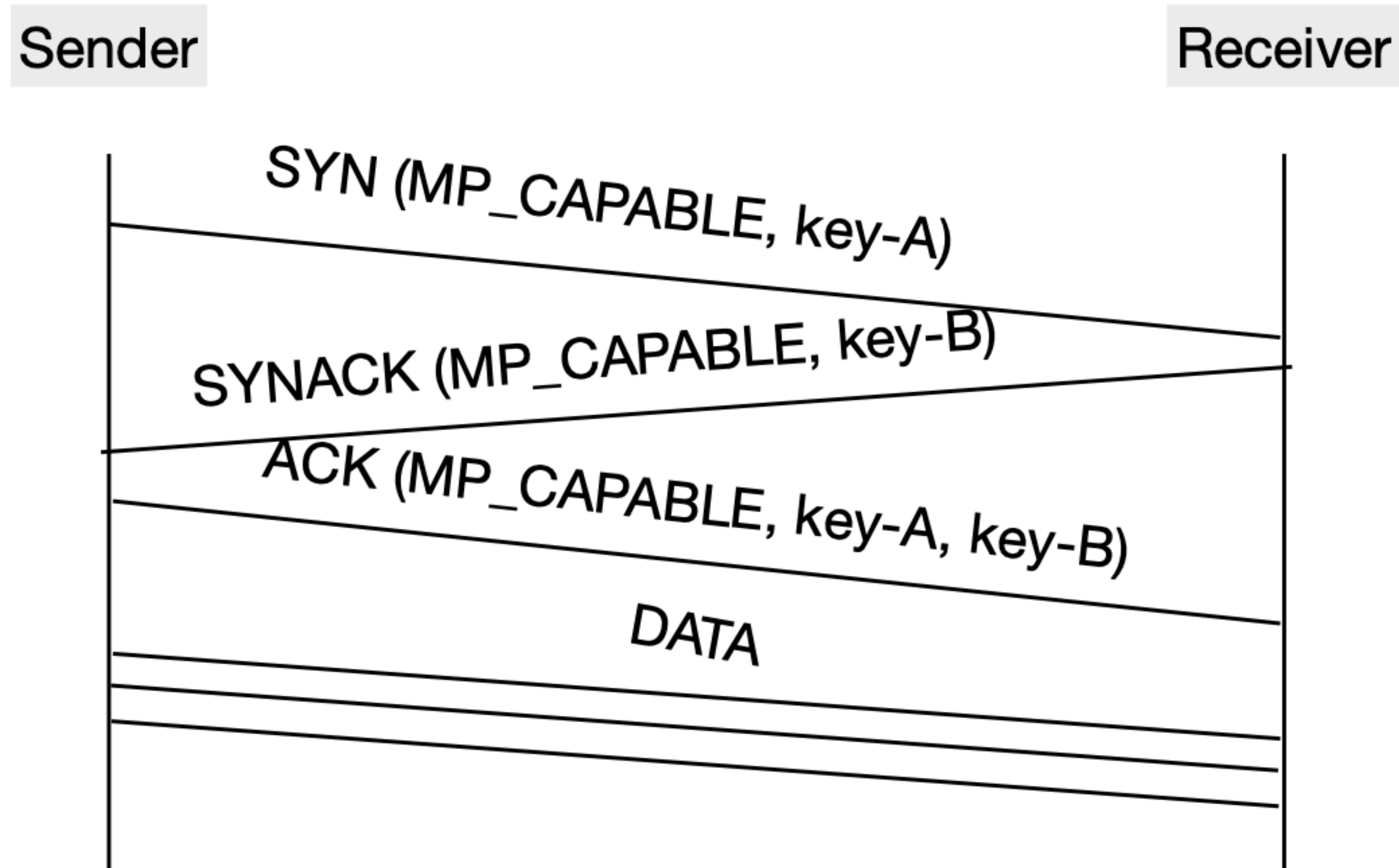
Working with unmodified applications

- Present the same socket API and expectations
 - Connection identified by the five tuples (src/dest IP, src/dest port, protocol)
- Establish TCP connections in the normal way
 - Create a socket to a single IP address/port
- Single flow by default, adding sub-flows if possible



MPTCP

- Capability negotiation
 - If SYNACK does not contain MP_CAPABLE flag, do not try to add sub-flows



During the TCP 3-way handshake phase, set the option “MP_CAPABLE” in TCP header

MPTCP

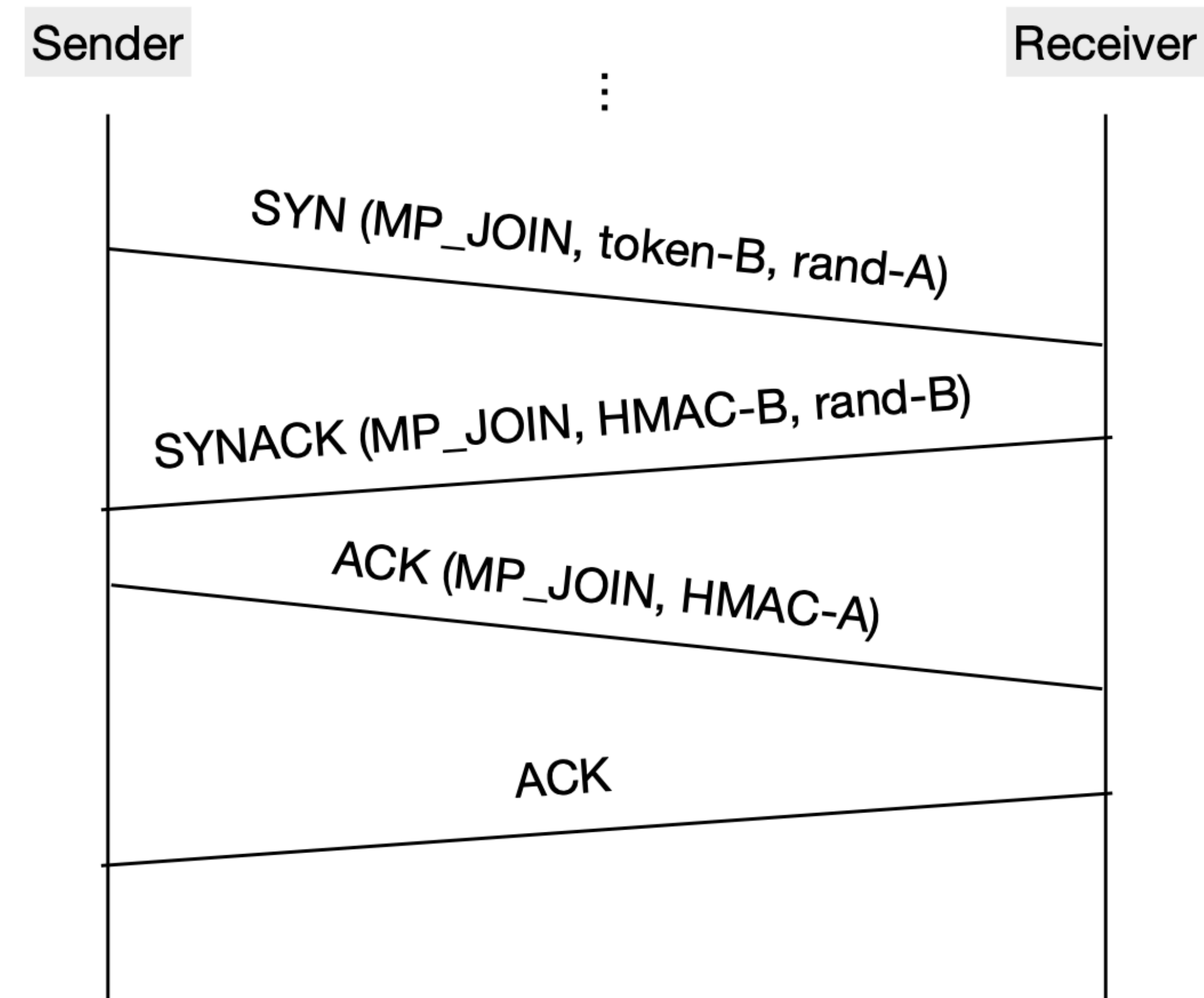
- Adding sub-flows

Question #1)

How to associate a new sub-flow with the connection



- Use a token hashed from the exchanged key
- Hash-based Message Authentication code (HMAC) for authentication based on exchanged keys



MPTCP

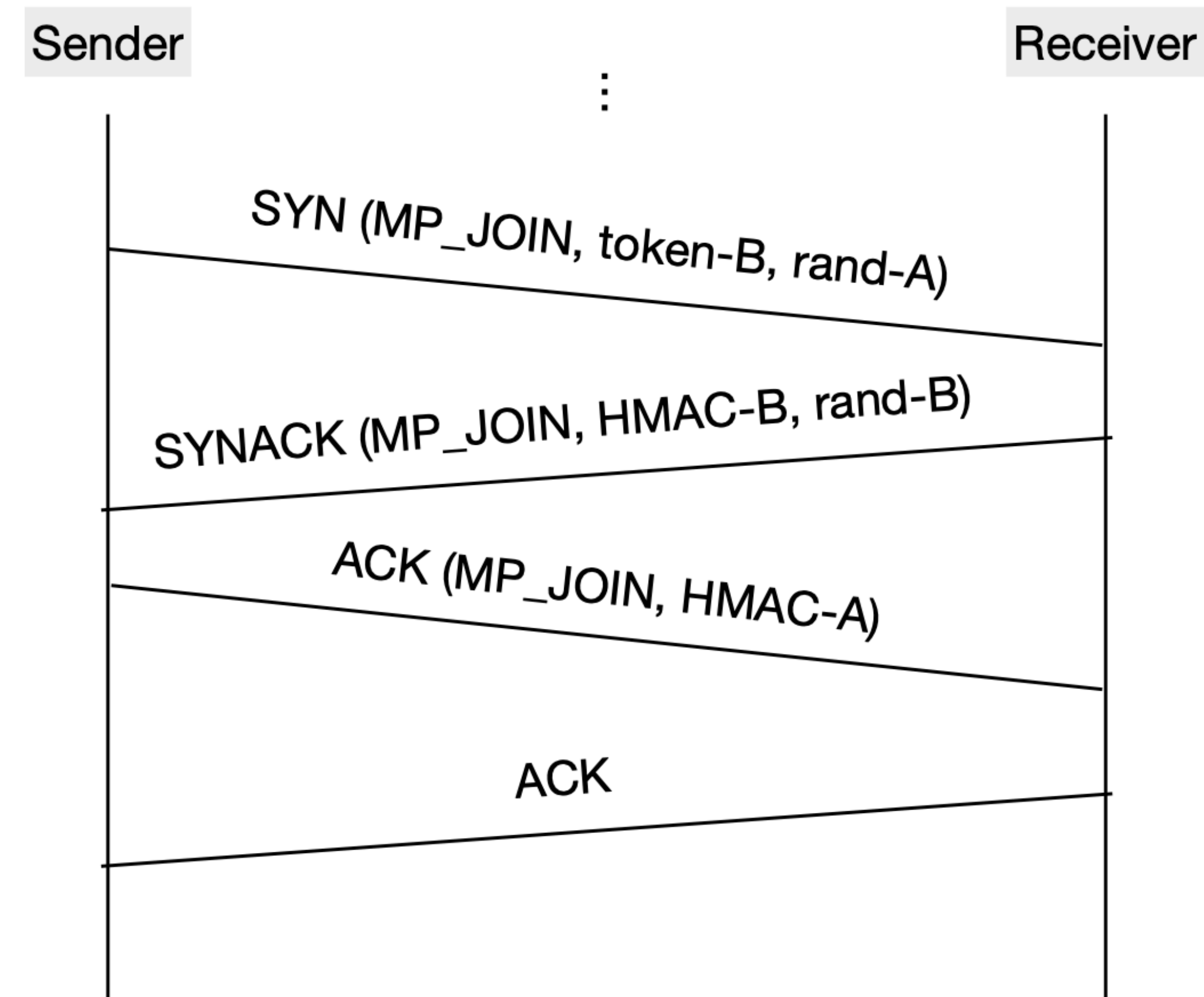
- Adding sub-flows

Question #2)

How to start using the new sub-flow(s)



- Start sending packets with new IP/port pair
- Associate the sub-flow(s) with the existing connection



MPTCP

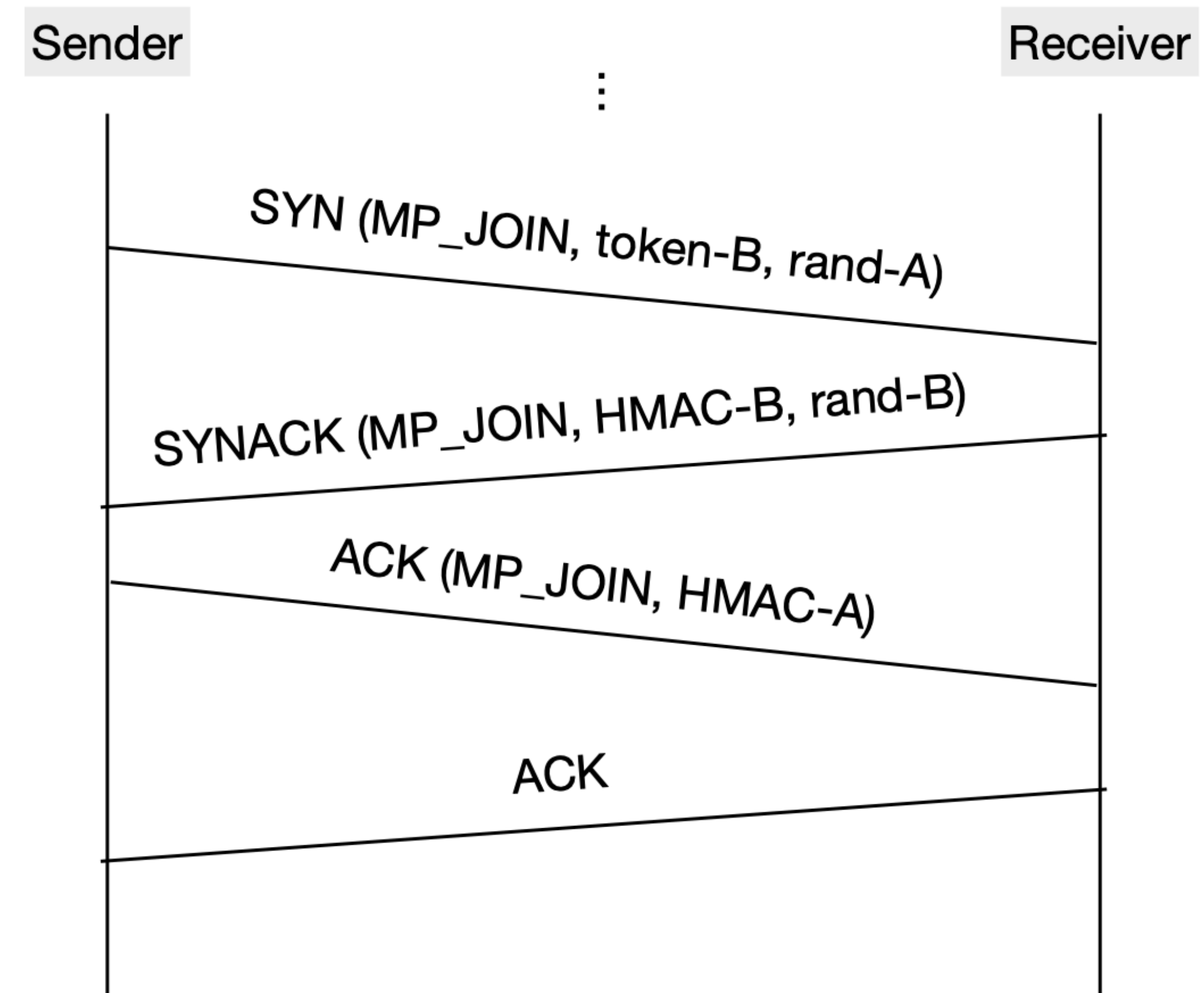
- Adding sub-flows

Question #3)

How to learn about extra IPs for new sub-flow

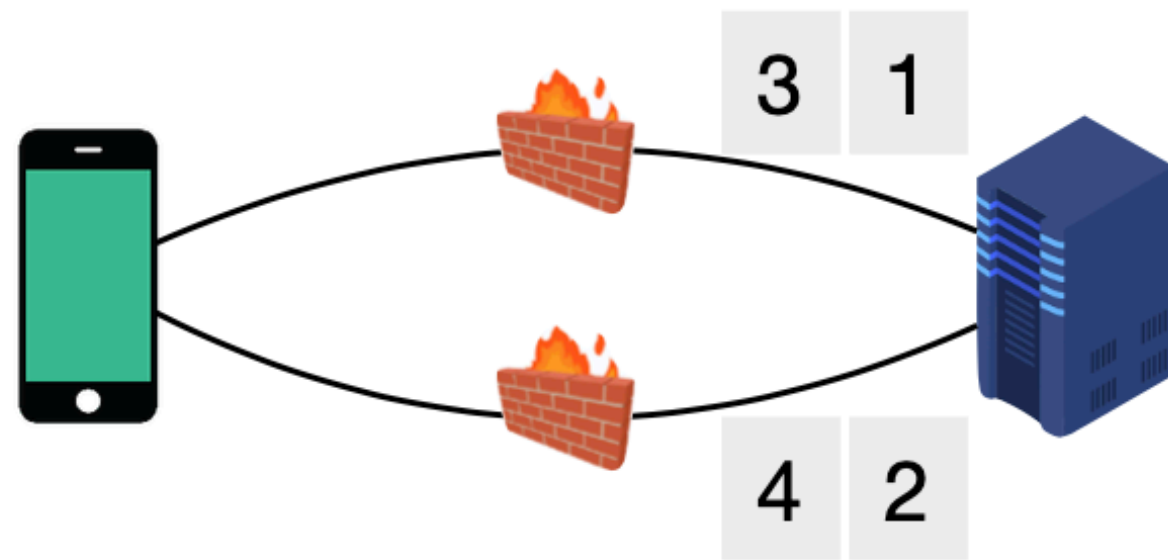


- One end host first establishes a new sub-flow to a known address at the other end-host

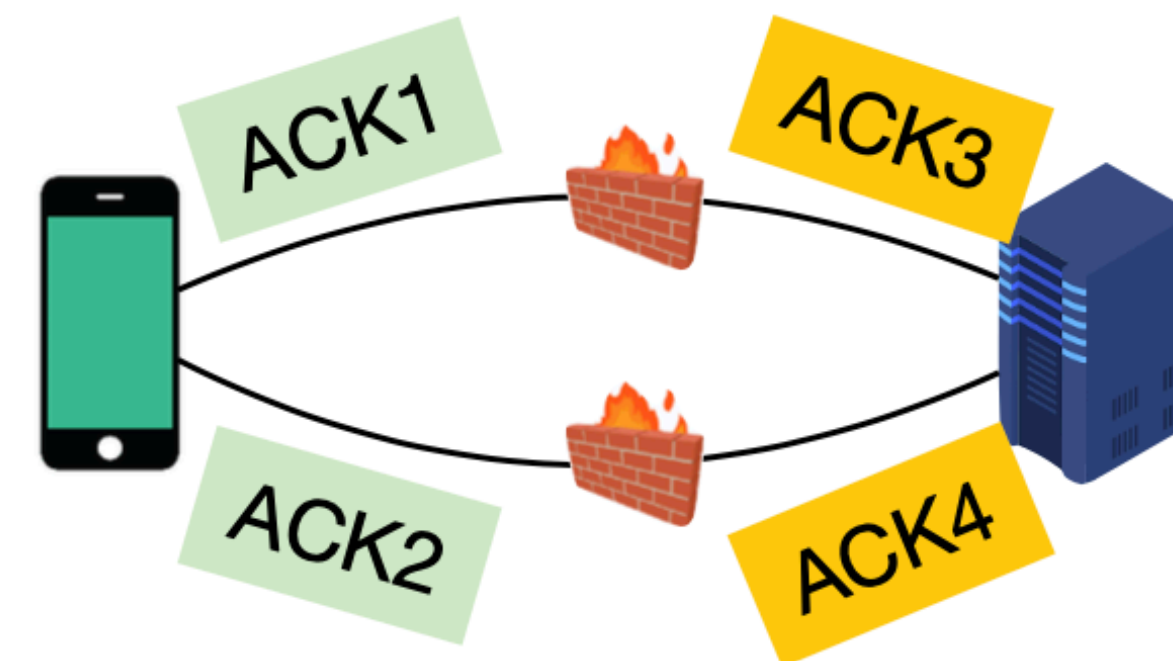


The curse of the middle box

- Middleboxes are network equipments that apply special operations on the path of network packets
 - Firewall, network address translator (NAT), deep packet inspection, etc.
 - Some of them inspect TCP traffic - check TCP sequence number



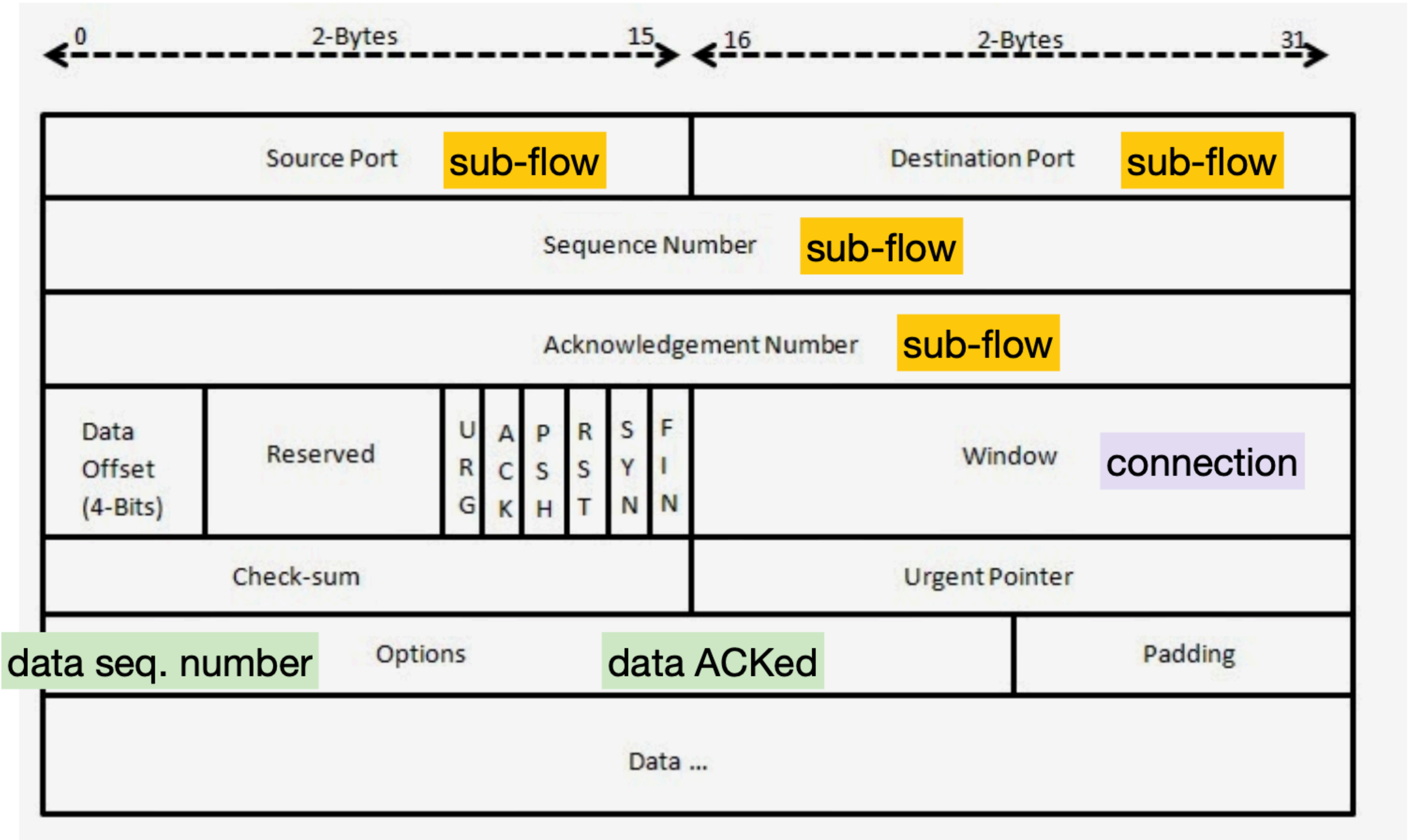
The middlebox may rewrite
ACK3 to ACK2 since it has not
seen ACK2 yet



The middlebox may drop ACK4
since ACK3 is not seen

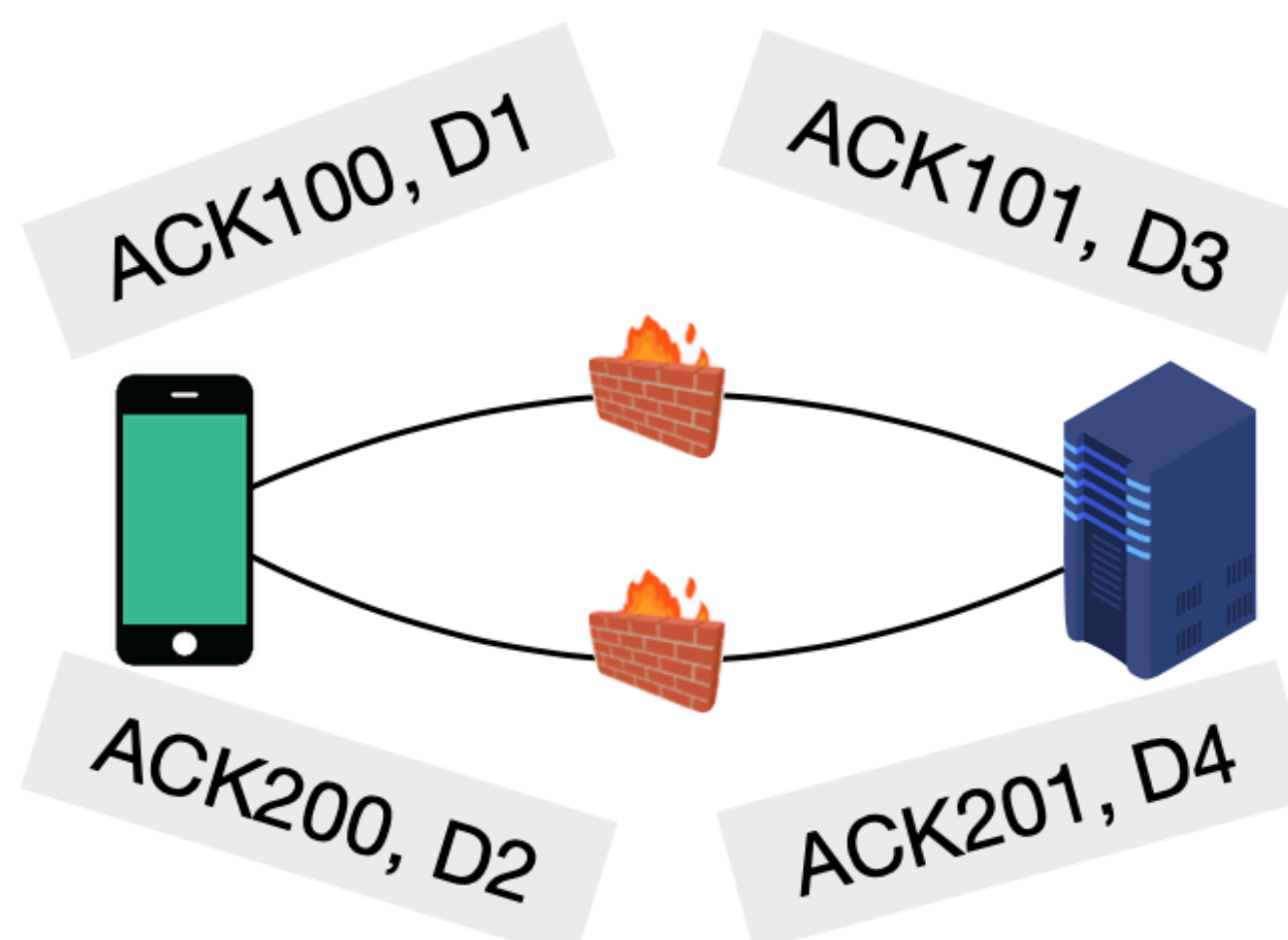
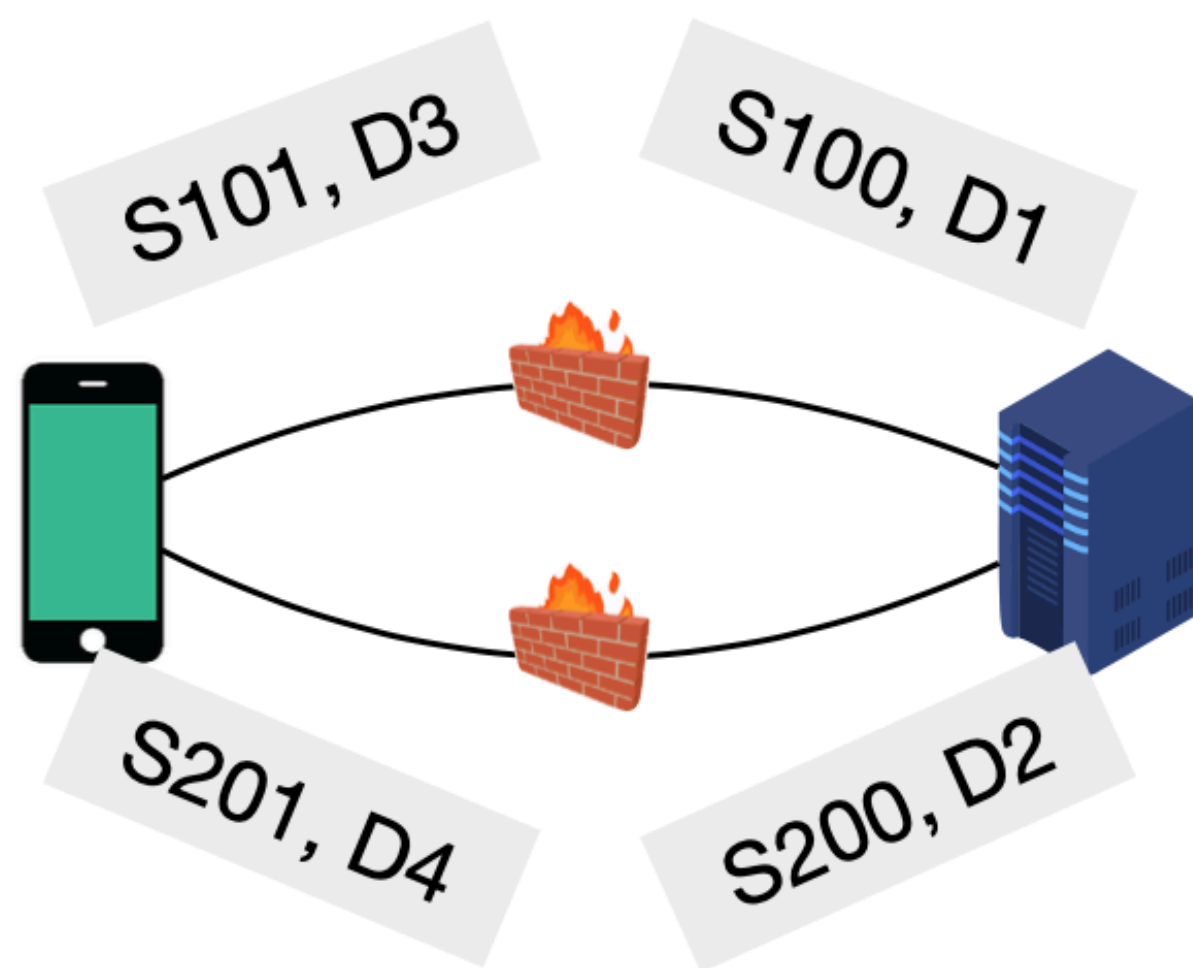
Solution

- Per-flow sequence number

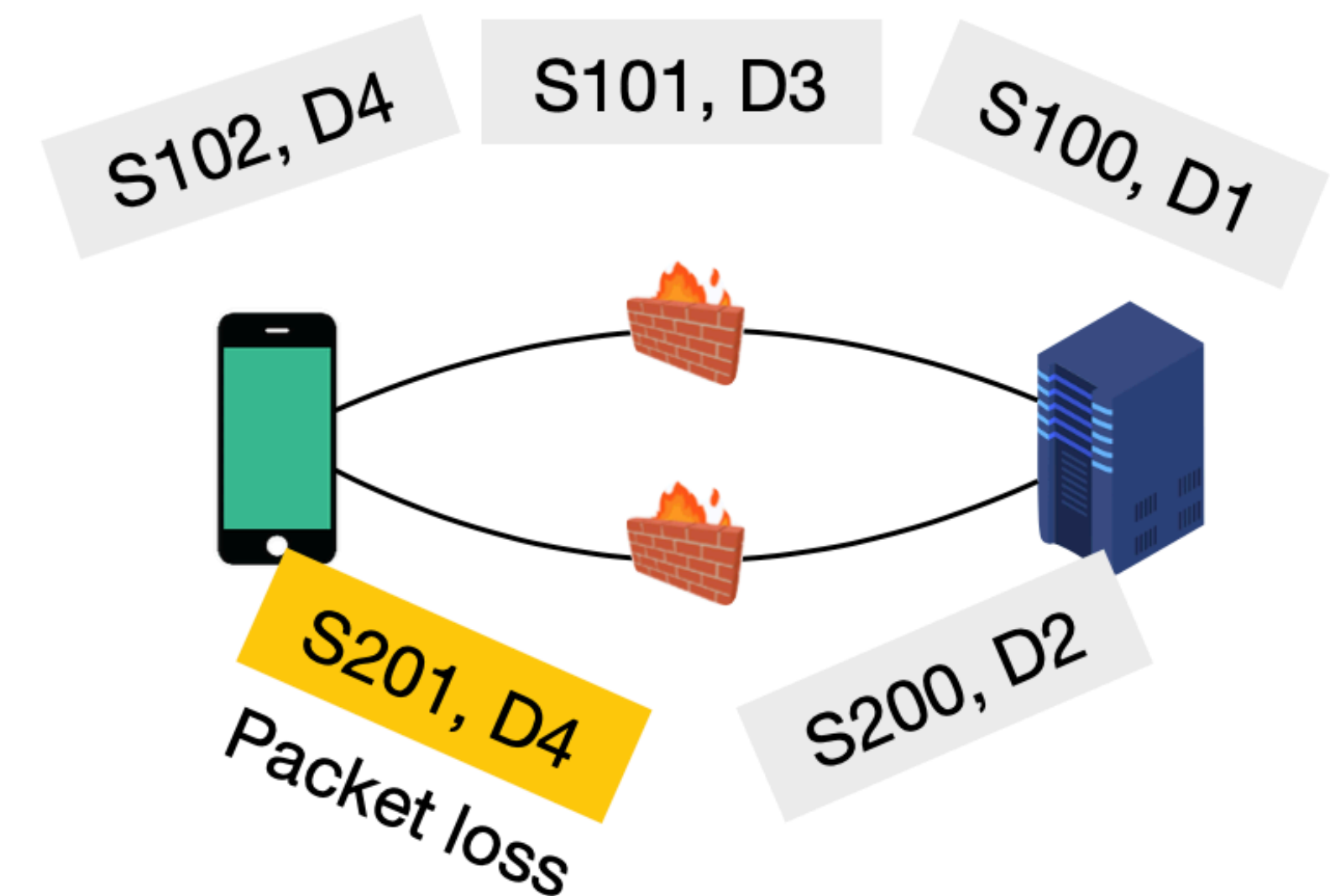


MPTCP

- All sub-flow share the same receive buffer and use the same receive window
 - Prevent starvation of sub-flows
 - Achieve TCP-level fairness
 - Prevent from buffer fragmentation

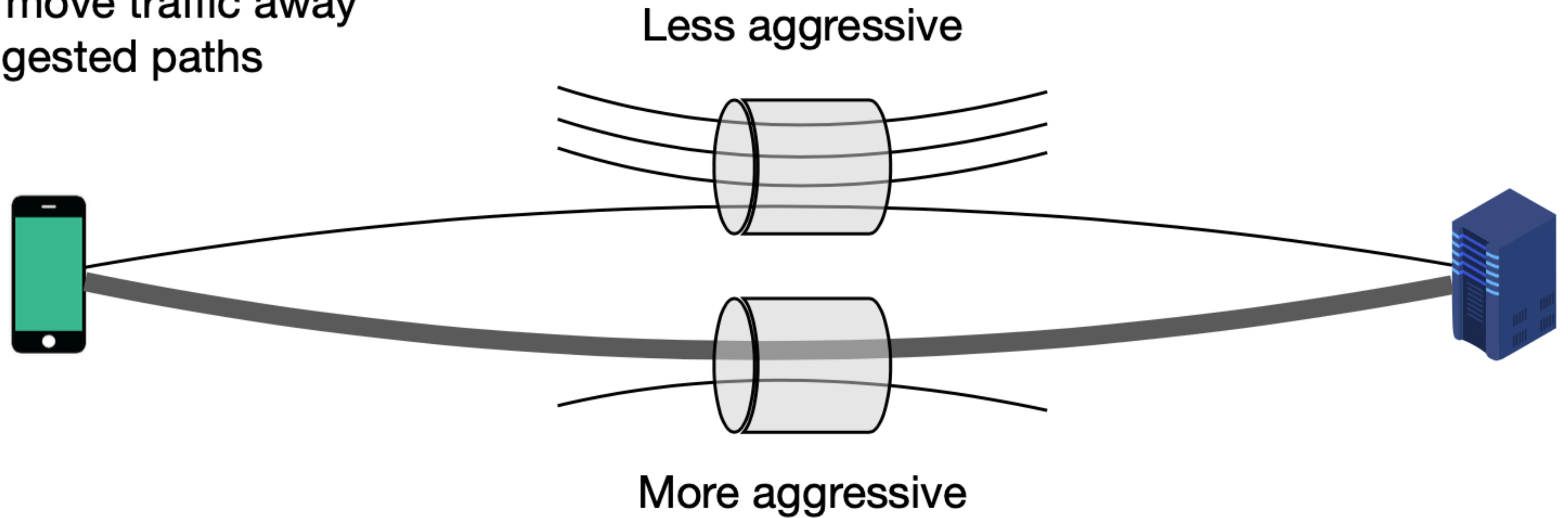


Retransmission on the
other sub-flow



MPTCP congestion control

Congestion control parameters for all sub-flows are linked to move traffic away from the more congested paths



Each sub-flow runs its own congestion control policy, to detect and respond to congestion it sees.

MPTCP congestion control

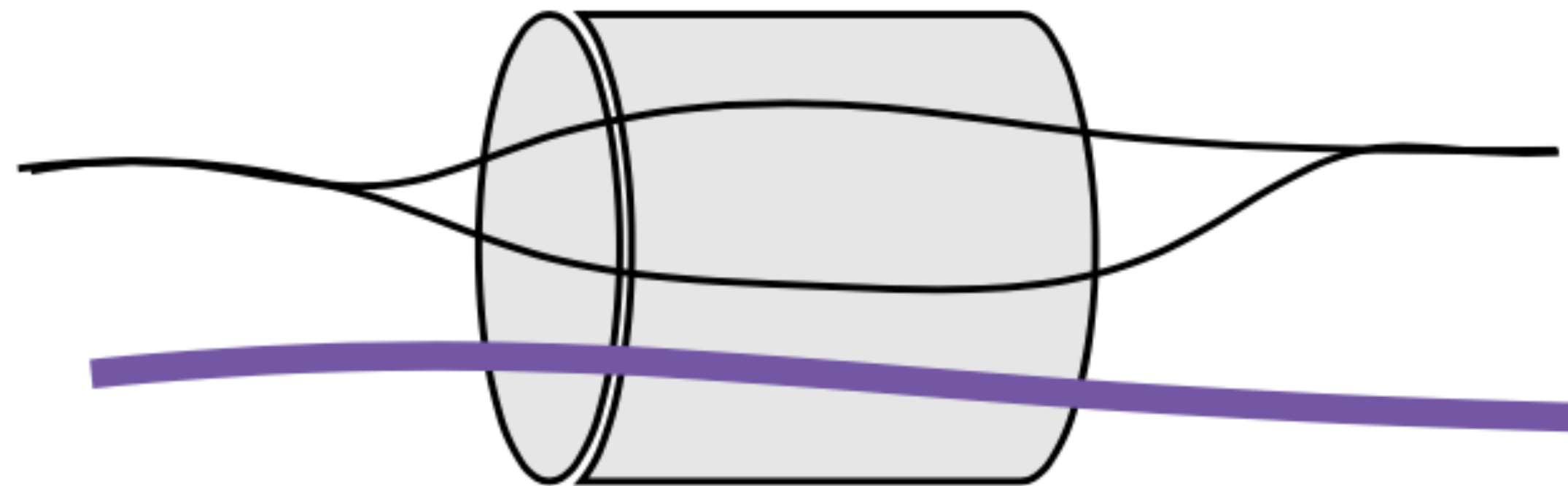
- Objectives
 - Be fair to TCP at bottleneck links
 - Use efficient paths
 - Perform as well as TCP
 - Do not oscillate

Be fair to TCP

- To be fair, MPTCP should take as much capacity as TCP at bottleneck link, no matter how many paths it is using

A MPTCP flow with two
sub-flows

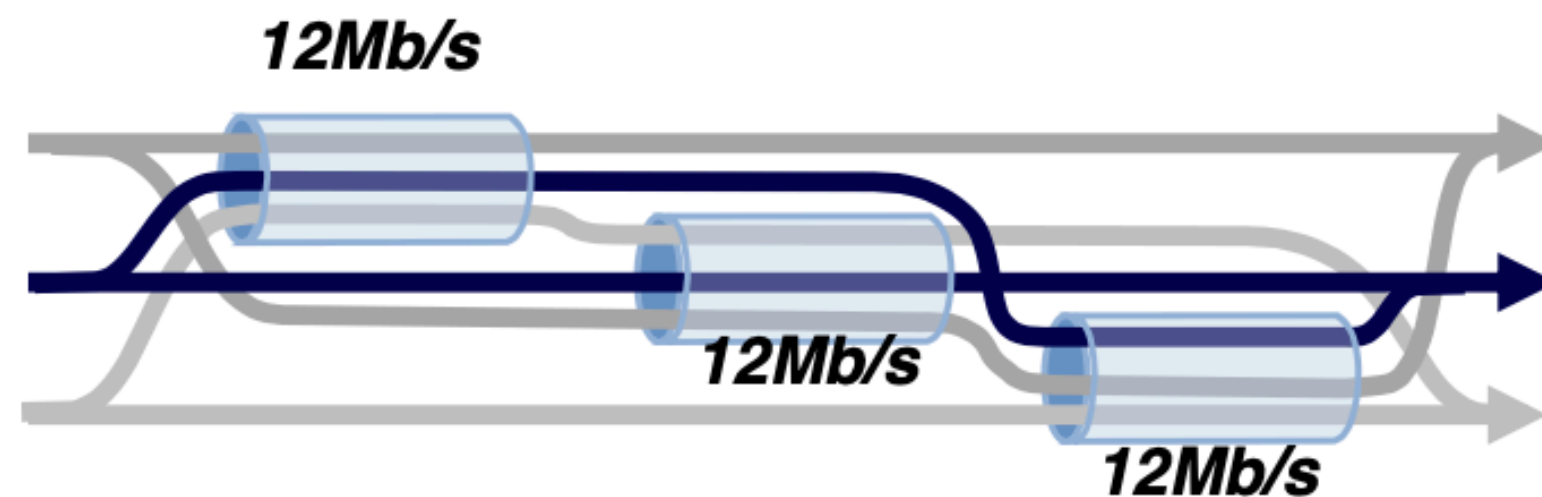
A regular TCP flow



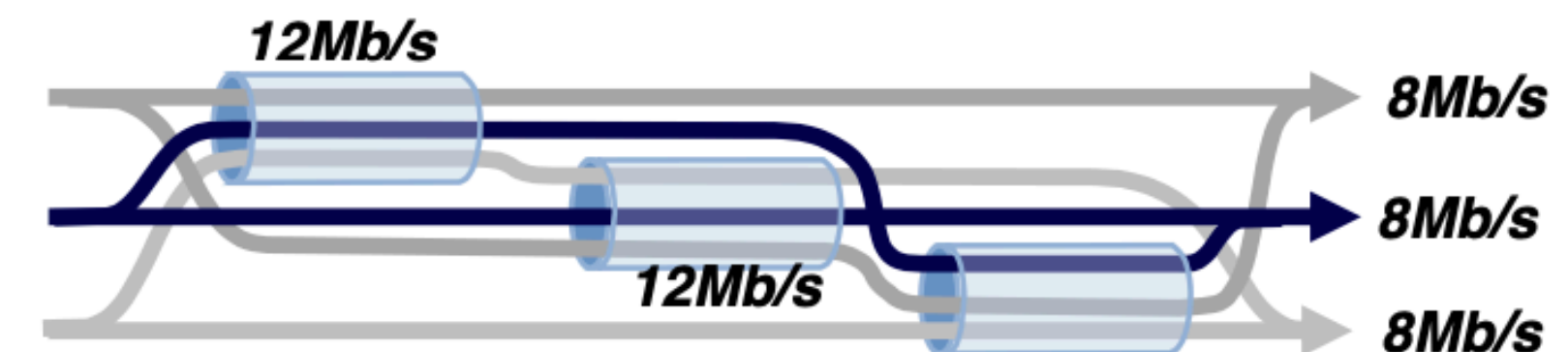
Use efficient paths

- Each connection should send all traffic on the least congested paths, but keep some traffic on the alternative paths as a prob.

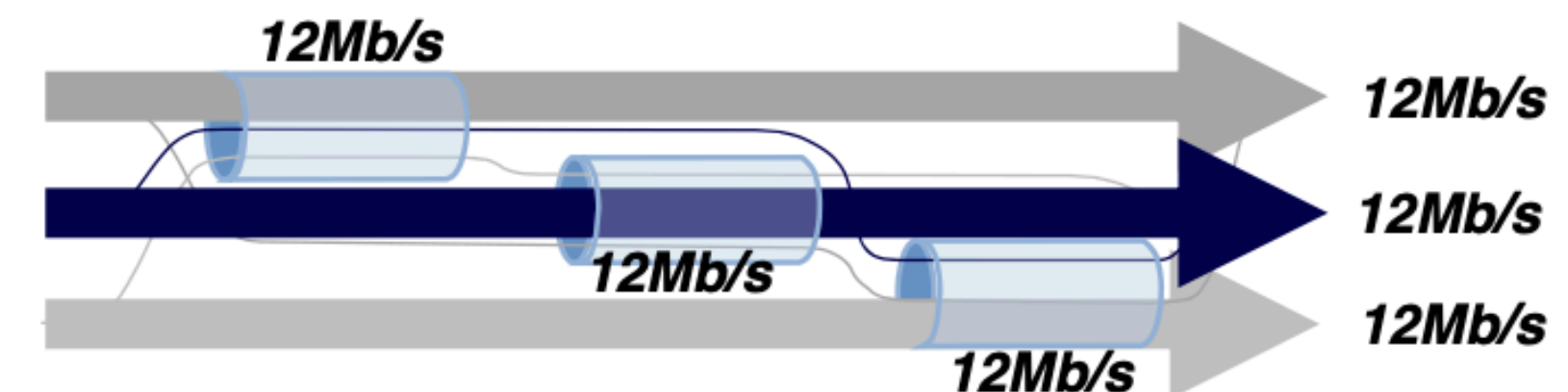
Each flow has a choice of a 1-hop and a 2-hop path. How should its traffic be split?



Fair-share

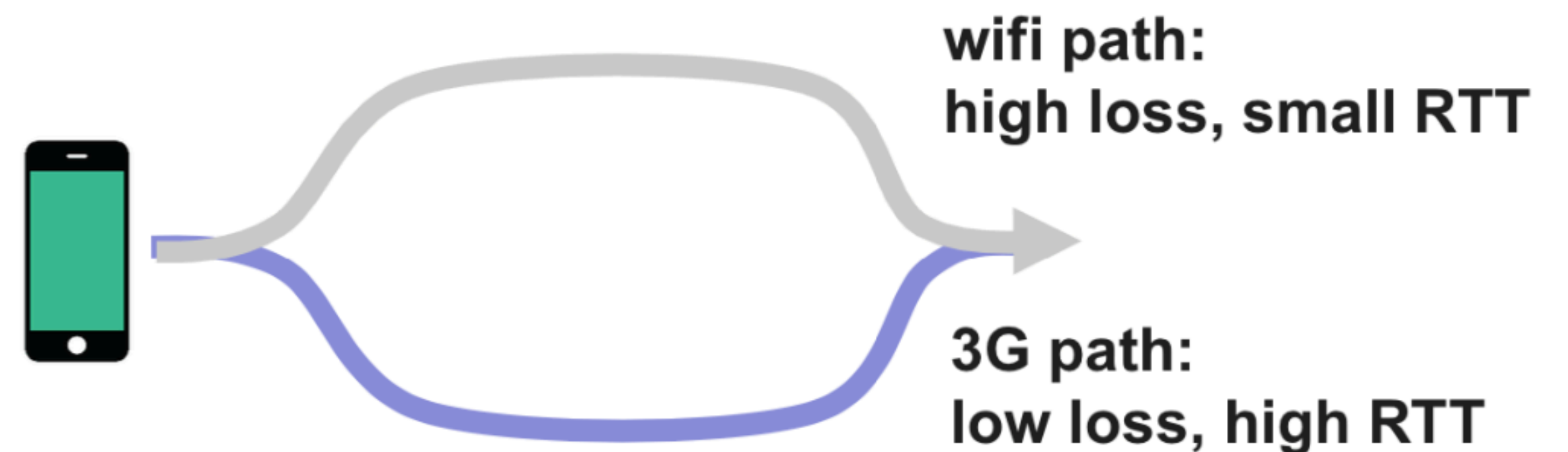


1-hop-only



Perform as well as TCP

- An MPTCP user should get at least as much throughput as a single-path TCP would on the best of the available paths
- An MPTCP flow should take no more capacity on any link than a single-path TCP would



According to the previous principle, traffic should be sent on the least congested paths. 3G path in this case?

MPTCP congestion control

- Maintain a congestion window W_r for each sub-flow $r \in R$
- Increase W_r for each ACK on path r , by

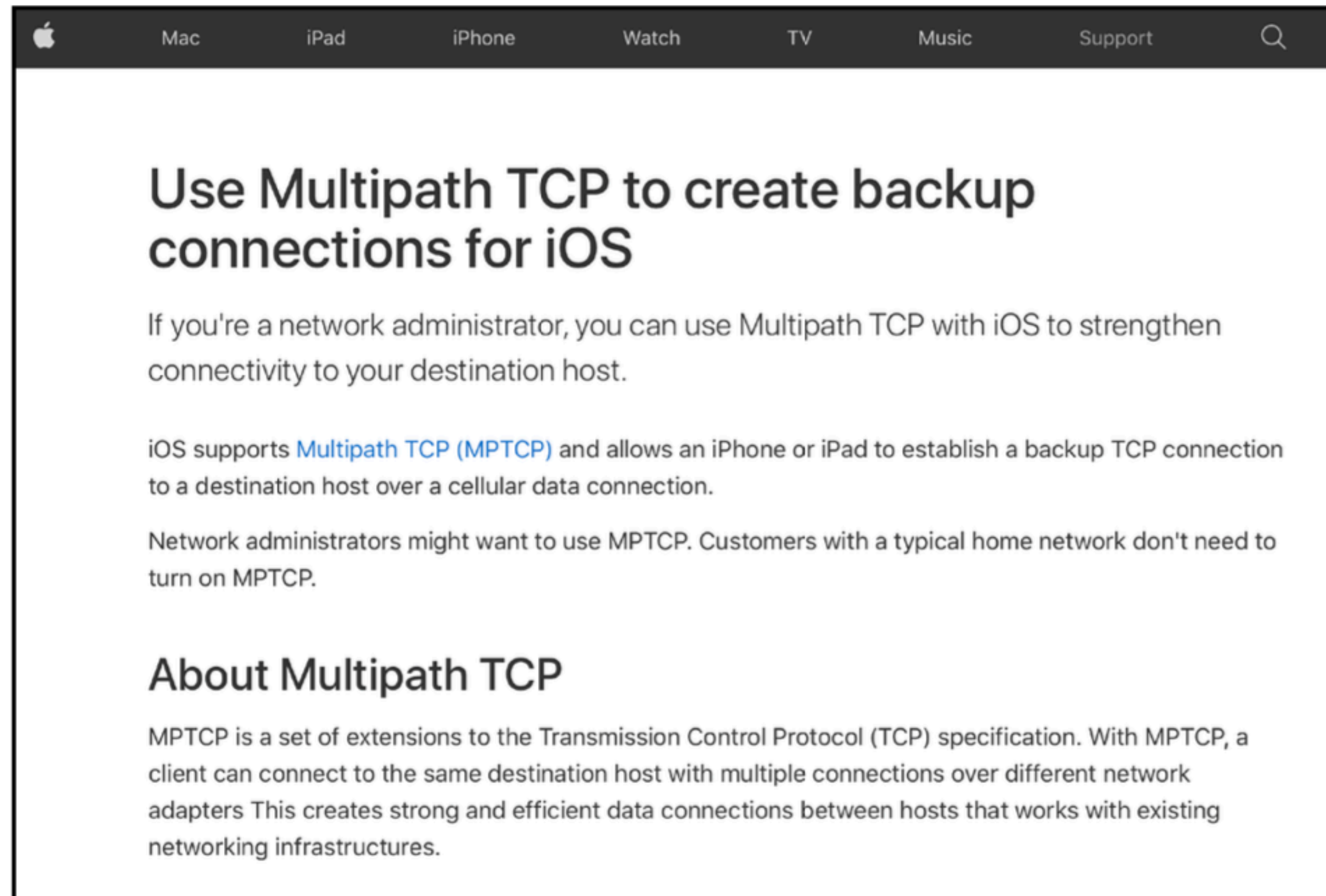
$$\frac{\alpha(W_r)}{\sum_{r \in R} W_r}$$

: α can be fine-tuned such that the aggregated throughput is similar to what you can get from a single path TCP

- Decrease W_r for each drop by

$$\frac{W_r}{2}$$

MPTCP use cases

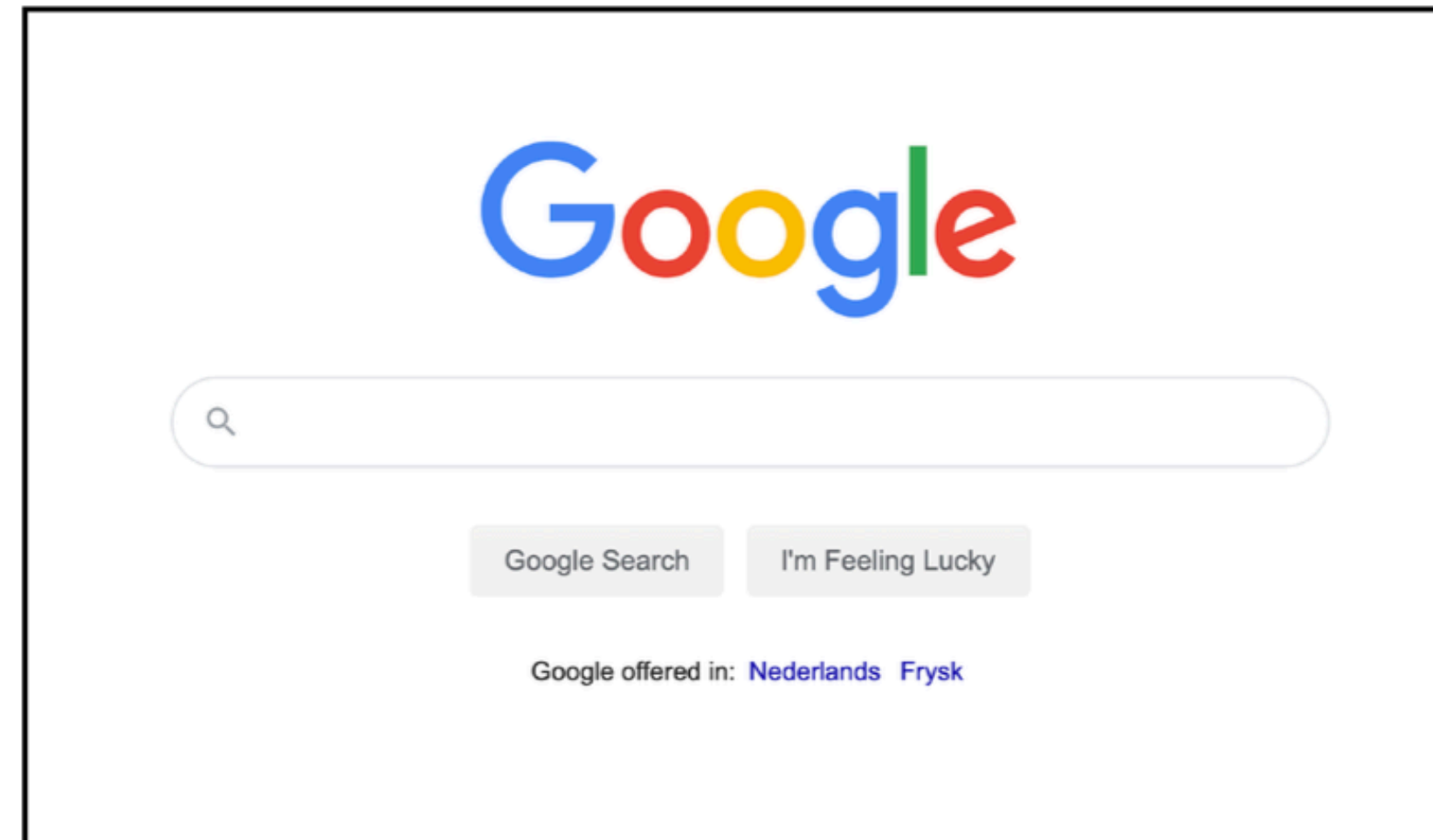
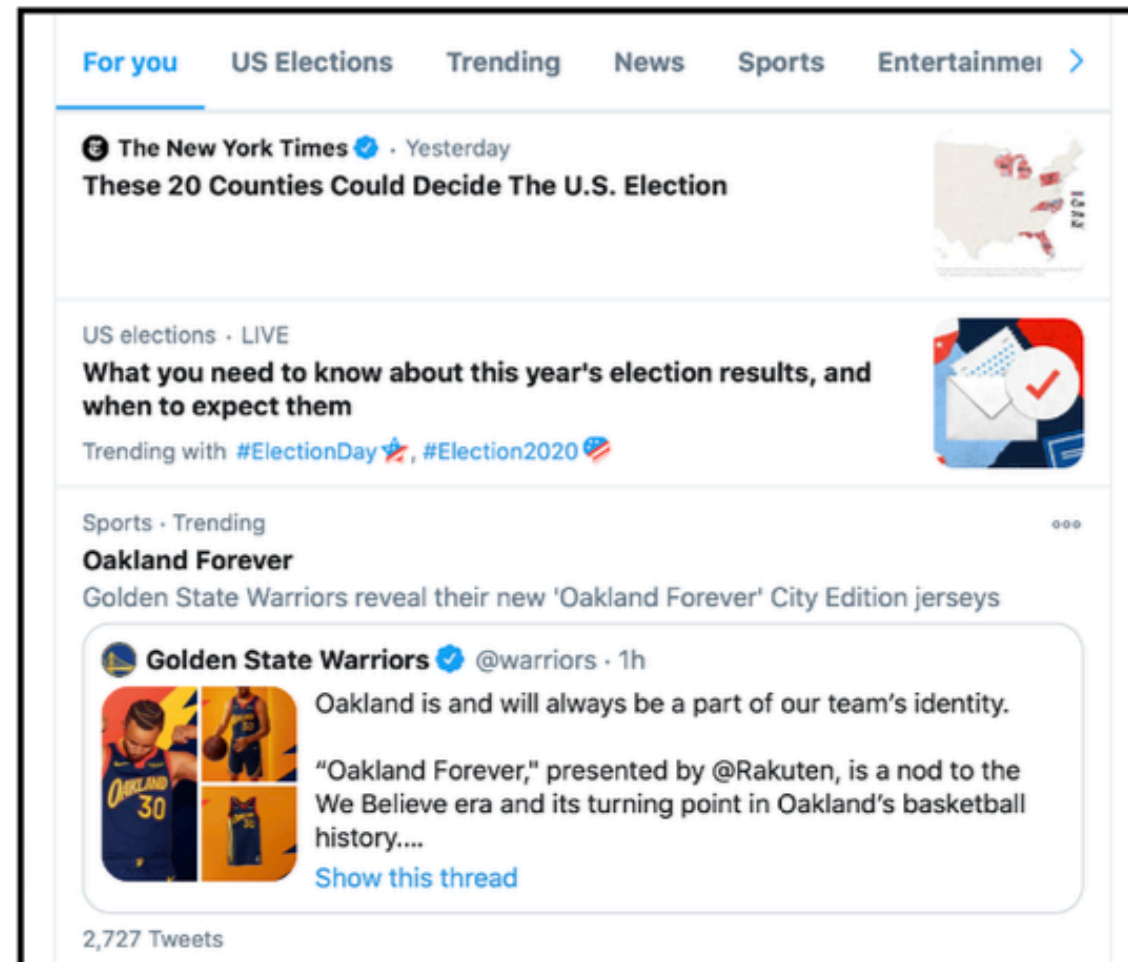


Since iOS 7 (2013), MPTCP is available on iOS devices. Cellular data is used as backup for WiFi connections when the WiFi signal is poor, for Apple services like Siri.

Advancements in network transport design

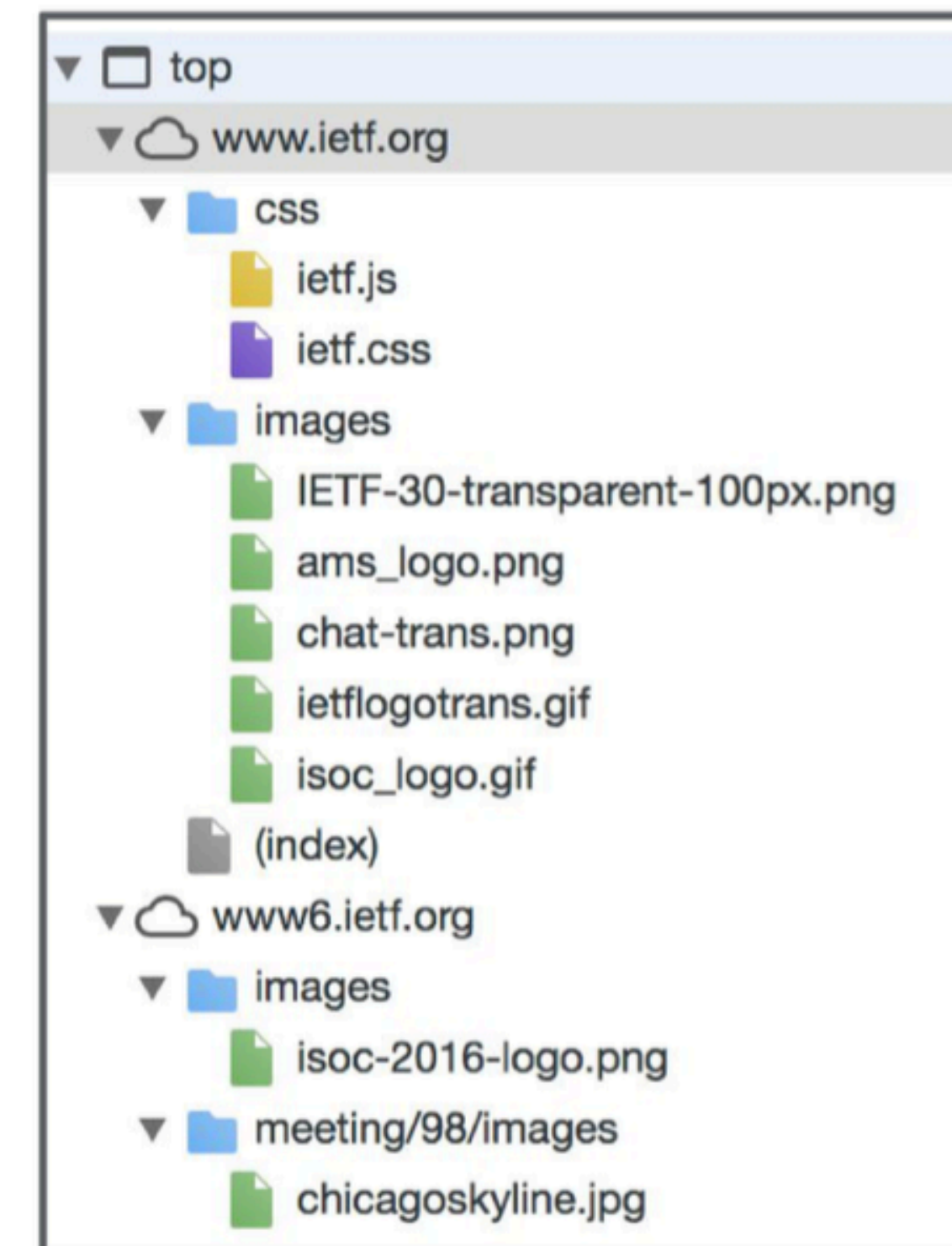
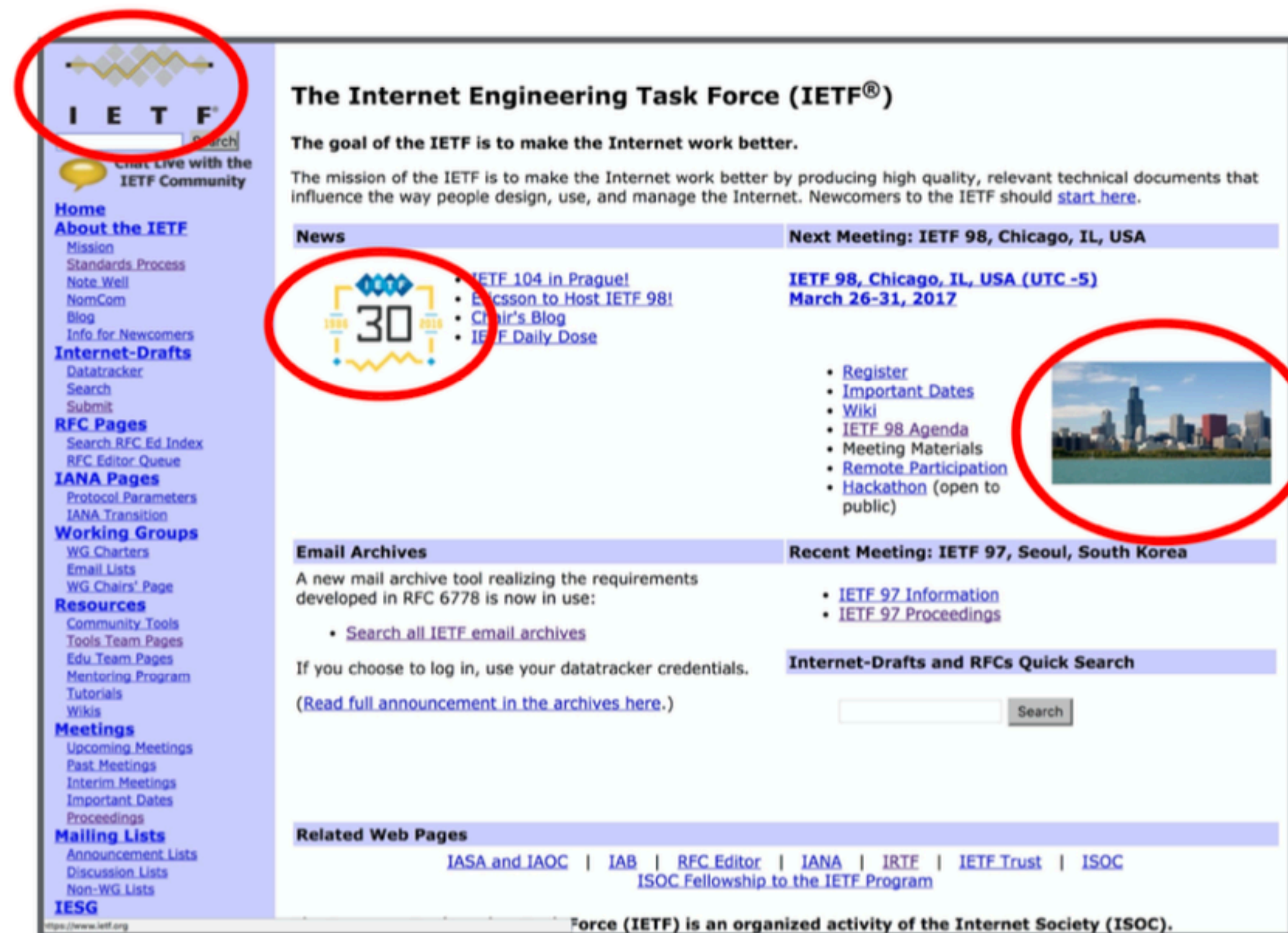
- TCP congestion control
- Multi-path TCP
- QUIC & HTTP 3.0

What is the most popular Internet application?



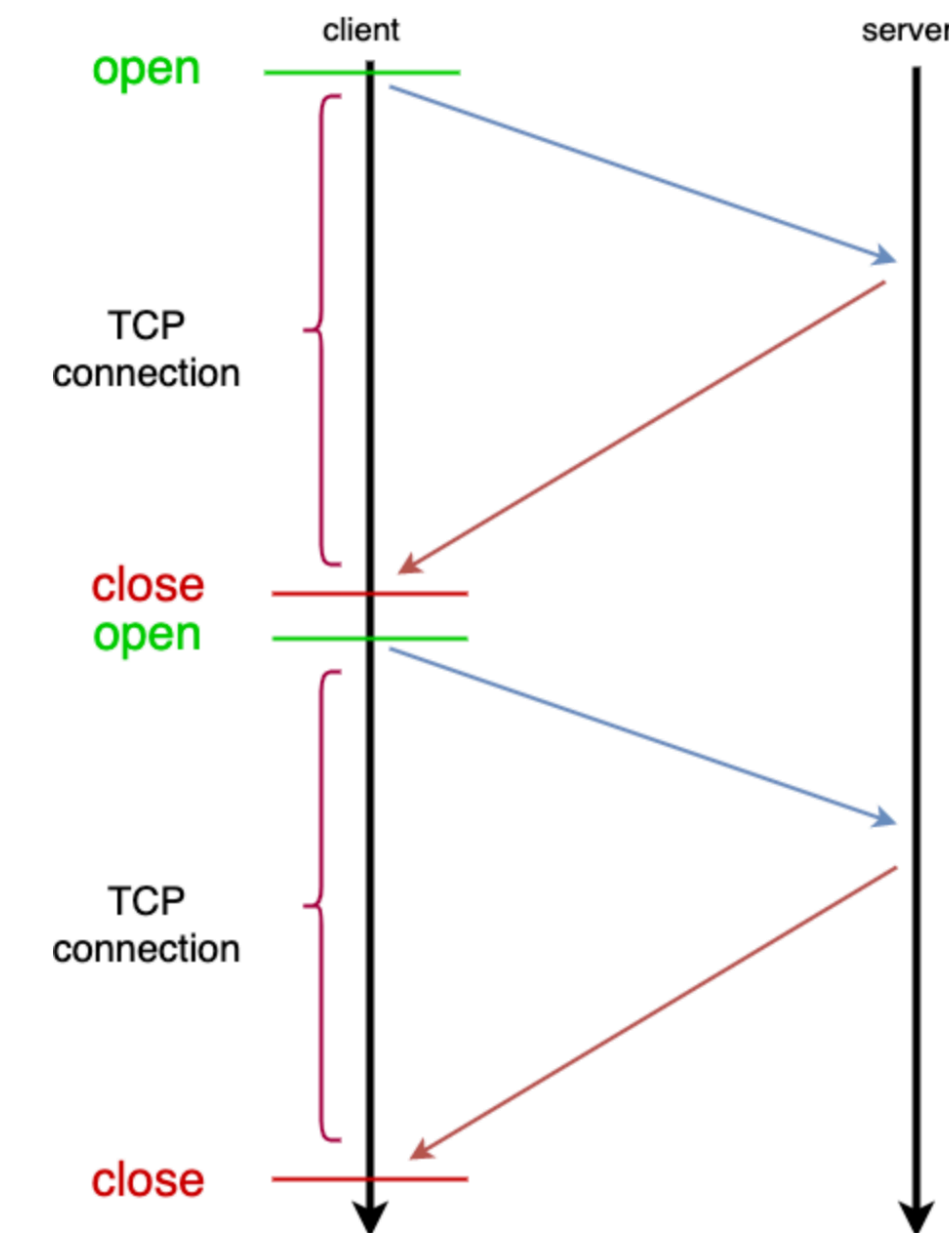
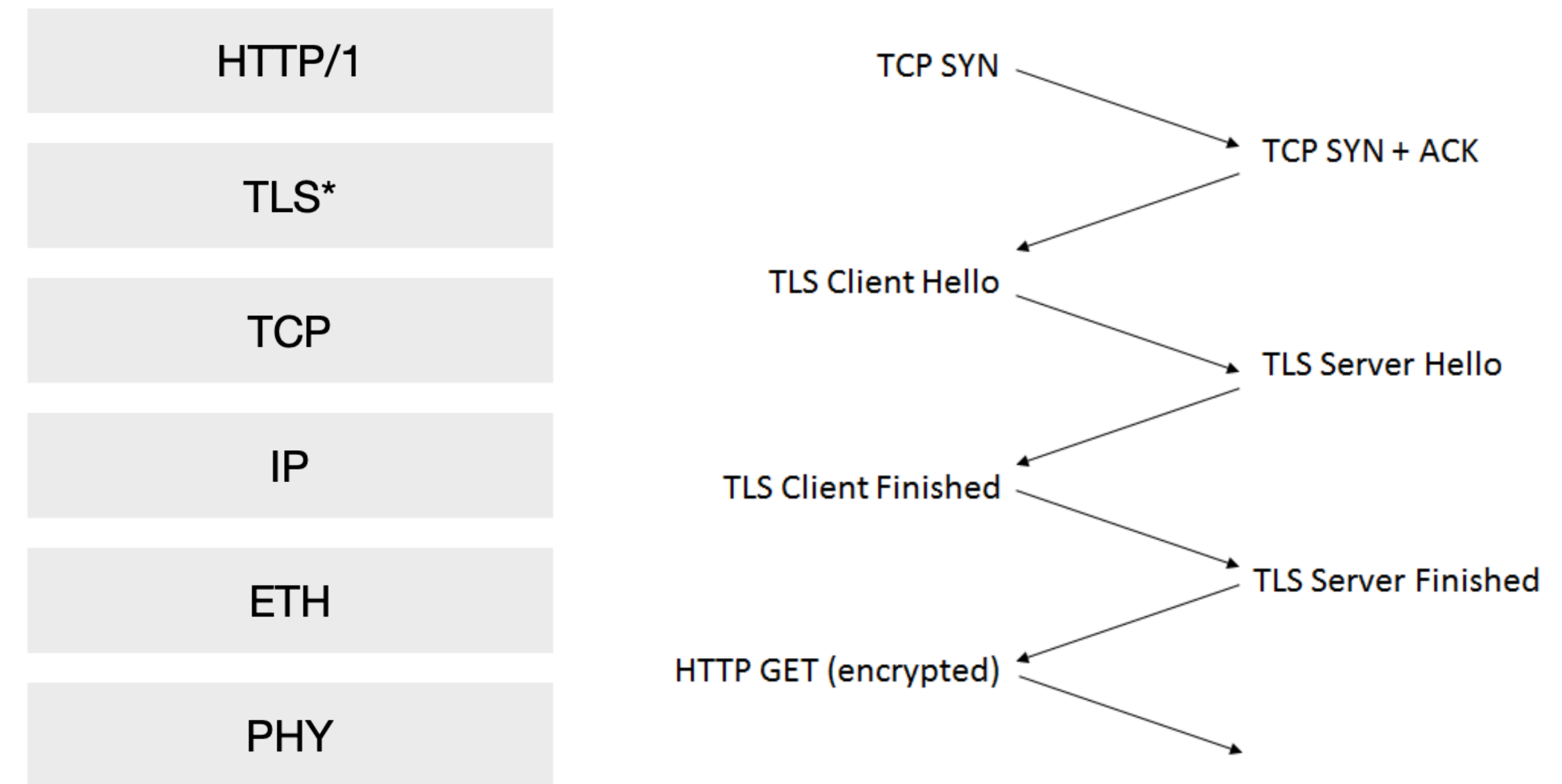
Web applications based on HTTP

- A typical HTTP webpage contains multiple objects (text, javascript, css, images, etc.)



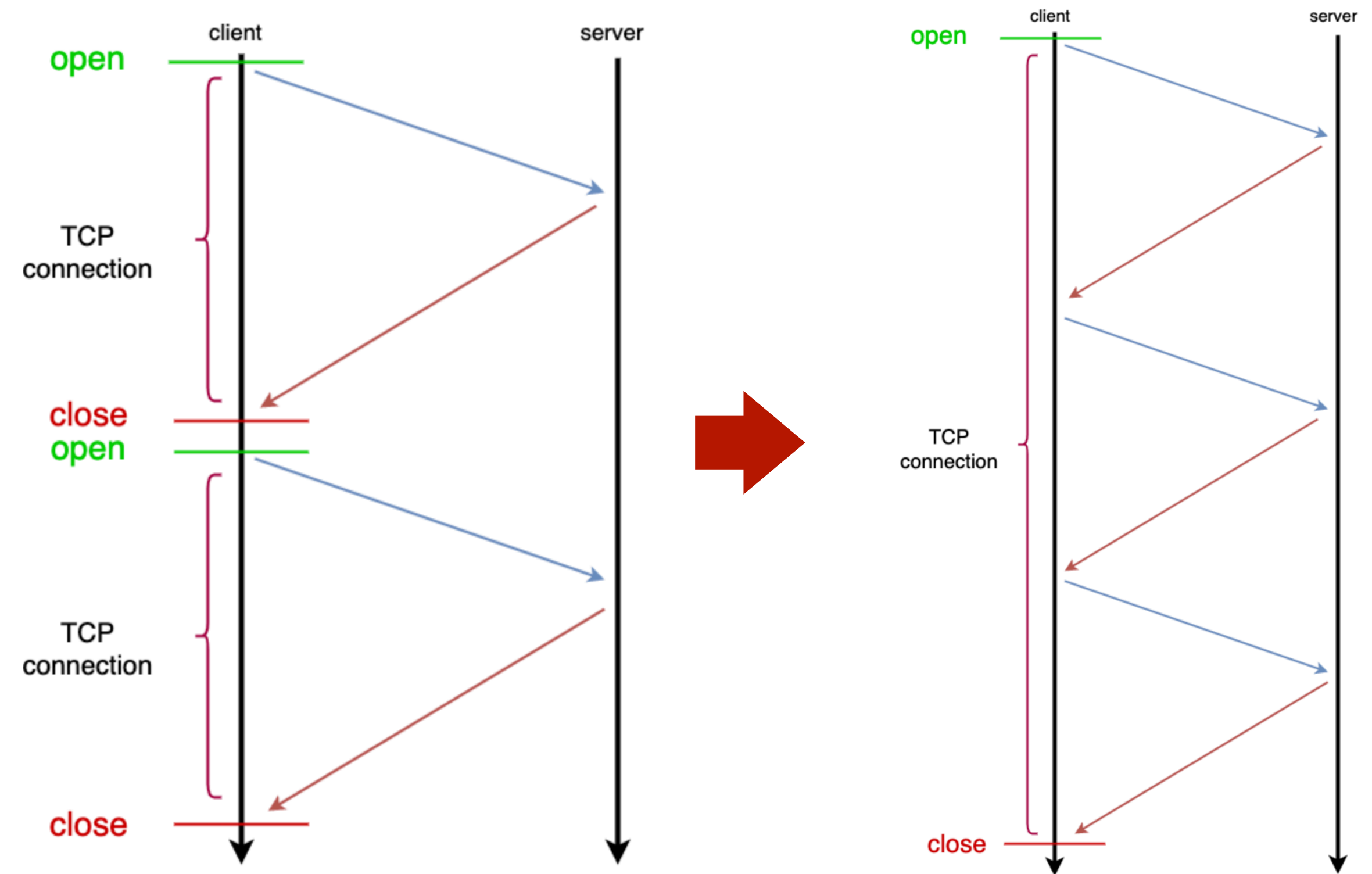
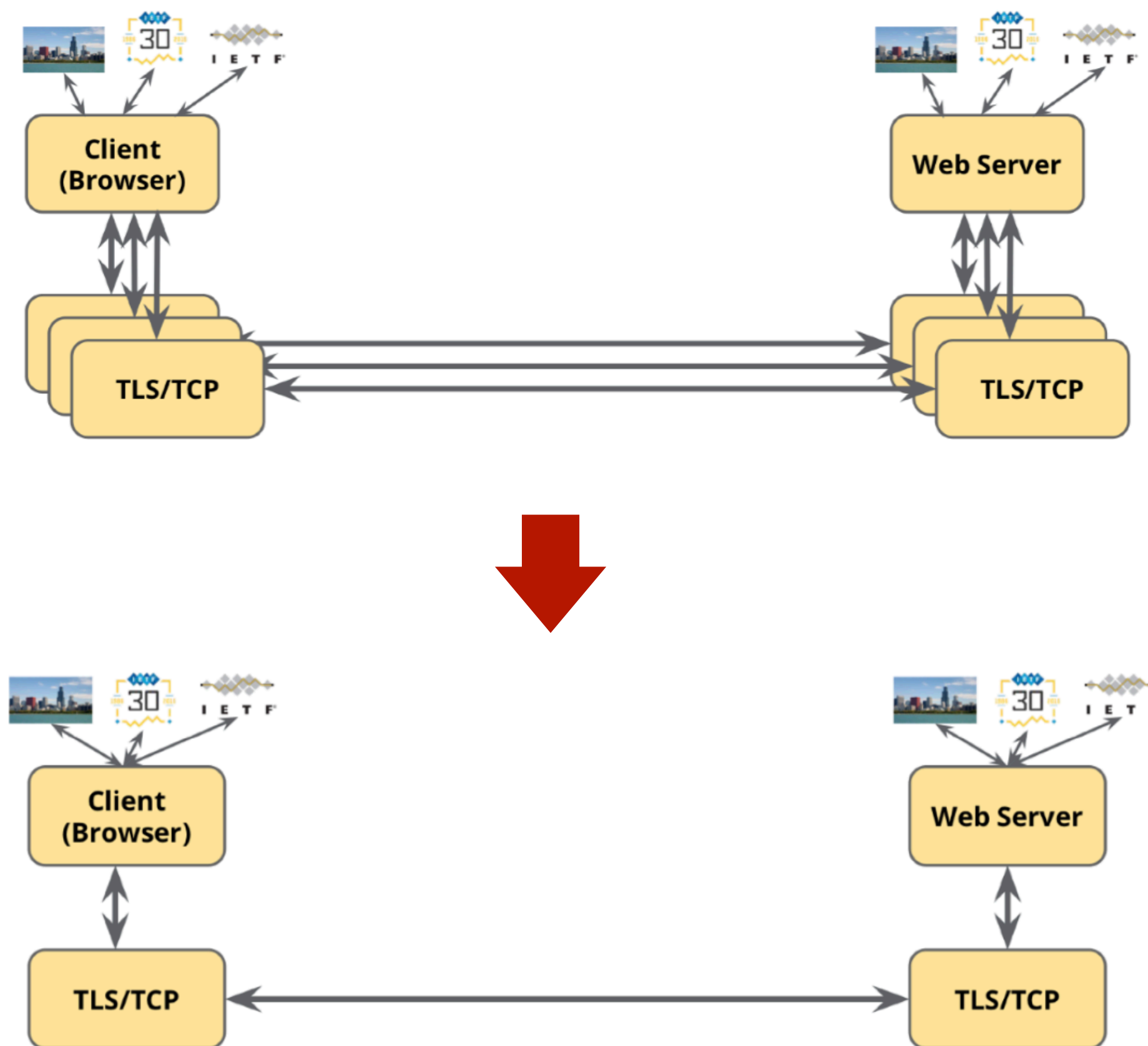
HTTP/1

- Connection setup ... the long way
 - 1 round trip to set up a TCP connection
 - 2 round trips to set up a TLS 1.2.
- After setup, HTTP requests/responses flow over the connection
 - Only one request/response is possible over a connection
→ inefficient!



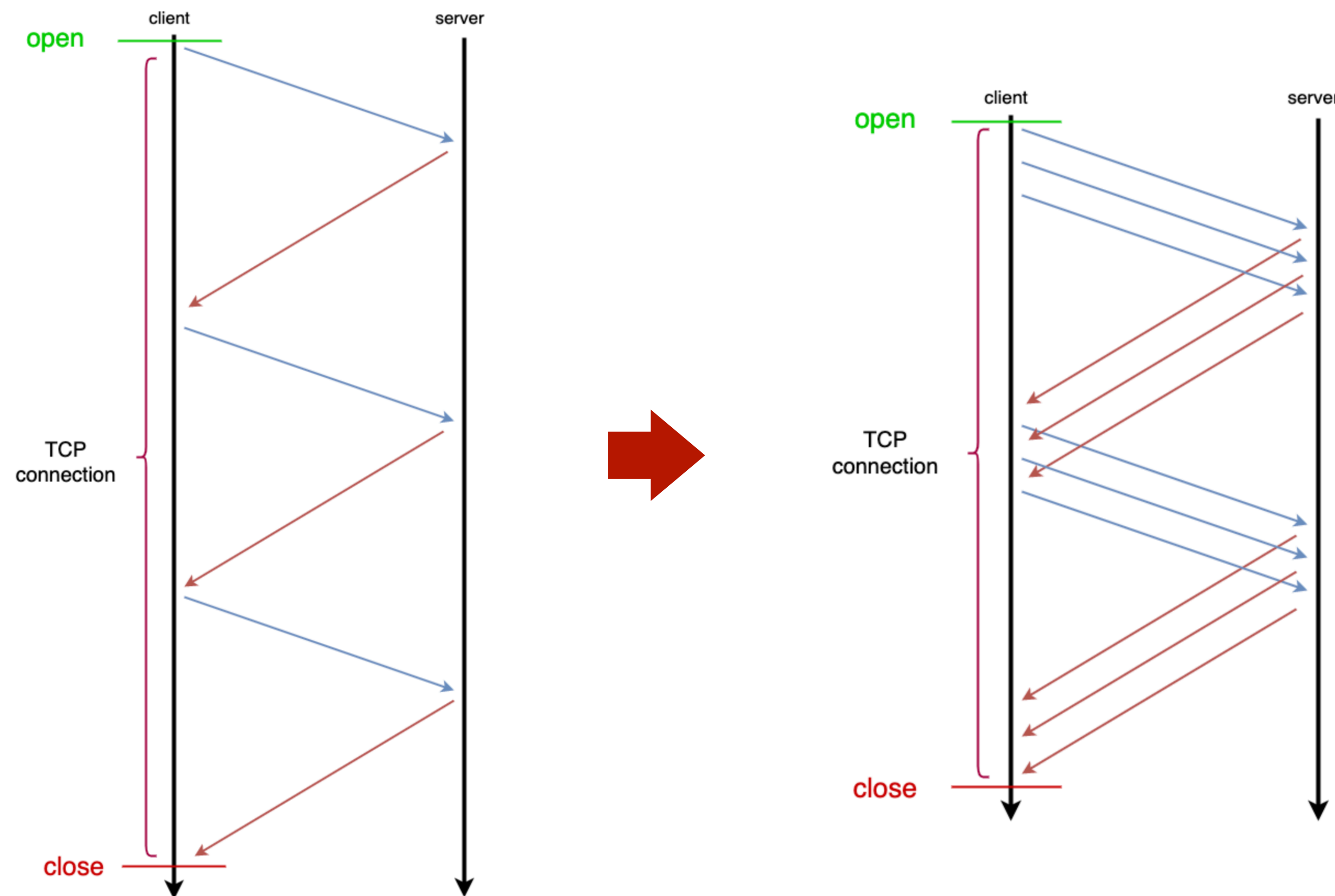
HTTP/1.1

- Persistent connection : multiple objects are delivered via a single TCP connection



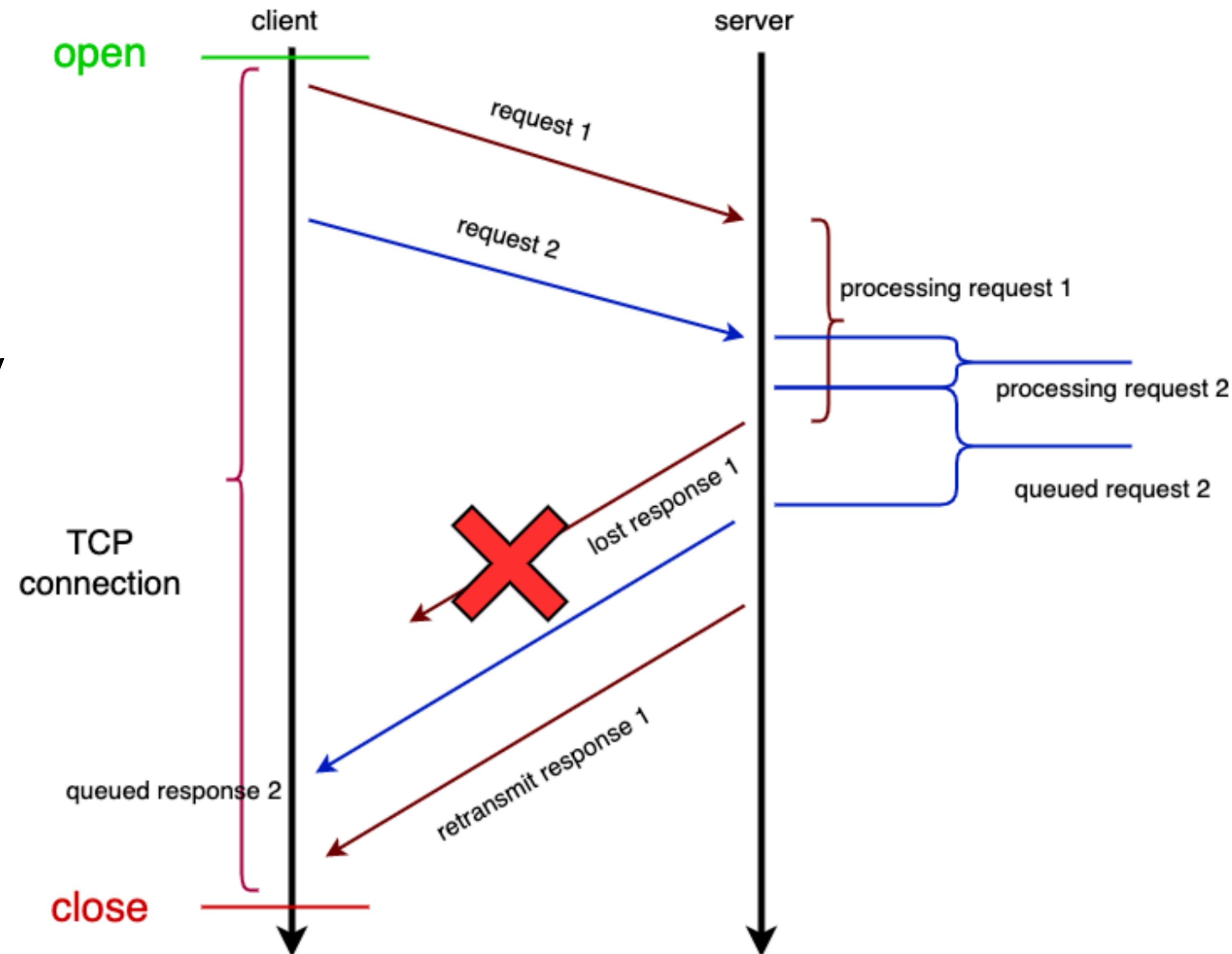
HTTP/1.1

- HTTP pipelining : multiple HTTP requests are sent over a single TCP connection without waiting for the corresponding responses



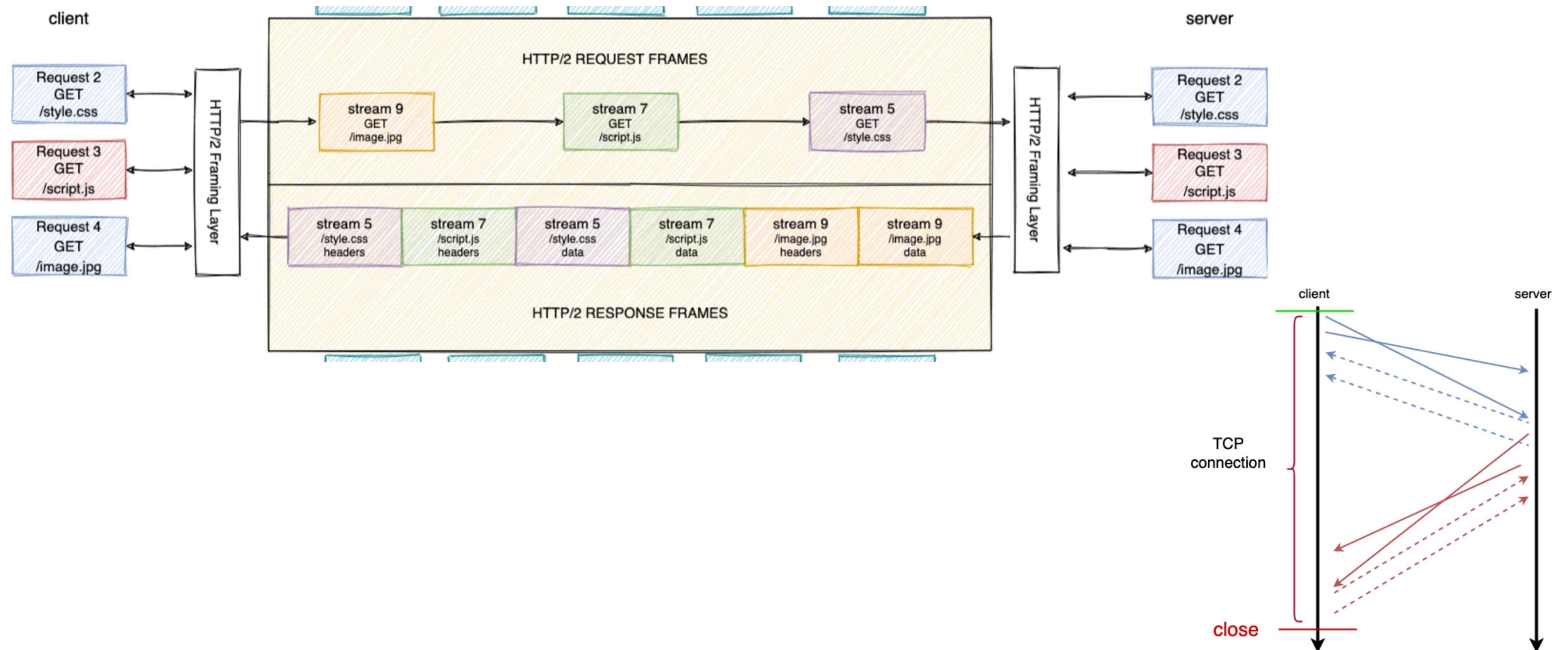
HTTP/1.1

- Head-of-Line blocking on the HTTP connection
 - The ordering of request/response is required
 - No other ways for client/server to figure out the corresponding requests/responses
- Two typical example HOLB cases
 - R1 takes longer processing time than R2
 - R1 is lost



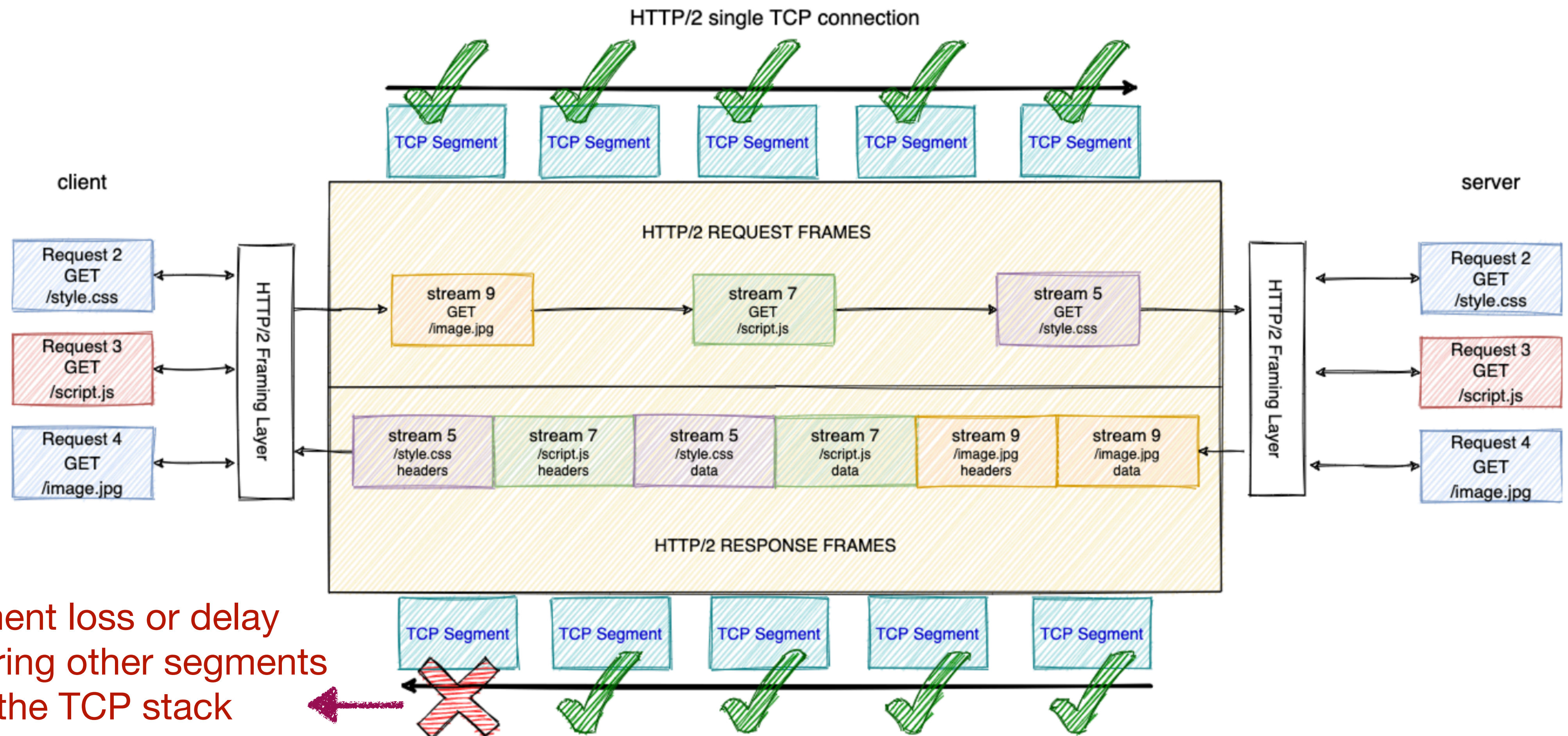
HTTP/2

- Stream multiplexing
 - Multiple streams (objects) are multiplexed on the same TCP connection



HTTP/2

- Head-of-Line blocking on the TCP connection



Head of TCP segment loss or delay would cause buffering other segments on the receiver by the TCP stack causing HOL blocking

QUIC

- A new streaming protocol to make faster web services
- Experimental protocol, developed at Google starting in 2014
 - Used in Google services and Chrome
- Improved page load time (PLT), video rebuffer rate
- More than 75% Internet traffic based on QUIC/HTTP3.0 by 2020
- Akamai deployment in 2016, Facebook deployment in 2020
- Included in the new standard of HTTP/3 in 2022



The QUIC Transport Protocol: Design and Internet-Scale Deployment

Adam Langley, Alistair Riddoch, Alyssa Wilk, Antonio Vicente, Charles Krasic, Dan Zhang, Fan Yang, Fedor Kouranov, Ian Swett, Janardhan Iyengar, Jeff Bailey, Jeremy Dorfman, Jim Roskind, Joanna Kulik, Patrik Westin, Raman Tenneti, Robbie Shade, Ryan Hamilton, Victor Vasiliev, Wan-Teh Chang, Zhongyi Shi *

Google

quic-sigcomm@google.com

ABSTRACT

We present our experience with QUIC, an encrypted, multiplexed, and low-latency transport protocol designed from the ground up to improve transport performance for HTTPS traffic and to enable rapid deployment and continued evolution of transport mechanisms. QUIC has been globally deployed at Google on thousands of servers and is used to serve traffic to a range of clients including a widely-used web browser (Chrome) and a popular mobile video streaming app (YouTube). We estimate that 7% of Internet traffic is now QUIC. We describe our motivations for developing a new transport, the principles that guided our design, the Internet-scale process that we used to perform iterative experiments on QUIC, performance improvements

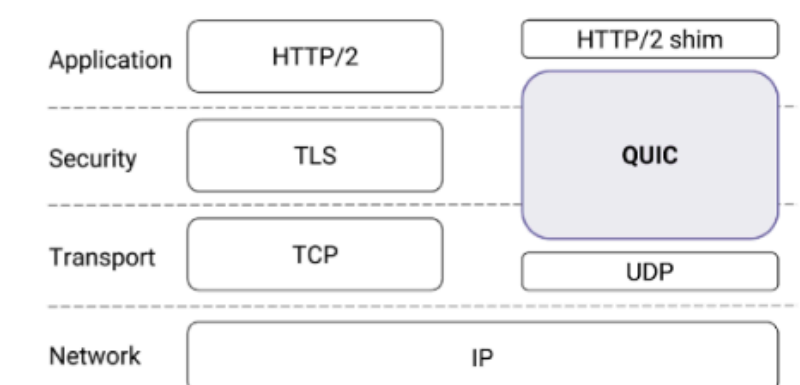
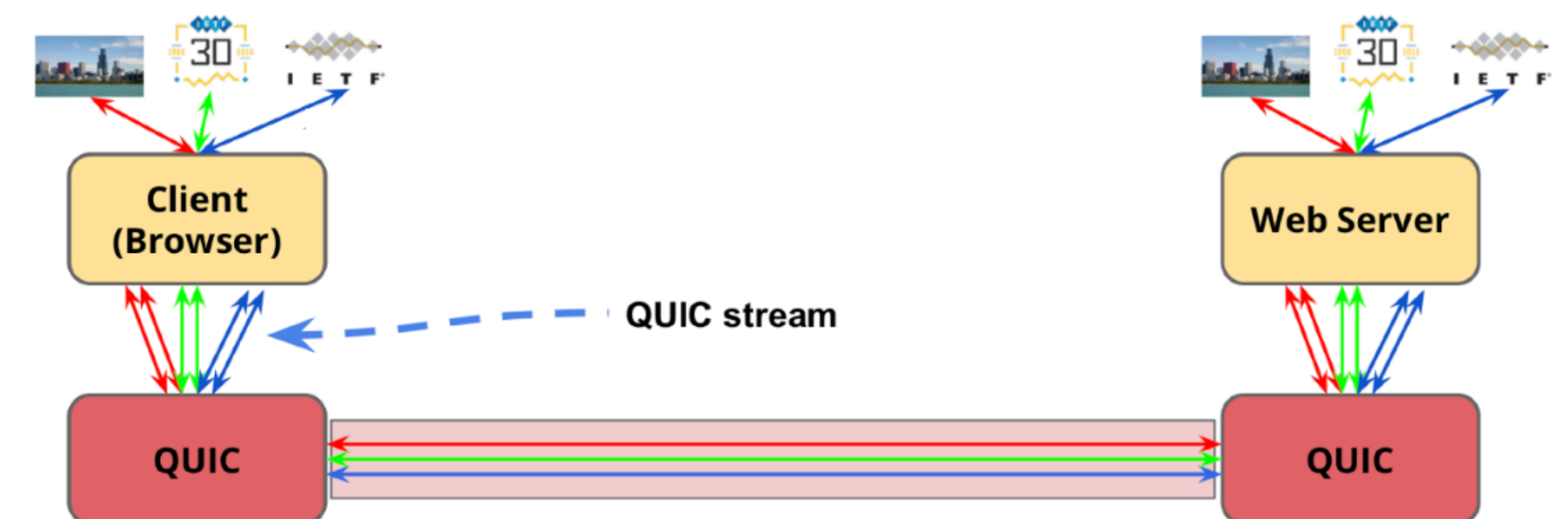
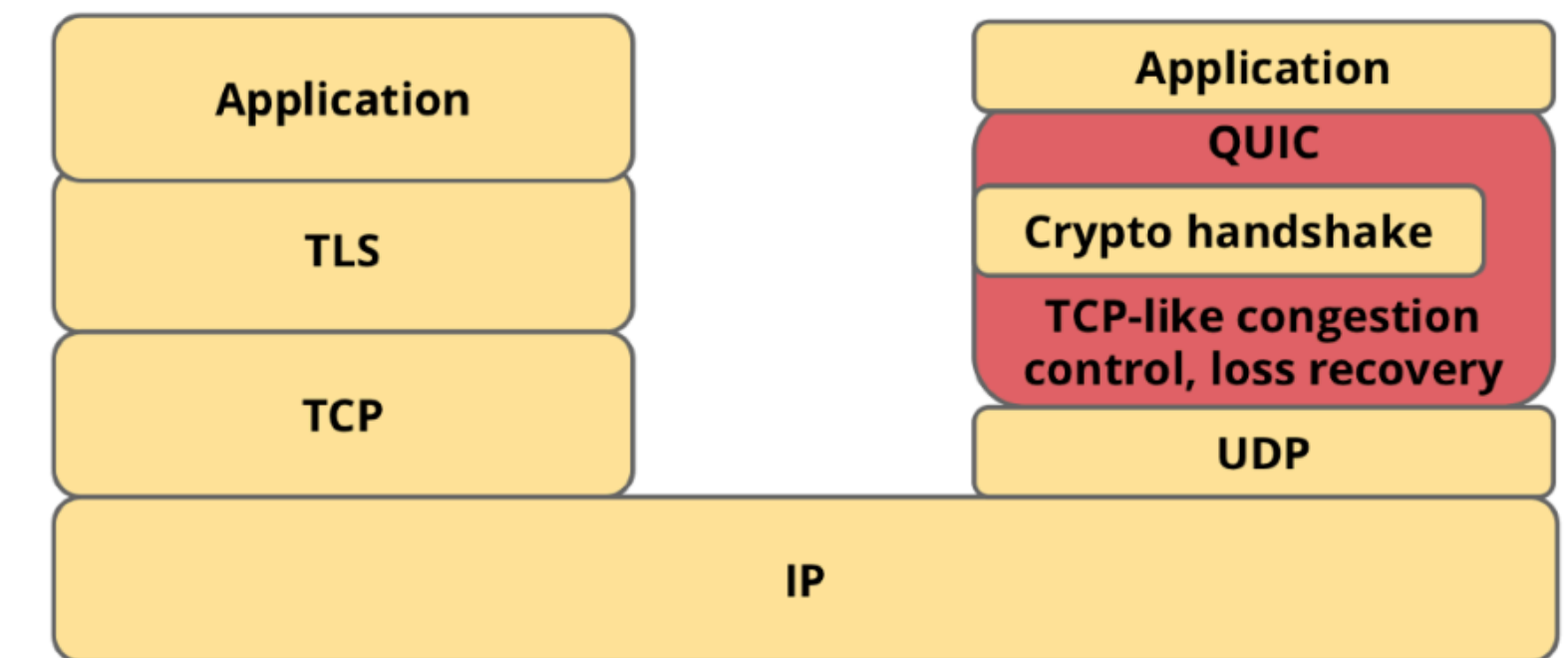


Figure 1: QUIC in the traditional HTTPS stack.

ACM SIGCOMM 2017

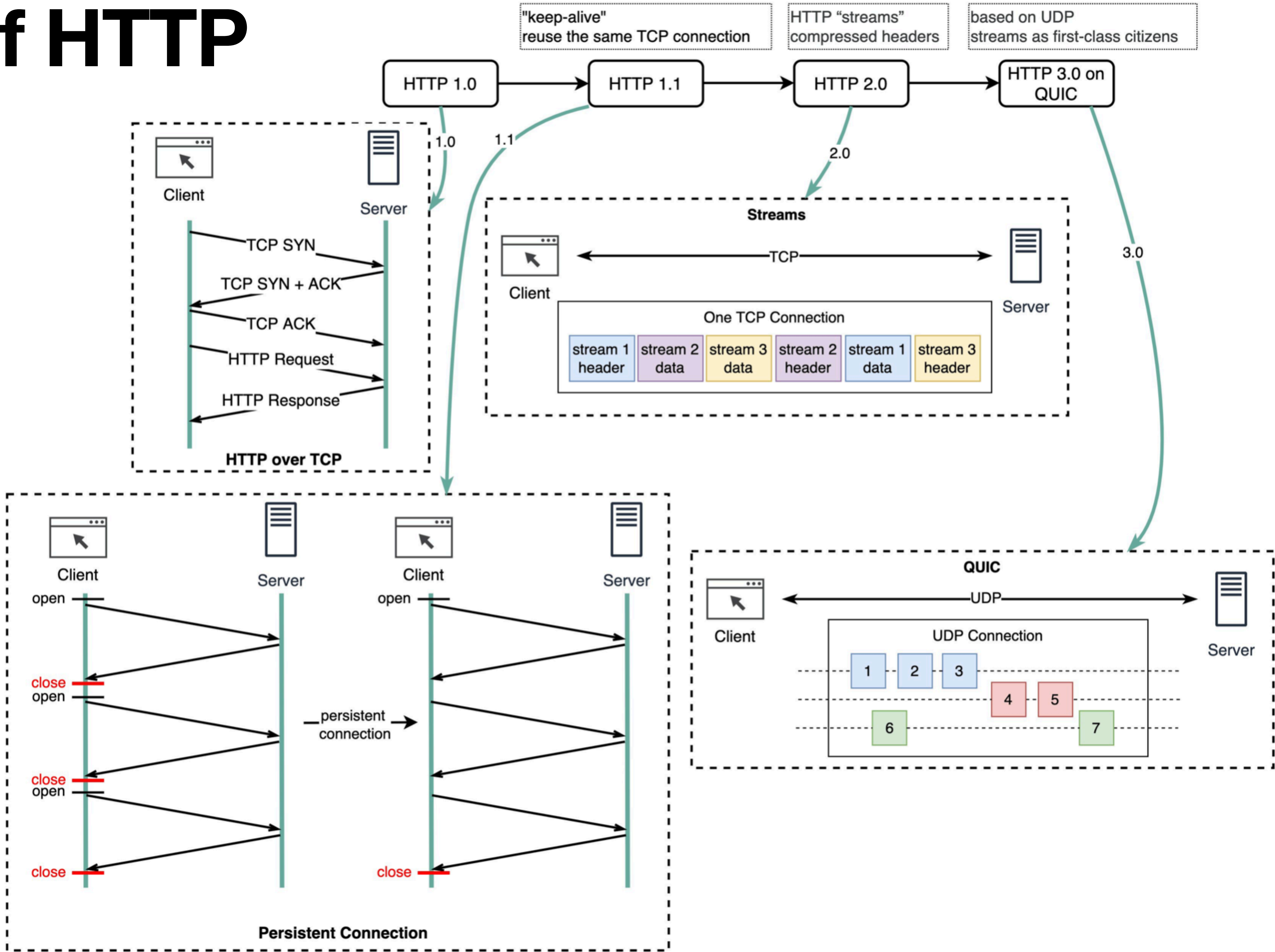
HTTP/3 over QUIC

- HTTP/2 → HTTP/3 for QUIC compatibility
- The connection setup, the QUIC way
 - 0 round-trip to a known server (common)
 - 0 round-trip if crypto-keys are not new
 - Even when IP address changes, connections survive
- After setup, HTTP requests/responses flow over the connection via QUIC streams
- User-space protocol, can adopt congestion control protocols like BBR



Avoid HoL blocking on the transport by having multiple streams over UDP

Evolution of HTTP



Summary

